



Tribhuvan University
Institute of Science and Technology
Advanced Java Programming

A lab report submission in partial fulfillment of the requirement
or practical assignment of Software Project Management of seventh semester
Of Bachelor of Computer Science And Technology

UNDER SUPERVISION OF

Mr. Dharnidhar Thakur

SUBMITTED BY:

Om Kishor Jung Shrestha(symbol No. 79011124)

Roll no: 29

SUBMITTED TO

TRINITY INTERNATIONAL COLLEGE
DILLIBAZAR HEIGHT, KATHMANDU, NEPAL

2026

S.N.	Lab Topic	Date	Signature
1	Introduction to Jira and Setting Up an Account		
2	Selecting Template and Making a Project		
3	Project Menu and Options (Team Members, Roadmap, Backlog, Board, Shortcuts)		
4	The Agile Methodology		
5	Introduction to Scrum and Sprint		
6	Epic and Roadmap – Creating Roadmap and Epic		
7	Story in Jira – Creating, Updating, Deleting; Story Points; Epic and Child Issue Linking		
8	Creating Sprint and Backlog		
9	Bug Issue in Jira		
10	Task in Jira		
11	Search (Basic and JQL)		
12	Adding New Issue Type and Label		
13	Connecting Git and Creating Dashboard		
14	Adding Field in Issue Template and Issue Workflow		
15	Enabling Features in Project		
16	Reports and Charts in Jira		
17	Sprint Burndown and Sprint Burnup Chart		
18	Cumulative Flow Diagram and Velocity Chart		
19	Cycle Time and Deployment Reports		

Table of Contents

Lab 1 – Introduction to Jira and Setting Up an Account	1
Steps Performed	1
Screenshots	1
Lab 2 – Selecting a Template and Making a Project	8
Steps Performed	8
Screenshots	8
Lab 3 – Project Menu and Options	13
3.1 – Adding Team Members	13
3.2 – Introduction to Roadmaps, Backlogs and Boards	13
3.3 – Adding Shortcuts and Project Setup	13
Screenshots	13
Lab 4 – The Agile Methodology	19
Key Agile Principles Covered	19
Agile in Jira	19
The Agile Manifesto	19
Agile Frameworks	19
Agile Ceremonies in Detail	20
Agile Artefacts	20
Agile vs Traditional (Waterfall) Development	20
Agile Metrics and Reporting in Jira	21
Lab 5 – Introduction to Scrum and Sprint	22
Steps Performed	22
Screenshots	22
Lab 6 – Epic and Roadmap (Creating Roadmap and Epic)	26
Creating an Epic	26
Using the Roadmap	26
Screenshots	26
Lab 7 – Story in Jira	30
7.1 – Creating, Updating and Deleting a Story	30
7.2 – Story Points, Linking with Epic, and Child Issues	30
Screenshots	30
Lab 8 – Creating Sprint and Backlog	35
Steps Performed	35
Screenshots	35
Lab 9 – Bug Issue in Jira	38
Steps Performed	38
Screenshots	38
Lab 10 – Task in Jira	40
Steps Performed	40
Lab 11 – Search in Jira	43
Basic Search	43
Advanced Search – JQL Examples	43
Lab 12 – Adding New Issue Type and Label	46
Adding a New Issue Type	46

Adding Labels	46
Screenshots	46
Lab 13 – Connecting Git and Creating Dashboard	51
Connecting Git (GitHub Integration)	51
Creating a Dashboard	51
Screenshots	51
Lab 14 – Adding Field in Issue Template and Issue Workflow	59
Adding a Custom Field	59
Configuring Issue Workflow	59
Lab 15 – Enabling Features in Project	65
Features Enabled	65
Steps Performed	65
Screenshots	65
Lab 16 – Reports and Charts in Jira	69
Reports Explored	69
Steps Performed	69
Lab 17 – Sprint Burndown and Sprint Burnup Chart	70
Sprint Burndown Chart	70
Sprint Burnup Chart	70
Steps Performed	70
Screenshots	70
Lab 18 – Cumulative Flow Diagram and Velocity Chart	73
Cumulative Flow Diagram (CFD)	73
Velocity Chart	73
Steps Performed	73
Screenshots	73
Lab 19 – Cycle Time and Deployment Reports	76
Cycle Time Report	76
Deployment Frequency Report	76
Steps Performed	76
Screenshots	76

List Of Figures

Figure 1 : Lab 1-Atlassian sign-up page	1
Figure 2 : Lab 1 – Email verification screen	2
Figure 3 : Lab 1 – Workspace name setup	2
Figure 4 :Lab 1 – Jira welcome/getting started page	3
Figure 5 :Lab 1 – Jira project creation prompt	3
Figure 6 :Lab 1 – Template selection screen	4
Figure 7 :Lab 1 – Scrum board overview	4
Figure 8 :Lab 1 – Project sidebar and navigation	5
Figure 9 :Lab 1 – Jira issue navigator	5
Figure 10 :Lab 1 – Jira dashboard home	6
Figure 11 :Lab 1 – Account settings page	6
Figure 12 :Lab 1 – Project settings overview	7
Figure 13 :Lab 1 – Completed Jira workspace	7
Figure 14 :Lab 2 – Create project button	8
Figure 15 :Lab 2 – Template gallery (Scrum/Kanban)	9
Figure 16 :Lab 2 – Scrum template selection	9
Figure 17 :Lab 2 – Project name and key entry form	10
Figure 18 :Lab 2 – Project creation confirmation	10
Figure 19 :Lab 2 – Project board (empty)	11
Figure 20 :Lab 2 – Backlog view after project creation	11
Figure 21 :Lab 2 – Project in left sidebar	12
Figure 22 :Lab 2 – Scrum board columns	12
Figure 23 :Lab 3 – Project Settings – Access/Members page	13
Figure 24 :Lab 3 – Add member invitation dialog	14
Figure 25 :Lab 3 – Roadmap view	14
Figure 26 :Lab 3 – Backlog view with issues	15
Figure 27 :Lab 3 – Board view (Scrum columns)	15
Figure 28 :Lab 3 – Add shortcut dialog	16
Figure 29 :Lab 3 – Shortcut added to sidebar	16
Figure 30 :Lab 3 – Project setup configuration	17
Figure 31 :Lab 3 – Issue types list	17
Figure 32 :Lab 3 – Workflow scheme	18
Figure 33 :Lab 3 – Component configuration	18
Figure 34 :Lab 5 – Backlog with sprint container	22
Figure 35 :Lab 5 – Create sprint dialog	23
Figure 36 :Lab 5 – Issues dragged into sprint	23
Figure 37 :Lab 5 – Start sprint confirmation	24
Figure 38 :Lab 5 – Active sprint on Scrum board	24
Figure 39 :Lab 5 – Sprint countdown timer	25
Figure 40 :Lab 5 – Sprint board with issues in columns	25
Figure 41 :Lab 6 – Roadmap timeline view	26
Figure 42 :Lab 6 – Create Epic dialog	27
Figure 43 :Lab 6 – Epic colour selection	27
Figure 44 :Lab 6 – Epic in Epic panel	28
Figure 45 :Lab 6 – Roadmap with Epic bar	28
Figure 46 :Lab 6 – Epic date range adjustment	29
Figure 47 :Lab 7 – Create story form	30
Figure 48 :Lab 7 – Story with summary and description	31
Figure 49 :Lab 7 – Story assigned and prioritised	31

Figure 50 :Lab 7 – Story on Scrum board	32
Figure 51 :Lab 7 – Update story – status transition	32
Figure 52 :Lab 7 – Delete story confirmation	33
Figure 53 :Lab 7 – Story points field	33
Figure 54 :Lab 7 – Epic link field on story	34
Figure 55 :Lab 7 – Child issues section on story	34
Figure 56 :Lab 8 – Backlog re-prioritisation	35
Figure 57 :Lab 8 – New sprint container in backlog	36
Figure 58 :Lab 8 – Sprint goal and dates dialog	36
Figure 59 :Lab 8 – Issues assigned to sprint	37
Figure 60 :Lab 8 – Active sprint badge on backlog	37
Figure 61 :Lab 9 – Create bug form	38
Figure 62 :Lab 9 – Bug with priority and assignee	39
Figure 63 :Lab 12 – Issue types configuration page	41
Figure 64 :Lab 12 – Add issue type dialog	41
Figure 65 :Lab 12 – New issue type in create dropdown	41
Figure 66 :Lab 12 – Labels field on issue	42
Figure 67 :Lab 12 – Label creation	43
Figure 68 :Lab 12 – Label filter in search	43
Figure 69 :Lab 12 – Issues filtered by label	44
Figure 70 :Lab 13 – GitHub for Jira marketplace app	44
Figure 71 :Lab 13 – GitHub authorisation screen	45
Figure 72 :Lab 12 – issues	46
Figure 73 :Lab 12 – Development panel on Jira issue	46
Figure 74 :Lab 12 – issues linked	47
Figure 75 :Lab 12 – issues cration	47
Figure 76 :Lab 12 –issue creaton	48
Figure 77 :Lab 12 – form filled for issue	48
Figure 78 :Lab 12 – issue created well	49
Figure 79 :lab 12 issue seen	50
Figure 80 :Lab 13 – Assigned to Me gadget	51
Figure 81 :Lab 13 – Sprint Health gadget	51
Figure 82 :Lab 13 – Activity Stream gadget	52
Figure 83 :Lab 13 – Issue Statistics pie chart	52
Figure 84 :Lab 13 – Dashboard layout view	53
Figure 85 :Lab 13 – Dashboard – full populated view	53
Figure 86 :Lab 13 – Dashboard set as home page	54
Figure 87 :Lab 13 – Issue Statistics by priority	54
Figure 88 :Lab 13 – GitHub commit log in Jira	55
Figure 89 :Lab 13 – Branch list on issue	55
Figure 90 :Lab 13 – Dashboard gadget arrangement	56
Figure 91 :Lab 13 – Final dashboard overview	56
Figure 92 :Lab 13 – Dashboard sharing settings	57
Figure 93 :Lab 15 – Project Settings – Features panel	57
Figure 94 :Lab 15 – Features toggles enabled	58
Figure 95 :Lab 15 – Releases and Components in sidebar	59
Figure 96 :Lab 17 – Burndown chart – ideal vs actual	60
Figure 97 :Lab 17 – Burndown chart – sprint details	61
Figure 98 :Lab 17 – Burnup chart – scope line	62
Figure 99 :Lab 17 – Burnup chart – completed vs total	62

Figure 100 :Lab 18 – Cumulative Flow Diagram	62
Figure 101 :Lab 18 – CFD – bottleneck identified	63
Figure 102 field created well	64
Figure 103 :Lab 15 – Project Settings – Features panel	65
Figure 104 :Lab 15 – Features toggles enabled	66
Figure 105 :Lab 15 – Releases and Components in sidebar	66
Figure 106 :Lab 15 feautres panel	66
Figure 107 :Lab 15 fearutes section	67
Figure 108 :Lab 15 details	67
Figure 109 : Lab15 end	68
Figure 110 : lab 16 report section in jira	69
Figure 111 : Lab 17 -charts	70
Figure 112 lab 17-burnup chart	71
Figure 113 : lab 17-burndown chart	71
Figure 114 Lab 17- final chart	72
Figure 115 :lab 18-cumulative flow diagram	73
Figure 116 : lab 18-velocity report 1	74
Figure 117 : lab 18-velocity report 2	74
Figure 118 : Lab 18-cycle time	75
Figure 119 : lab 19-deployment frequency	76

Lab 1 – Introduction to Jira and Setting Up an Account

Objective: To understand what Jira is, its role in software project management, and to create a personal Atlassian account to access Jira Software.

Jira is a project tracking and issue management tool developed by Atlassian. It is widely used in Agile software development for managing tasks, bugs, stories, epics, and sprints. The cloud-based version is accessible at atlassian.net.

Steps Performed

1. Opened a web browser and navigated to <https://www.atlassian.com/software/jira>.
2. Clicked "Try it free" to begin the free-tier account setup process.
3. Entered a valid email address and clicked "Agree – Sign up".
4. Checked the inbox and verified the account using the link sent by Atlassian.
5. Created a workspace/site name: `omkishorjungshrestha-79011124.atlassian.net`.
6. Selected "Scrum" as the project type and proceeded to the Jira dashboard.
7. Explored the main Jira interface including the left sidebar, project board, and settings menu.

Screenshots

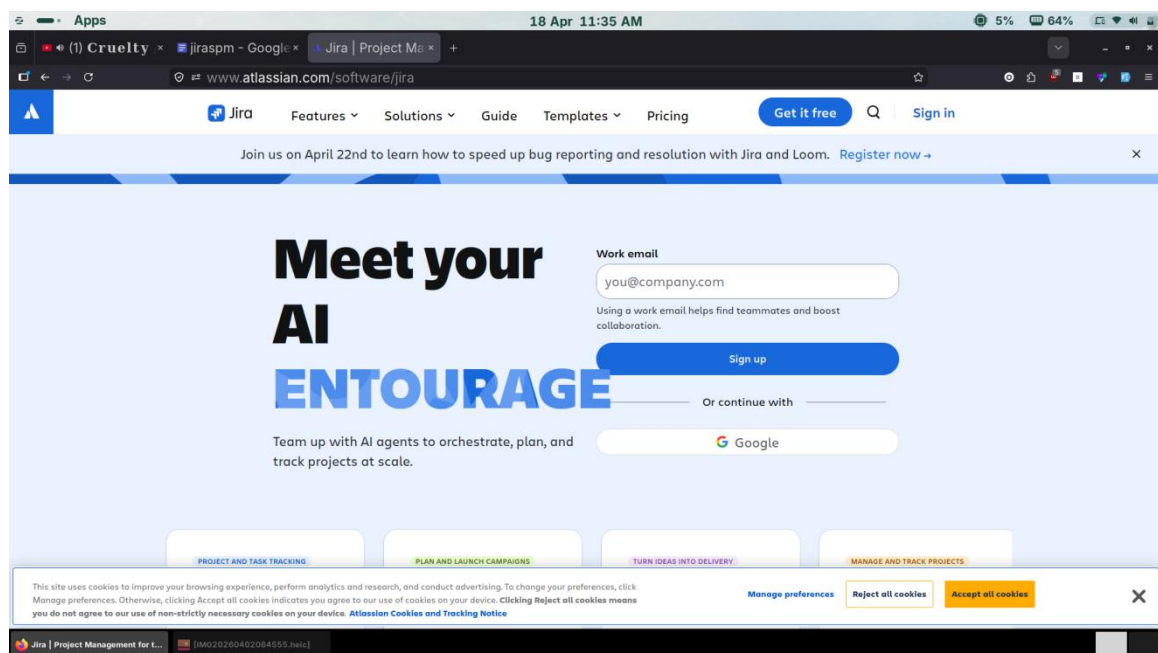


Figure 1: Lab 1-Atlassian sign-up page

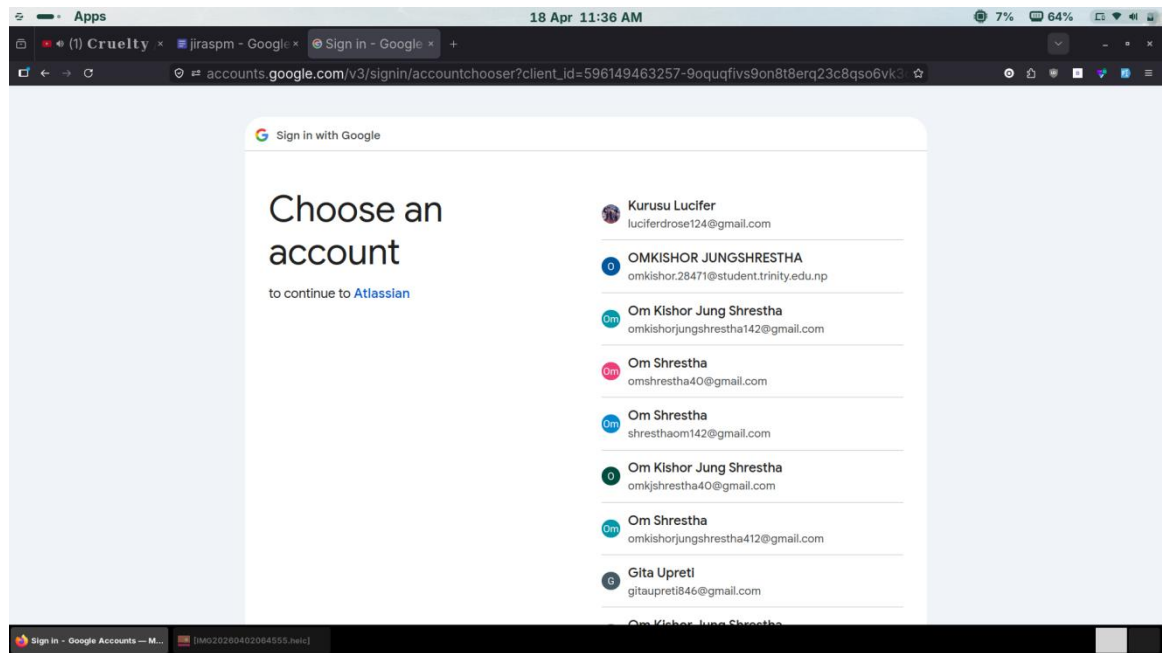


Figure 2: Lab 1 – Email verification screen

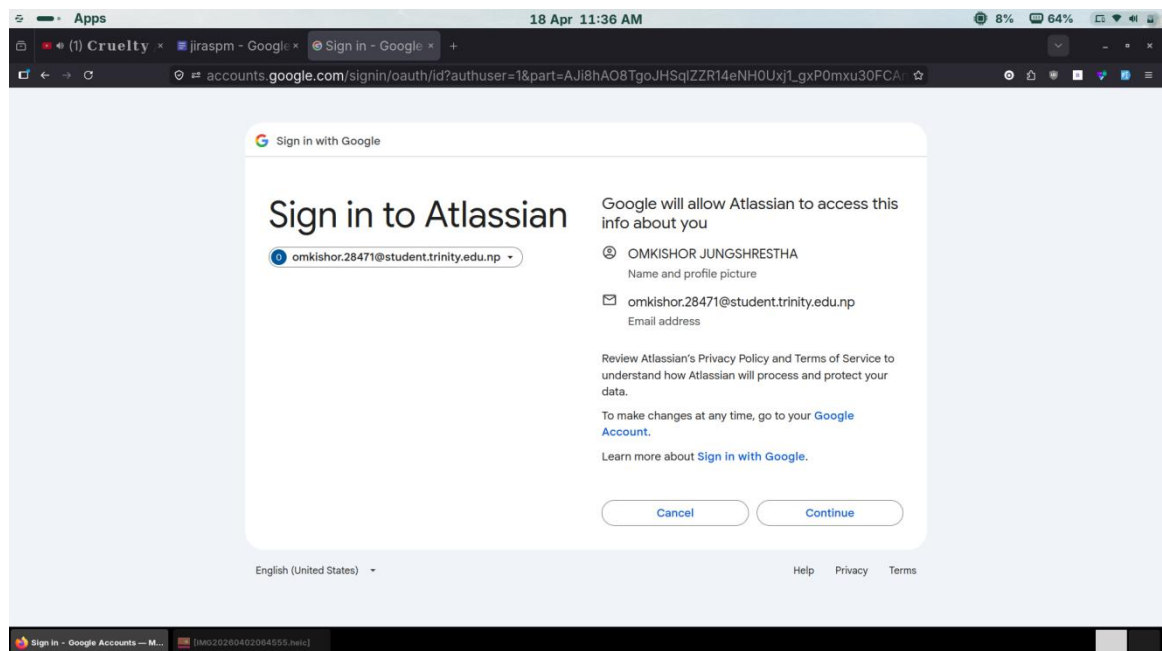


Figure 3: Lab 1 – Workspace name setup

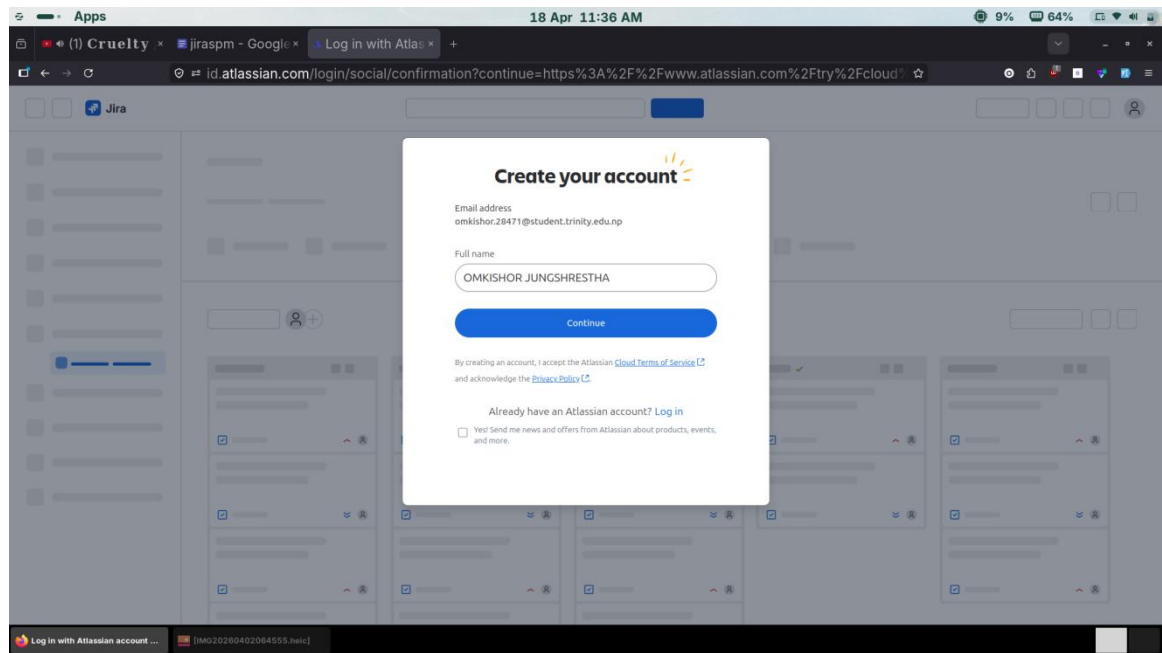


Figure 4:Lab 1 – Jira welcome/getting started page

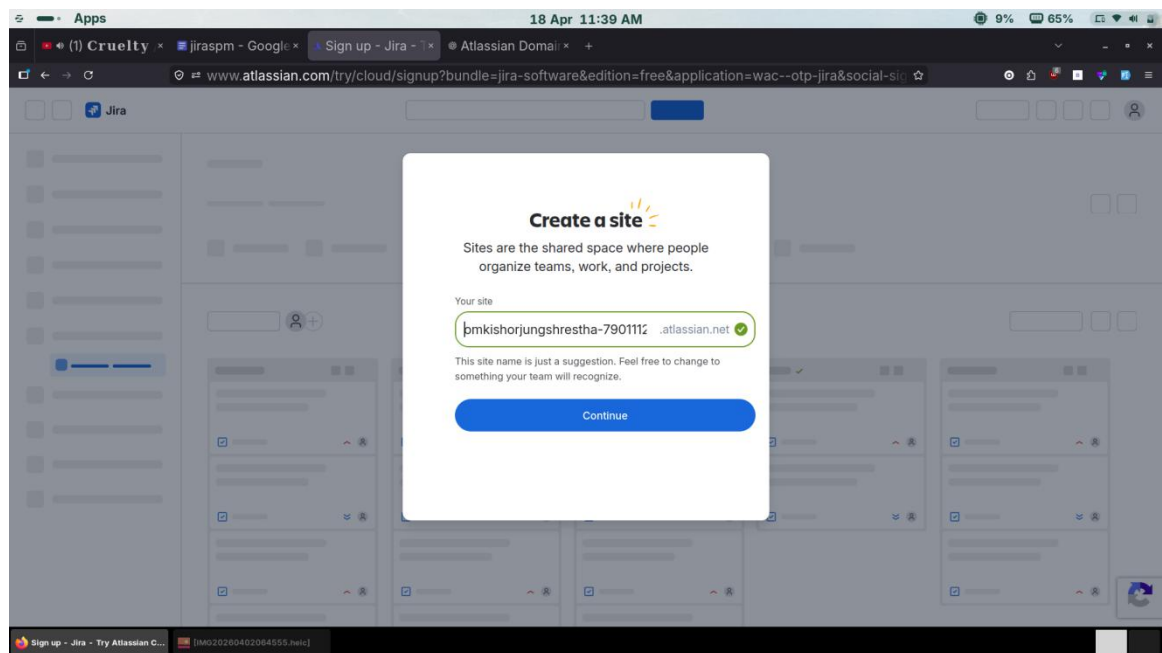


Figure 5:Lab 1 – Jira project creation prompt

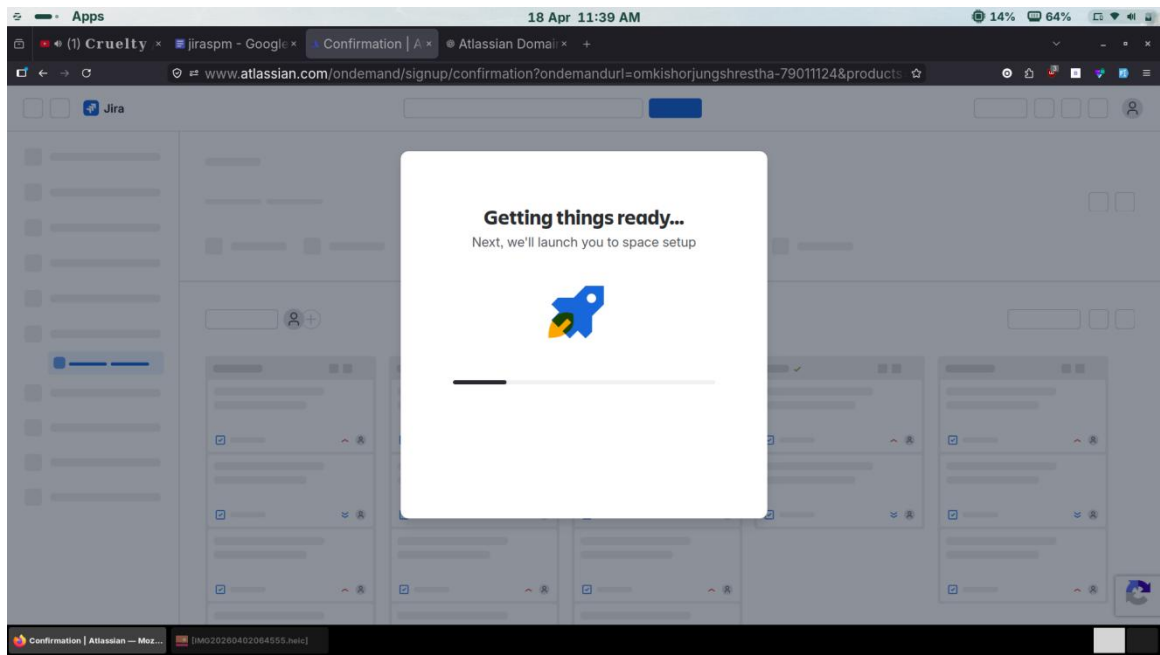


Figure 6:Lab 1 – Template selection screen

Figure 6: Lab 1 – Screenshot 6

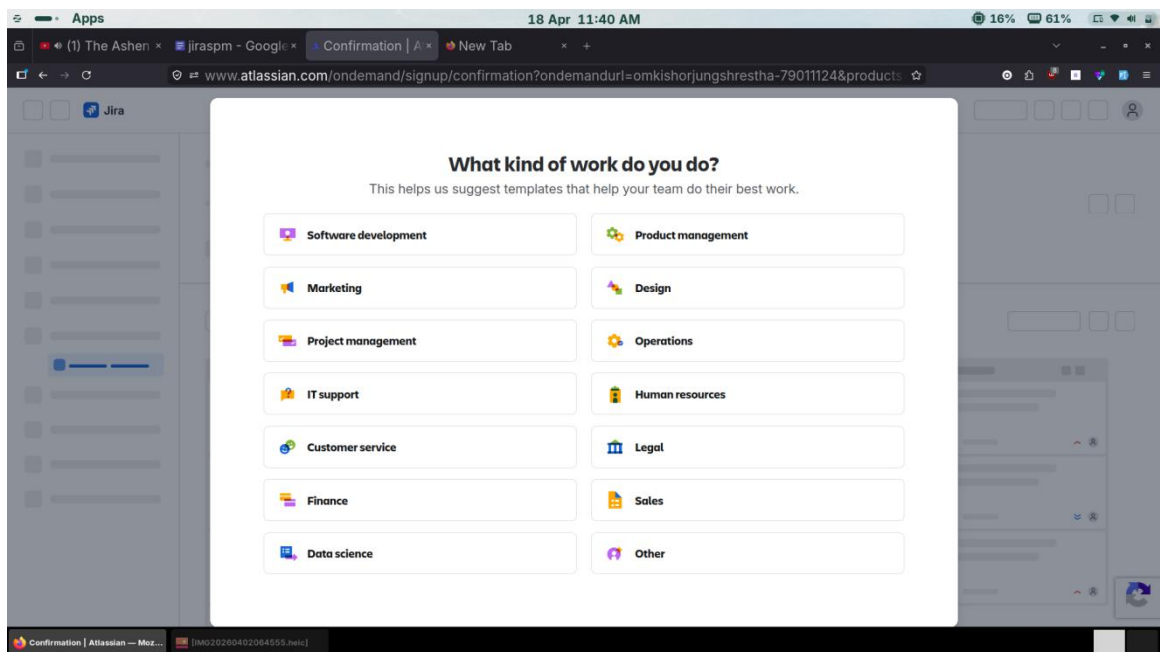


Figure 7:Lab 1 – Scrum board overview

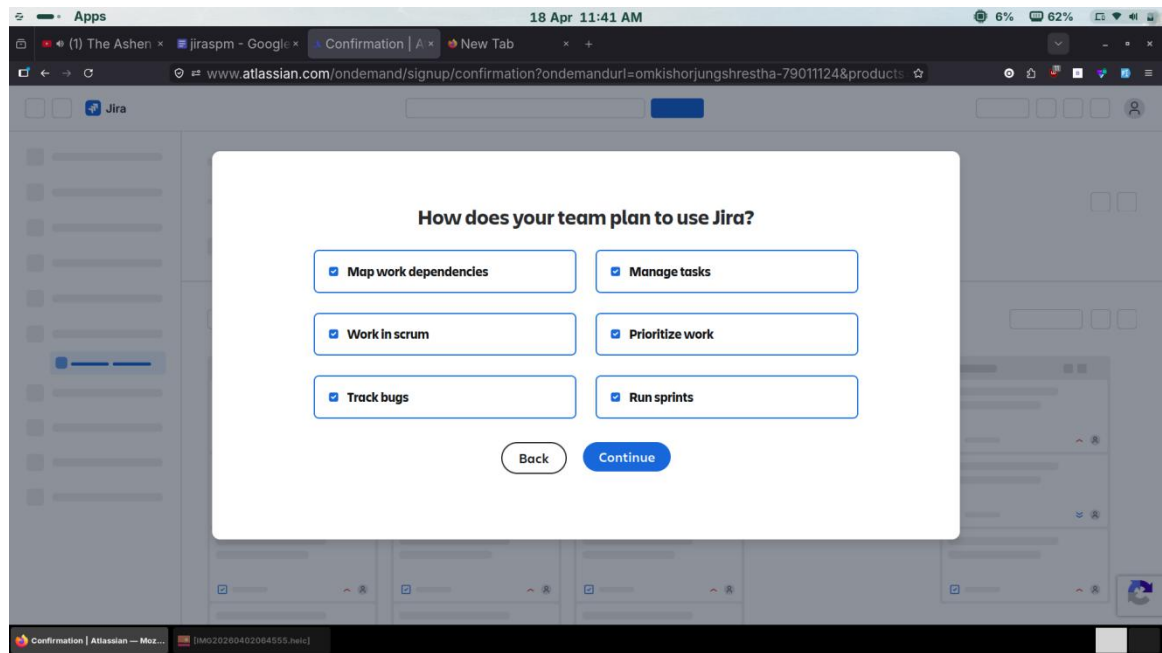


Figure 8:Lab 1 – Project sidebar and navigation

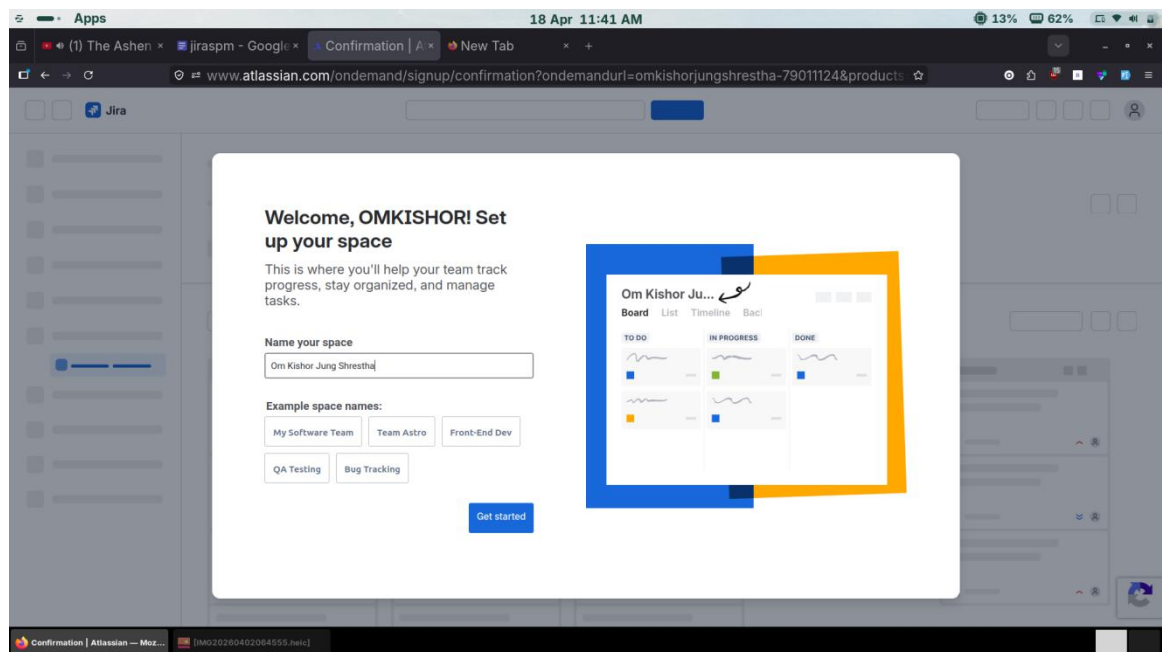


Figure 9:Lab 1 – Jira issue navigator

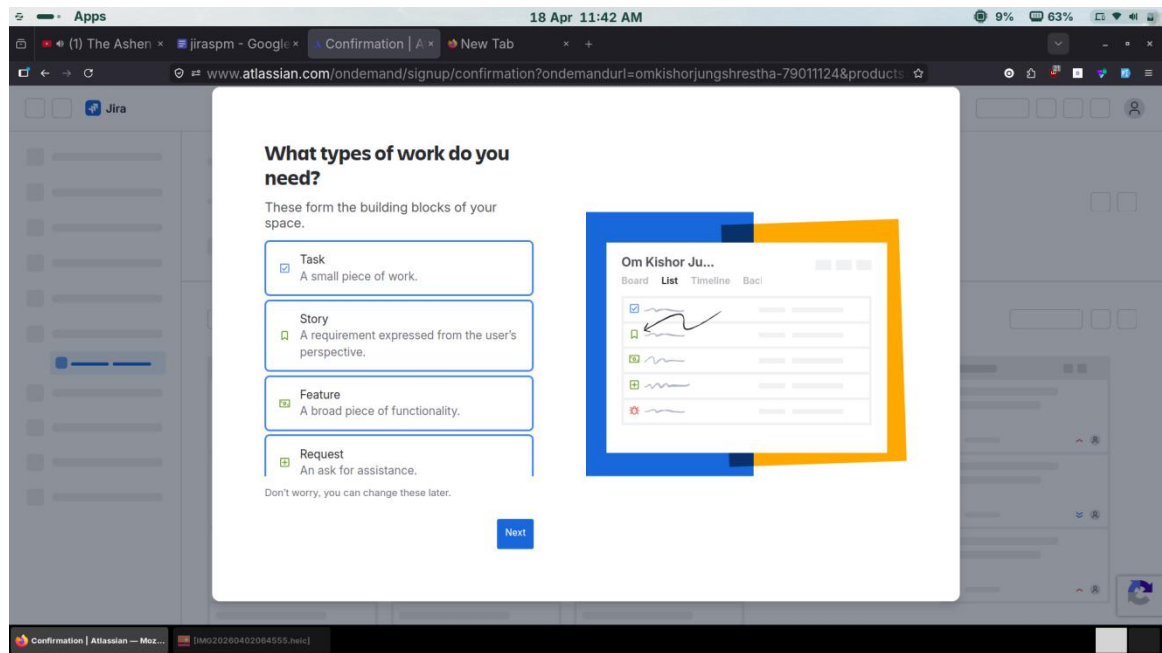


Figure 10:Lab 1 – Jira dashboard home

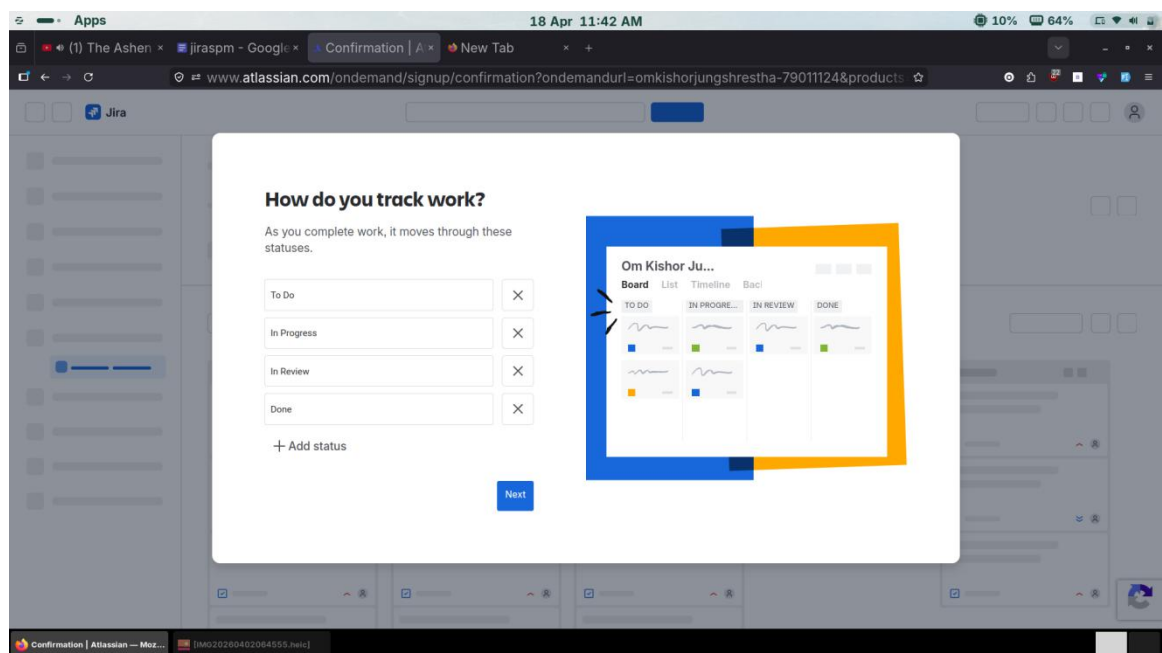


Figure 11:Lab 1 – Account settings page

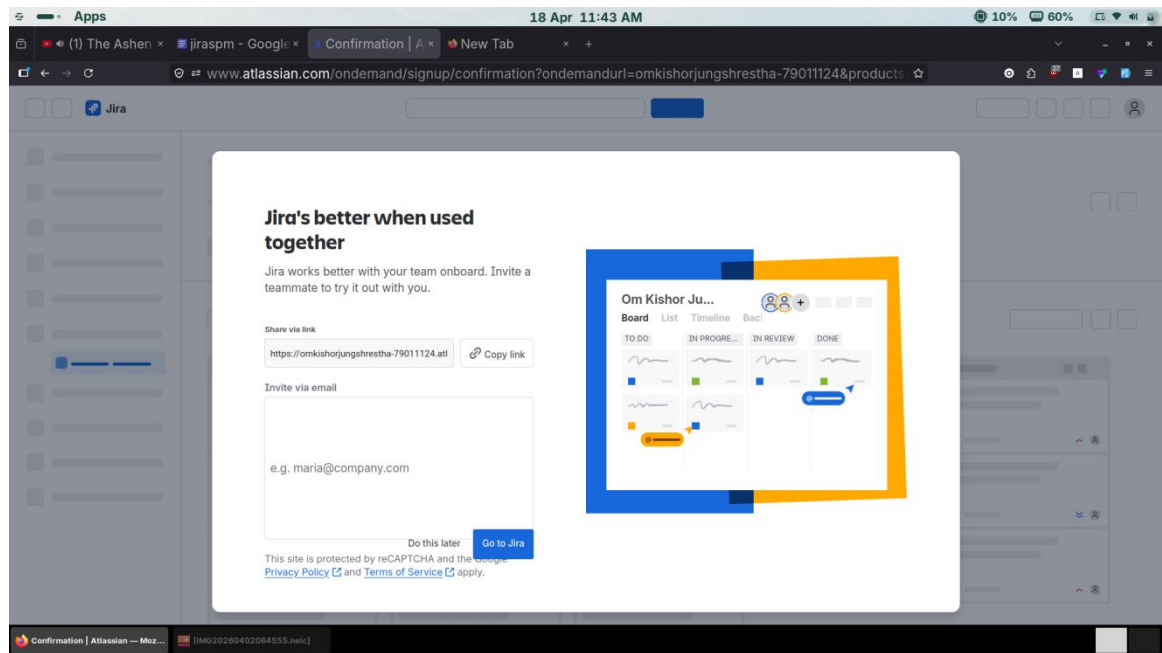


Figure 12:Lab 1 – Project settings overview

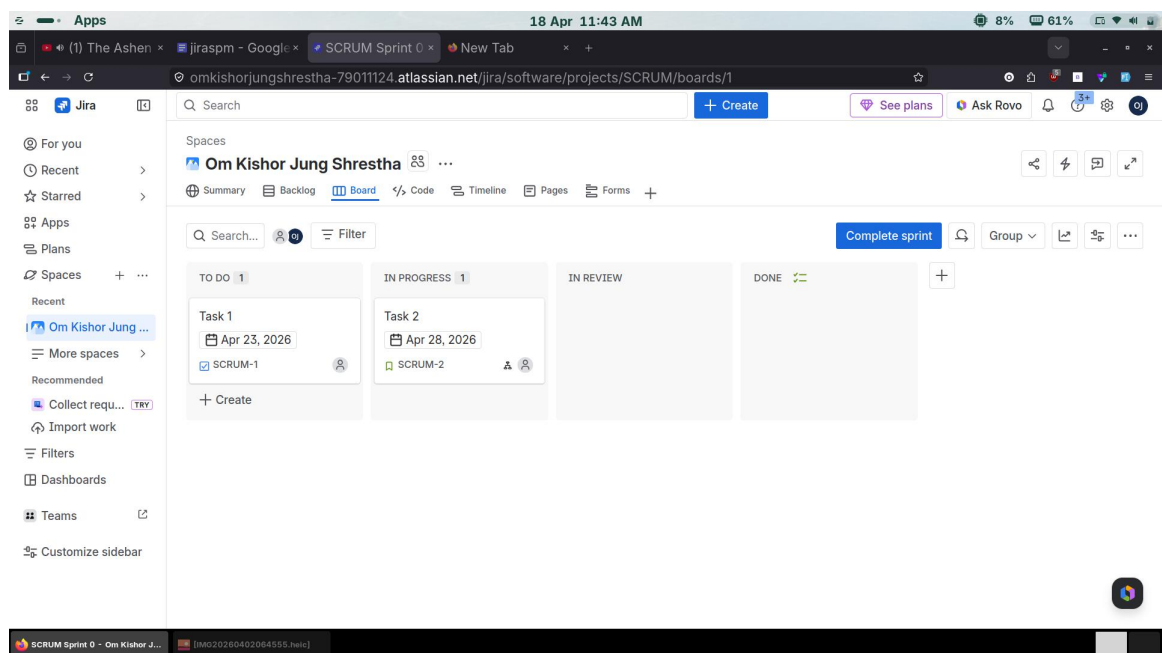


Figure 13:Lab 1 – Completed Jira workspace

Conclusion: An Atlassian account was successfully created and the Jira Software dashboard was accessed. The workspace is live at <https://omkishorjungshrestha-79011124.atlassian.net>.

Lab 2 – Selecting a Template and Making a Project

Objective: To learn how to create a new project in Jira by selecting an appropriate project template.

Jira provides various project templates such as Scrum, Kanban, and Business project. Each template comes pre-configured with a board, backlog, and workflow suited to the selected methodology.

Steps Performed

1. From the Jira dashboard, clicked "Create project" in the left navigation panel.
2. Browsed the available templates: Scrum, Kanban, Bug Tracking, and more.
3. Selected the "Scrum" template for sprint-based project management.
4. Entered the project name, auto-generated project key, and access level (Private).
5. Clicked "Create" to generate the project with a default backlog and sprint board.
6. Verified the project appeared in the left sidebar under "Projects".

Screenshots

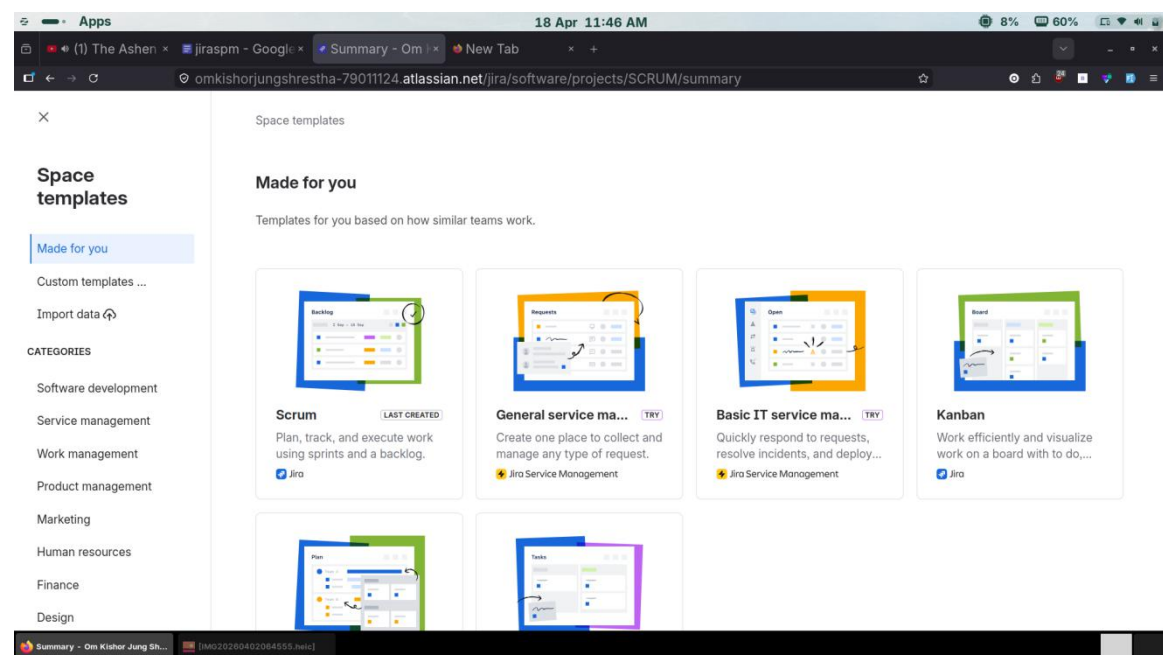


Figure 14:Lab 2 – Create project button

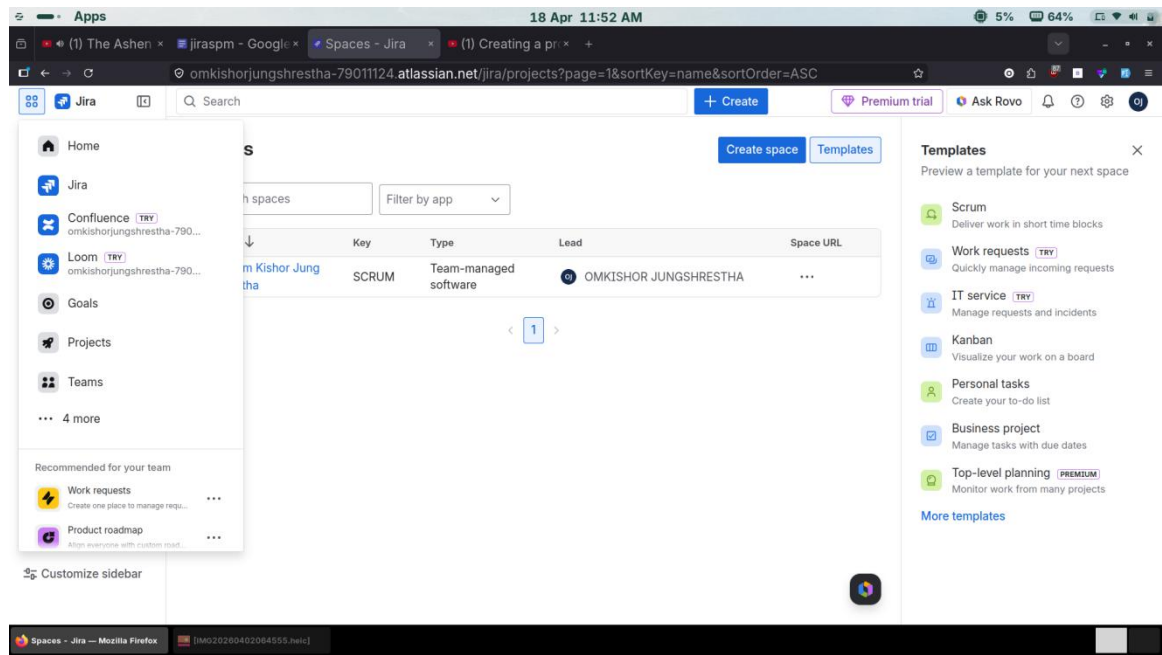


Figure 15:Lab 2 – Template gallery (Scrum/Kanban)

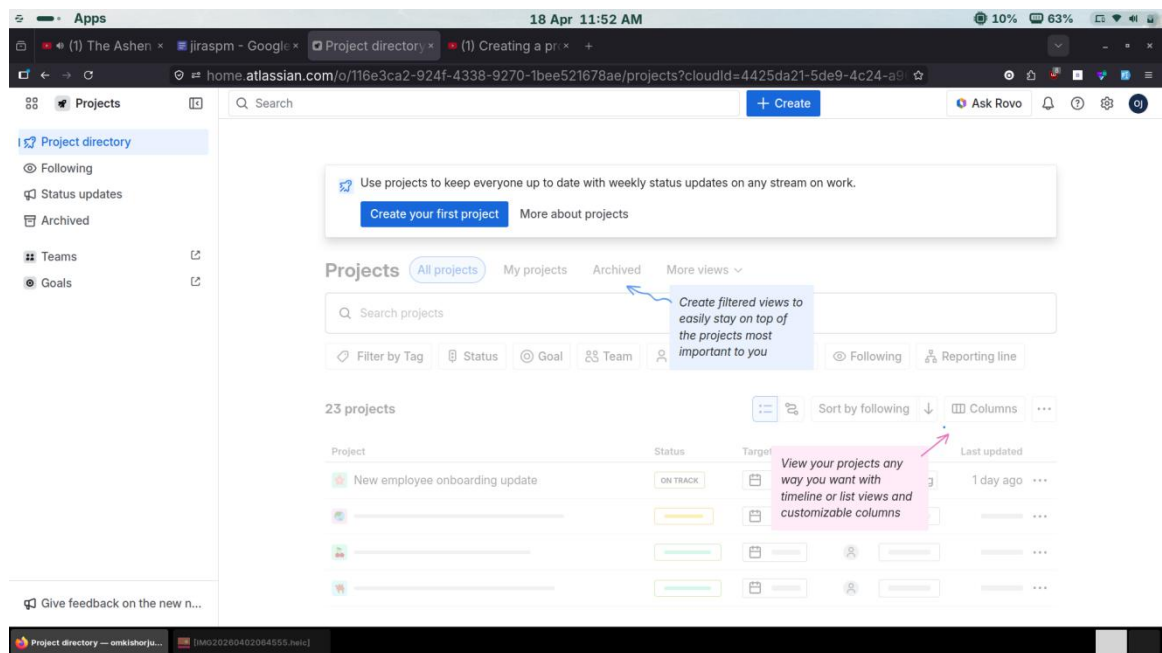


Figure 16:Lab 2 – Scrum template selection

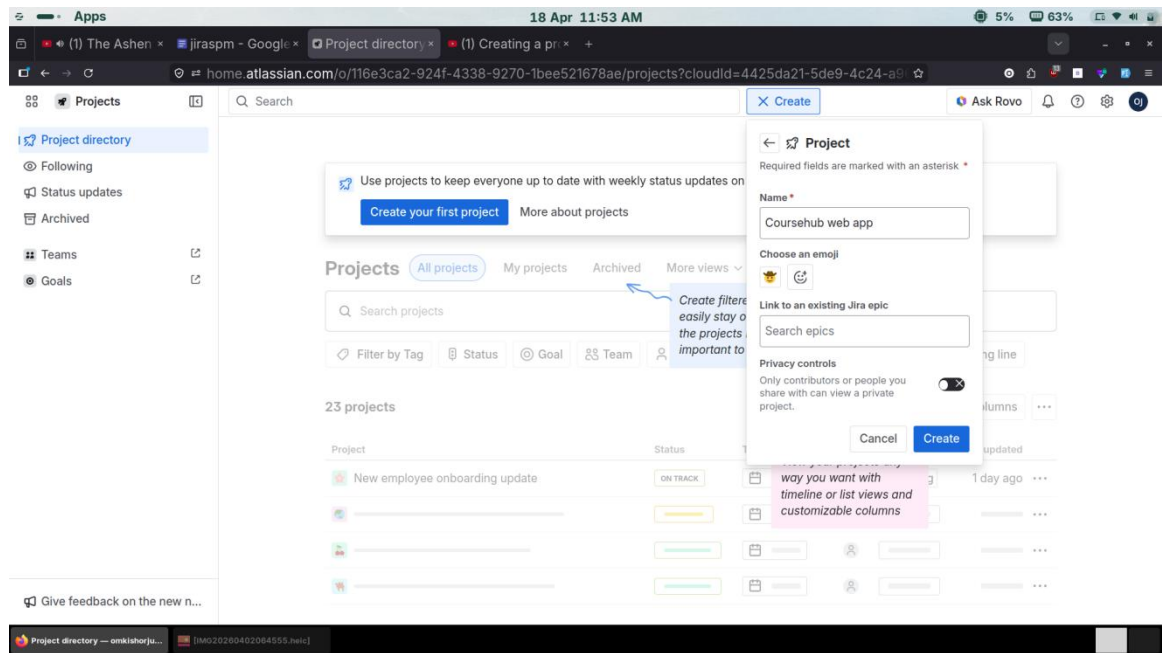


Figure 17:Lab 2 – Project name and key entry form

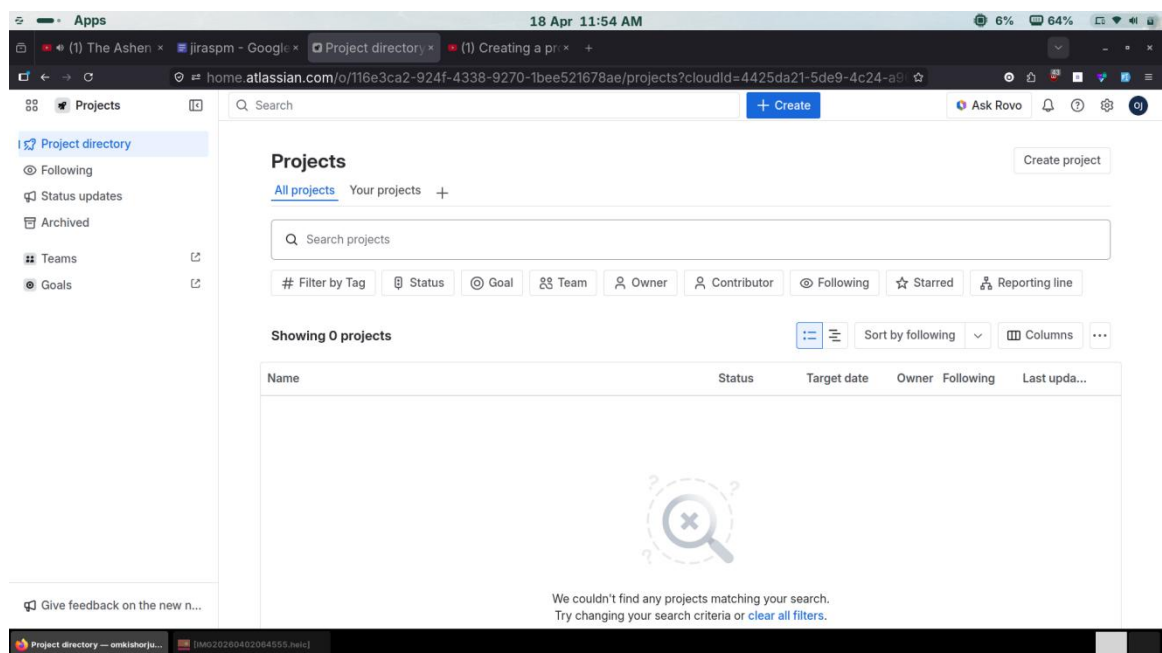


Figure 18:Lab 2 – Project creation confirmation

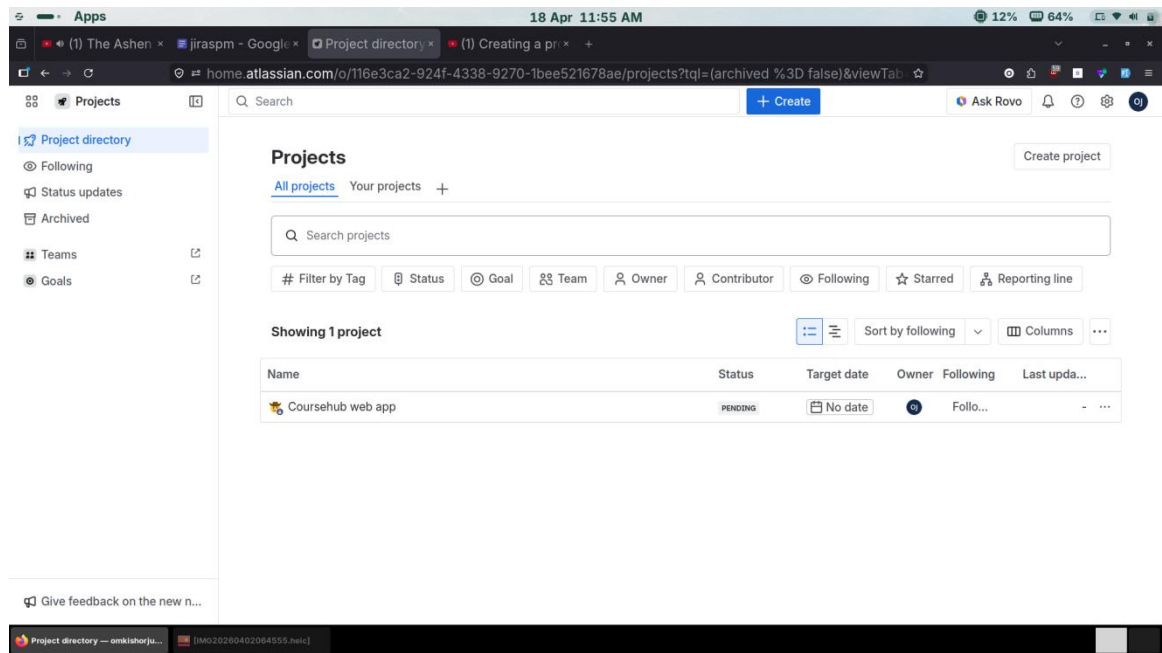


Figure 19:Lab 2 – Project board (empty)

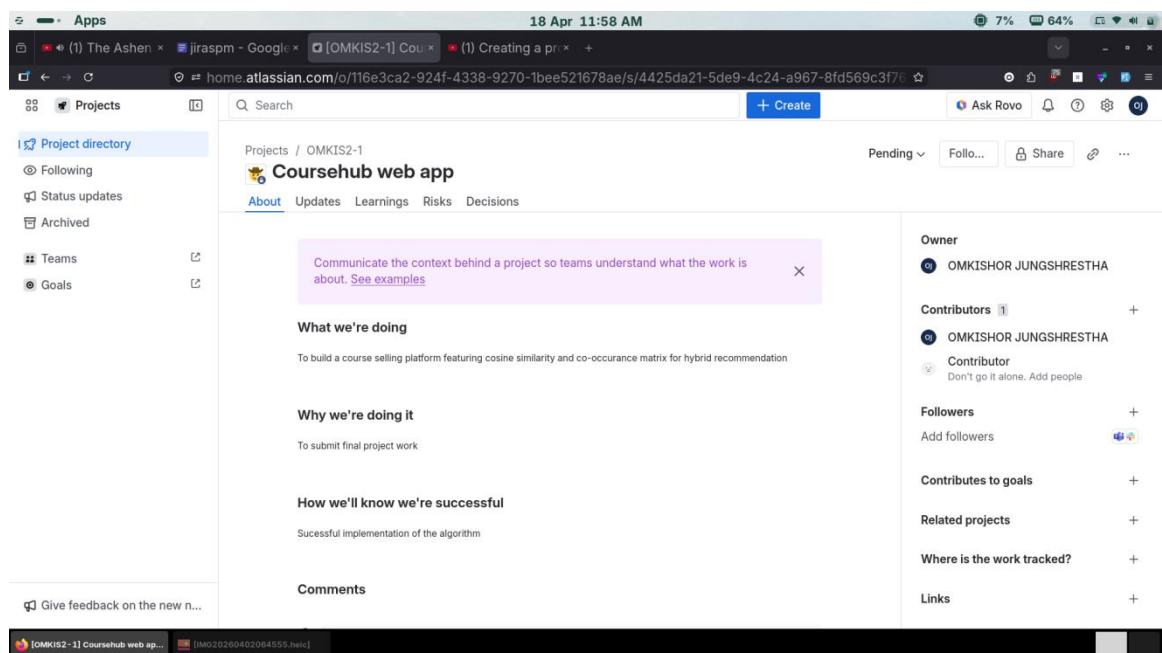


Figure 20:Lab 2 – Backlog view after project creation

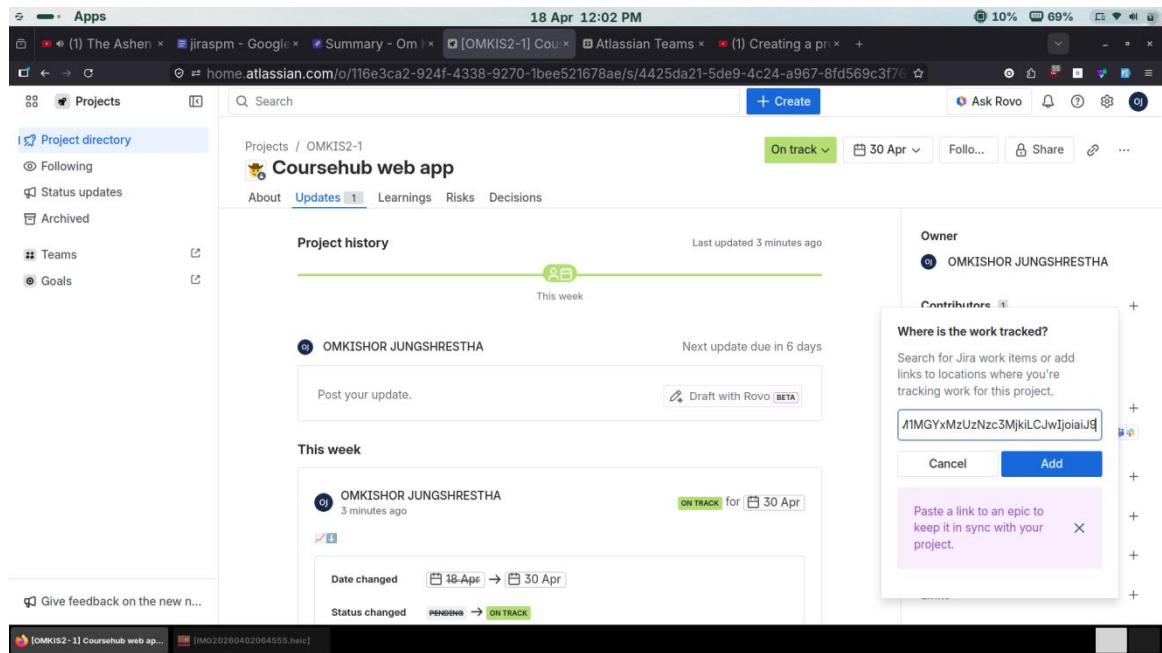


Figure 21:Lab 2 – Project in left sidebar

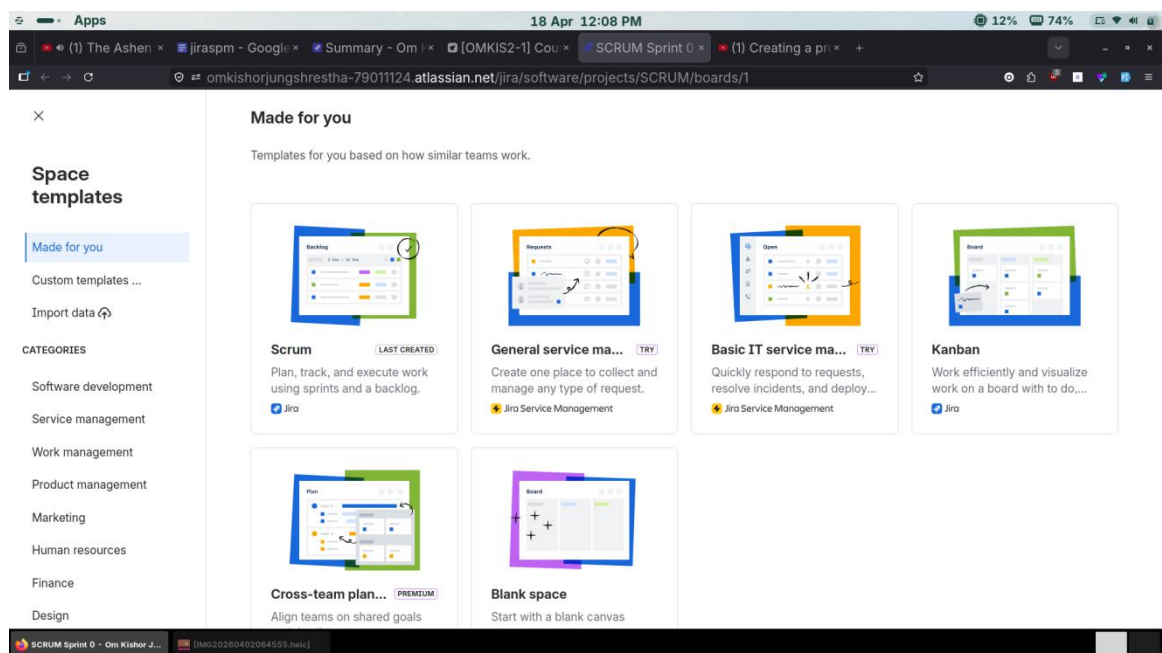


Figure 22:Lab 2 – Scrum board columns

Conclusion: A Scrum project was successfully created using Jira's project template wizard. The project board and backlog are ready for use.

Lab 3 – Project Menu and Options

3.1 – Adding Team Members

Objective: To add collaborators to the project so they can be assigned tasks and participate in sprints.

1. Opened Project Settings from the left sidebar.
2. Navigated to the "Access" section and clicked "Add Members".
3. Entered team members' email addresses and assigned roles (Administrator, Member, or Viewer).
4. Team members received invitation emails and joined the workspace.

3.2 – Introduction to Roadmaps, Backlogs and Boards

Roadmap: A timeline-based view showing all Epics with their planned date ranges. Used for high-level release planning.

Backlog: A prioritised list of all issues not yet added to an active sprint. Supports drag-and-drop ordering.

Board: The Scrum board displaying current sprint issues across "To Do", "In Progress", and "Done" columns.

3.3 – Adding Shortcuts and Project Setup

1. Clicked the "+" icon beside "Shortcuts" in the sidebar and added an external link (e.g., GitHub URL).
2. Configured project settings including issue types, workflow, components, and versions.

Screenshots

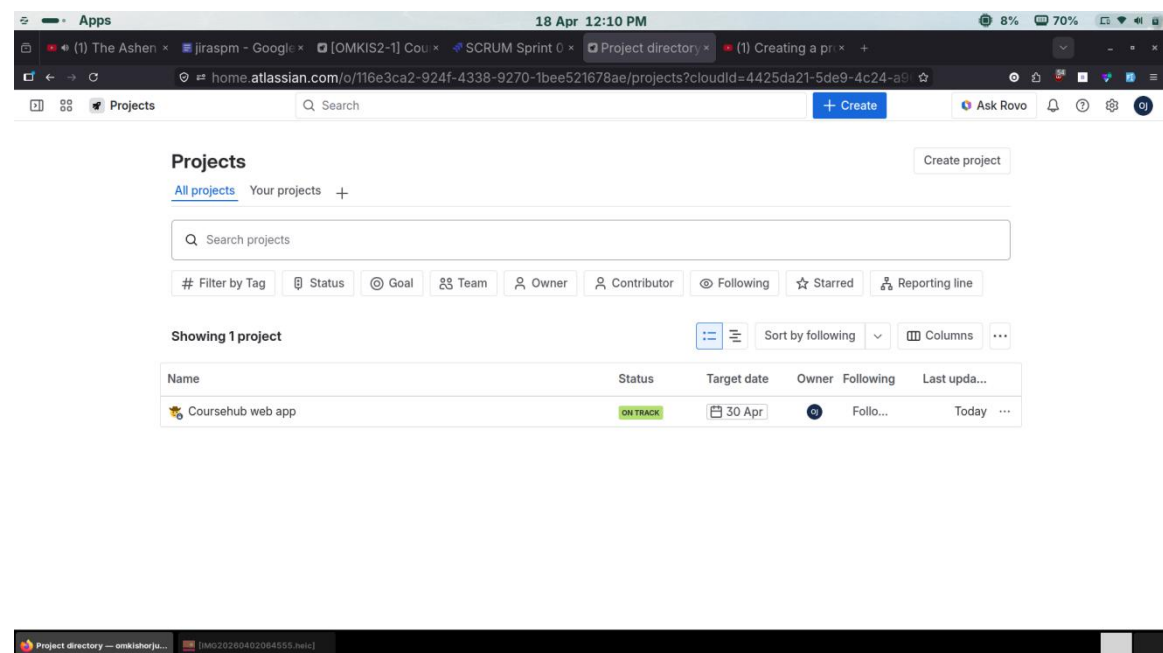


Figure 23:Lab 3 – Project Settings – Access/Members page

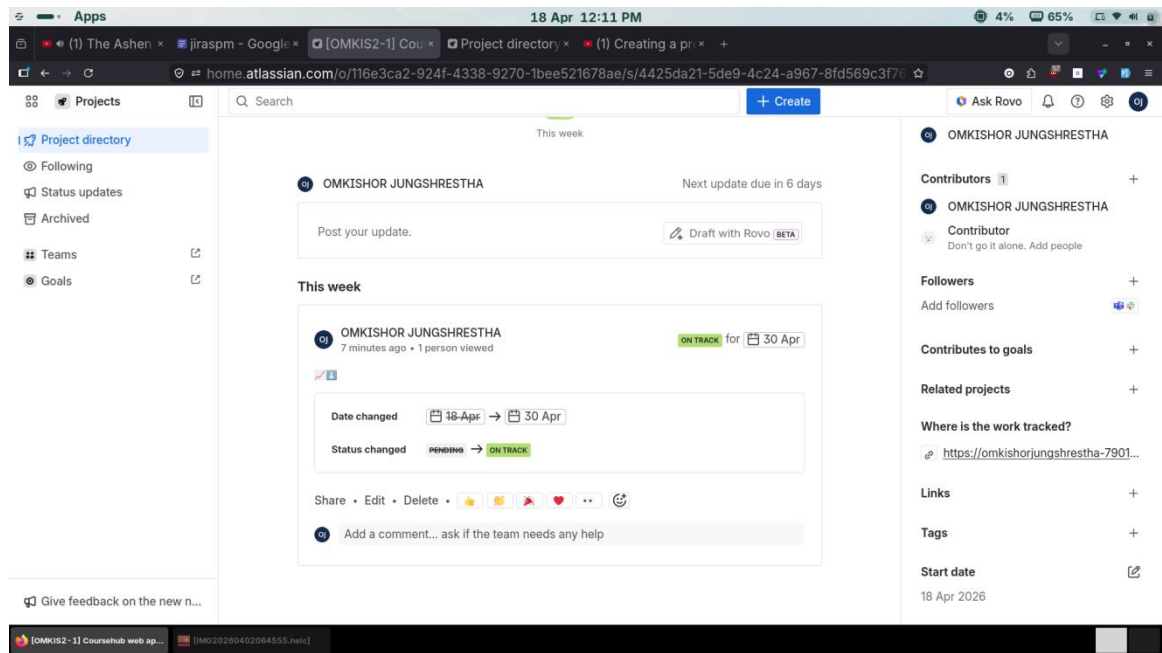


Figure 24:Lab 3 – Add member invitation dialog

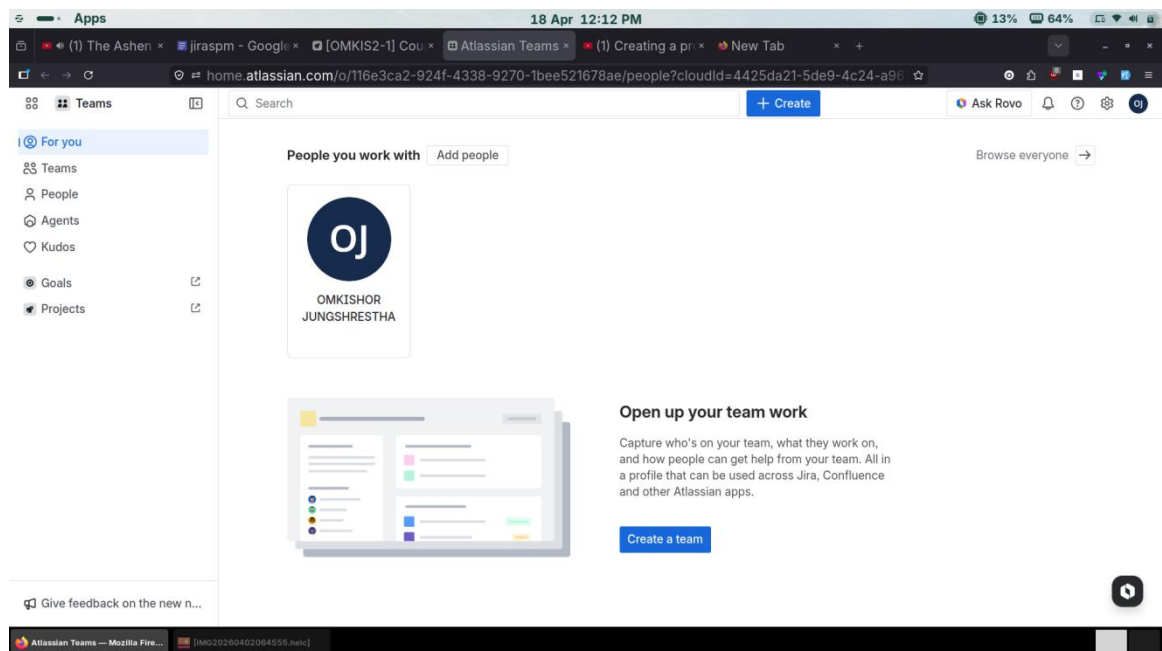


Figure 25:Lab 3 – Roadmap view

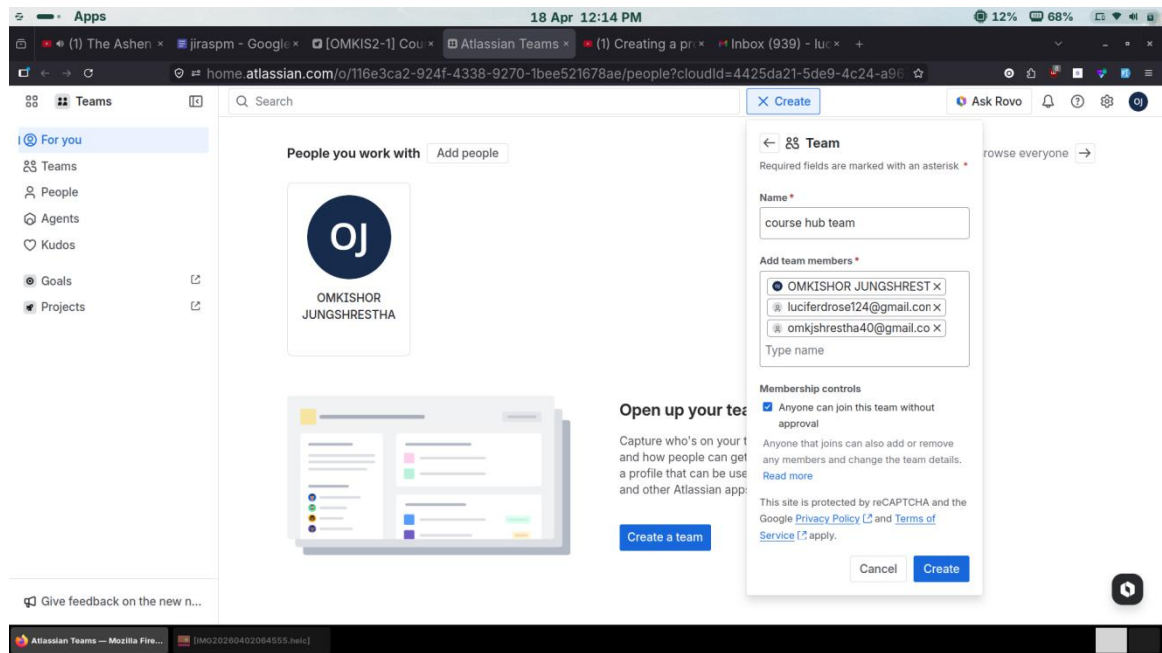


Figure 26:Lab 3 – Backlog view with issues

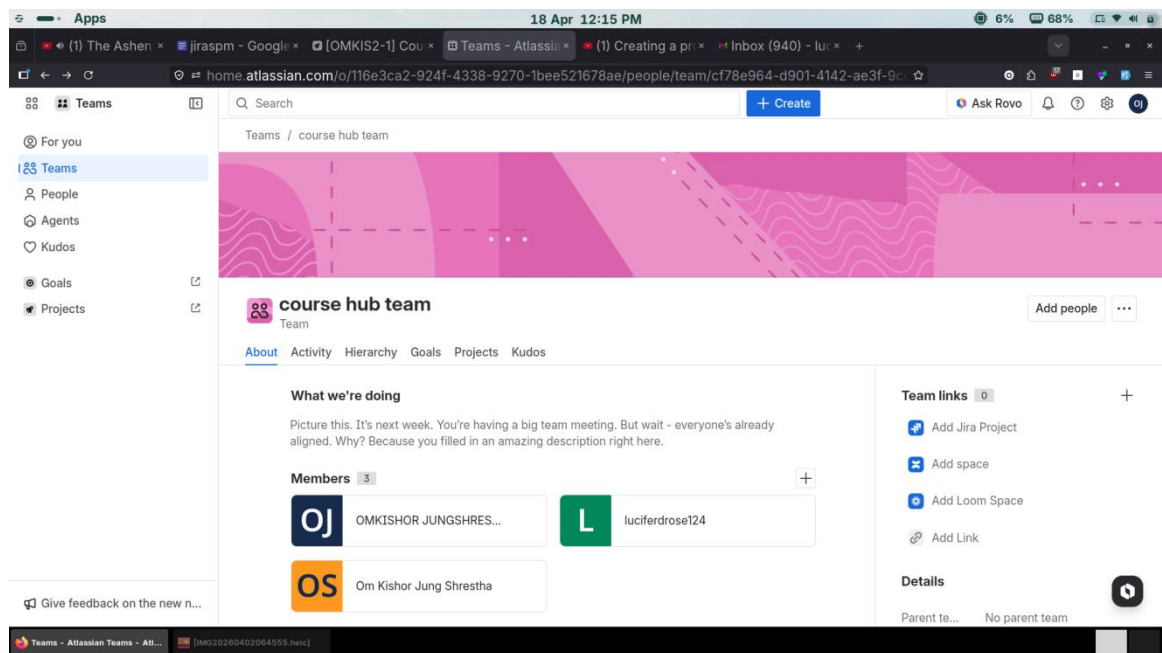


Figure 27:Lab 3 – Board view (Scrum columns)

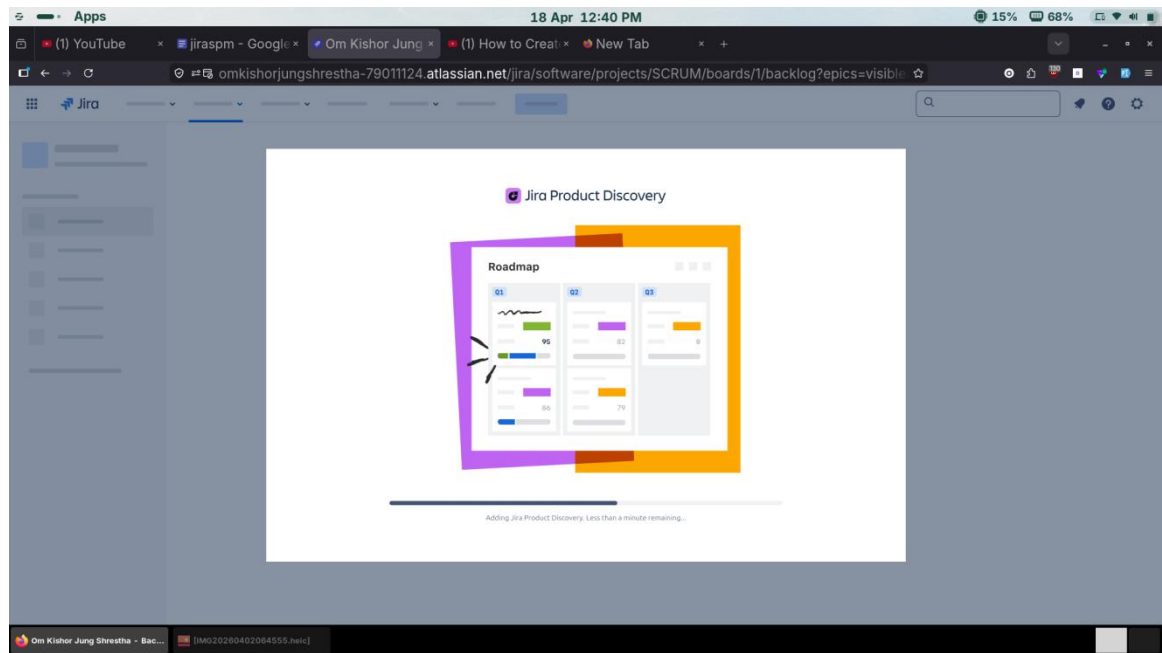


Figure 28:Lab 3 – Add shortcut dialog

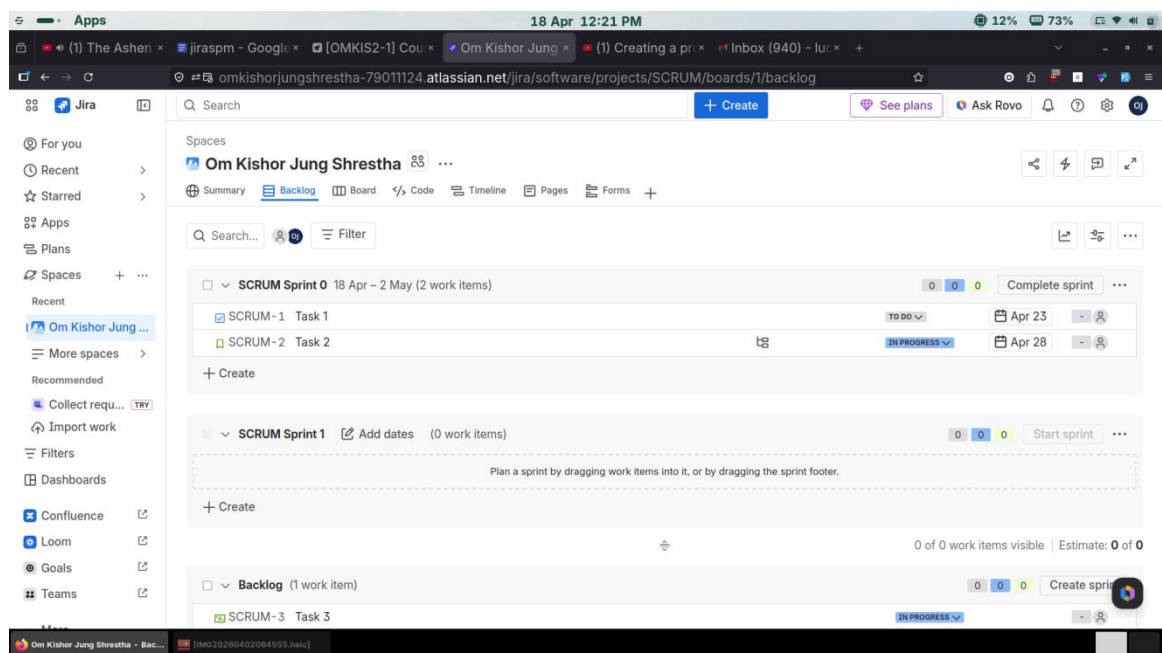


Figure 29:Lab 3 – Shortcut added to sidebar

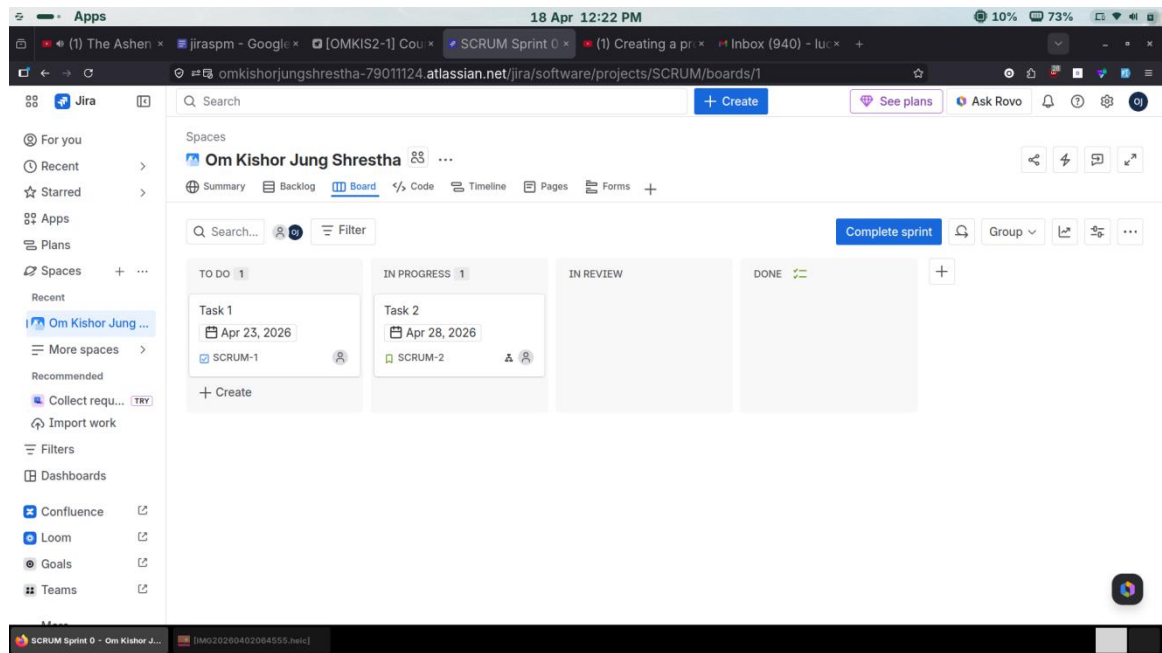


Figure 30:Lab 3 – Project setup configuration

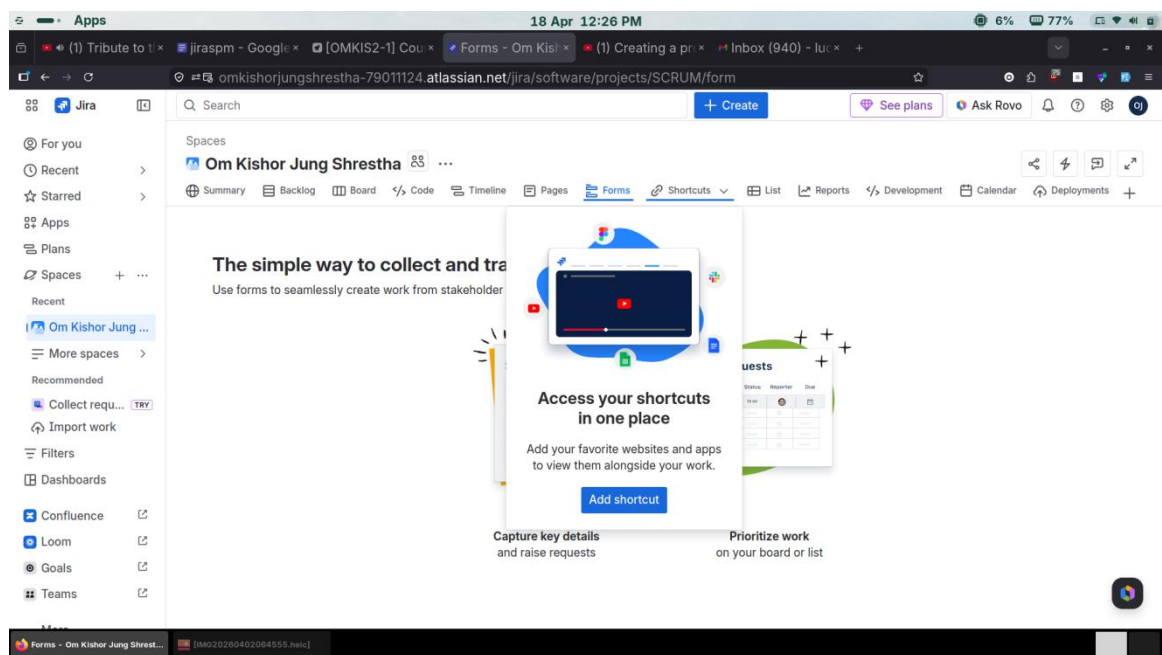


Figure 31:Lab 3 – Issue types list

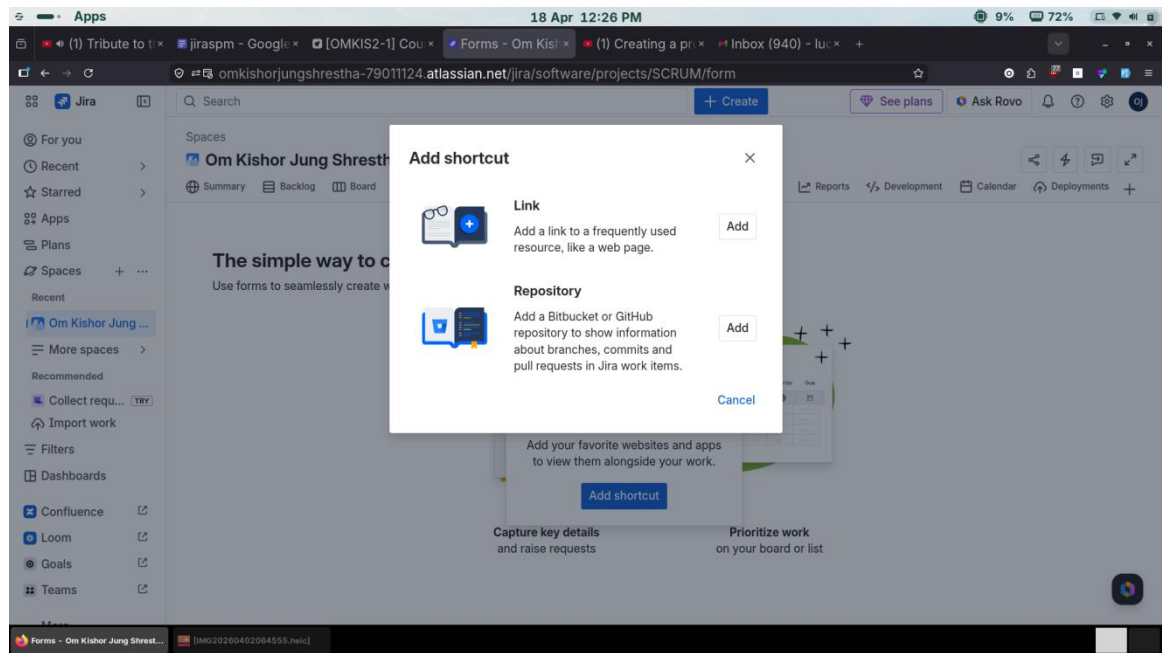


Figure 32:Lab 3 – Workflow scheme

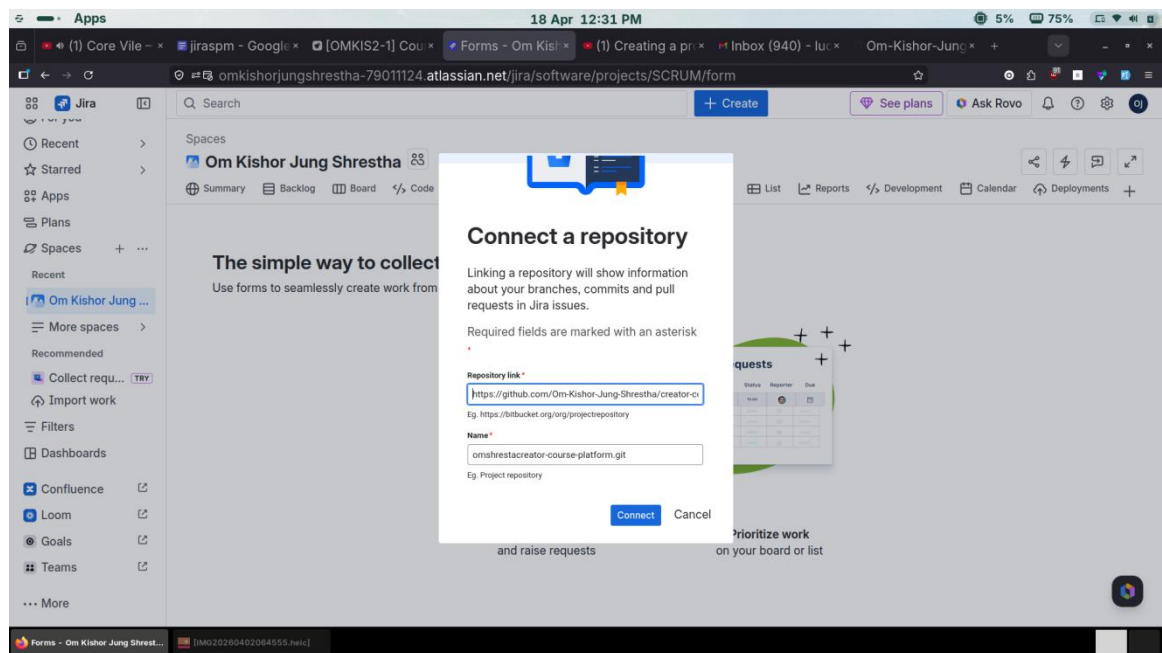


Figure 33:Lab 3 – Component configuration

Conclusion: Team members were added, the Roadmap/Backlog/Board views were explored, and a project shortcut was configured.

Lab 4 – The Agile Methodology

Objective: To understand the Agile software development methodology and how Jira supports Agile principles.

Agile is an iterative, incremental approach to software development that emphasises collaboration, customer feedback, and working software delivered in short cycles called sprints.

Key Agile Principles Covered

1. Customer Collaboration: Continuous stakeholder engagement to gather and refine requirements.
2. Responding to Change: Agile welcomes changing requirements even late in development.
3. Working Software: Functional software delivered at the end of each sprint is the primary measure of progress.
4. Sprint Ceremonies: Sprint Planning, Daily Standup, Sprint Review, and Retrospective.
5. Roles: Product Owner (manages backlog), Scrum Master (removes blockers), Development Team (delivers increments).

Agile in Jira

Jira supports Agile through Scrum boards (sprints, backlog, velocity charts) and Kanban boards (WIP limits, continuous flow). Agile metrics such as velocity, burndown, and CFD charts are built in.

The Agile Manifesto

The Agile Manifesto, published in 2001 by seventeen software practitioners, is the foundation of Agile development. It declares four core values: (1) Individuals and interactions over processes and tools; (2) Working software over comprehensive documentation; (3) Customer collaboration over contract negotiation; and (4) Responding to change over following a plan. These values do not dismiss the items on the right but affirm that the items on the left are valued more.

The manifesto is accompanied by twelve principles that guide Agile teams: delivering working software frequently, welcoming changing requirements, maintaining a sustainable pace, and striving for technical excellence. The emphasis on face-to-face communication, self-organising teams, and regular reflection makes Agile fundamentally people-centred rather than process-centred.

Agile Frameworks

Several frameworks implement Agile principles in practice. Scrum is the most widely adopted framework and organises work into fixed-length sprints (typically one to four weeks). Each sprint produces a potentially shippable product increment. The Scrum framework defines three roles: the Product Owner who owns the product vision and backlog; the Scrum Master who is a servant-leader ensuring the team follows Scrum practices; and the Development Team who self-organise to deliver the increment.

Kanban is another Agile framework that focuses on visualising work, limiting work-in-progress (WIP), and optimising flow. Unlike Scrum, Kanban has no fixed iterations; work items move through defined stages (To Do, In Progress, Done) and a new item is pulled

in only when the team has capacity. Kanban is well-suited to support and maintenance work where tasks arrive continuously.

Extreme Programming (XP) is a third framework emphasising engineering practices such as test-driven development (TDD), pair programming, continuous integration, and refactoring. XP advocates small, frequent releases and close customer involvement, making it ideal for projects with rapidly changing requirements.

SAFe (Scaled Agile Framework) and LeSS (Large-Scale Scrum) extend Agile principles to large organisations with many teams, introducing additional roles, ceremonies, and artefacts to coordinate work at programme and portfolio levels.

Agile Ceremonies in Detail

Sprint Planning is held at the start of each sprint. The team selects items from the Product Backlog, breaks them into tasks, and commits to a Sprint Goal. The Development Team decides how much work they can complete, while the Product Owner clarifies priorities and acceptance criteria.

The Daily Standup (or Daily Scrum) is a 15-minute time-boxed event held every working day. Each team member answers three questions: What did I do yesterday? What will I do today? Are there any impediments? The purpose is synchronisation and early identification of blockers, not a status report to management.

The Sprint Review (Demo) is held at the end of the sprint. The team demonstrates completed work to stakeholders, and the Product Owner accepts or rejects items. Stakeholder feedback is collected and may be added to the Product Backlog, ensuring the product evolves to meet real needs.

The Sprint Retrospective is the team's opportunity to inspect its own process and improve. The team discusses what went well, what did not, and what actions to take in the next sprint. Retrospectives foster a culture of continuous improvement and psychological safety.

Agile Artefacts

The Product Backlog is an ordered list of everything that is known to be needed in the product. It is the single source of requirements and is continuously refined by the Product Owner. Items at the top are more detailed and ready for development, while lower-priority items are coarser grained.

The Sprint Backlog is the set of Product Backlog items selected for the sprint, plus a plan for delivering the product increment. It is owned by the Development Team and is updated daily as new information emerges. Any work added during the sprint must not jeopardise the Sprint Goal.

The Product Increment is the sum of all Product Backlog items completed during a sprint and all previous sprints. The increment must be in a usable condition and meet the team's Definition of Done, regardless of whether the Product Owner decides to release it.

Agile vs Traditional (Waterfall) Development

Traditional Waterfall development follows a sequential, phase-based approach: Requirements – Design – Implementation – Testing – Deployment. Each phase must be completed before the next begins, and change is expensive because it may require revisiting earlier phases. Waterfall works well for stable, well-understood requirements such as construction or manufacturing projects.

Agile, by contrast, delivers working software in short cycles and welcomes change at any stage. Risk is reduced because feedback is gathered early and frequently. However, Agile requires active stakeholder participation, disciplined teams, and cultural readiness for transparency. Organisations adopting Agile for the first time often struggle with incomplete backlog refinement and resistance to daily standups.

Agile Metrics and Reporting in Jira

Jira provides several built-in Agile metrics that help teams measure and improve performance. The Burndown Chart tracks remaining story points against the sprint timeline, showing whether the team is on pace to complete all committed work. The Velocity Chart shows committed vs completed points across multiple sprints, enabling data-driven sprint capacity planning.

The Cumulative Flow Diagram (CFD) is a stacked area chart that visualises the number of issues in each status column over time. A widening band in the In Progress column signals a bottleneck that needs attention. The Cycle Time Report measures the elapsed time between an issue entering In Progress and reaching Done, helping identify process inefficiencies at the individual issue level.

These metrics are not used to judge individual developers but to give the team a shared view of their process health. Teams review metrics in Sprint Reviews and Retrospectives to decide on process adjustments, such as reducing sprint scope, adding pair programming, or improving the Definition of Done.

Conclusion: The Agile methodology was studied in the context of Jira's Scrum features. Agile values and ceremonies were mapped to Jira's tooling.

Lab 5 – Introduction to Scrum and Sprint

Objective: To understand the Scrum framework and create/manage sprints in Jira.

Scrum organises work into time-boxed Sprints (1–4 weeks). Key artefacts: Product Backlog, Sprint Backlog, and Increment.

Steps Performed

1. Navigated to the Backlog view of the Scrum project.
2. Clicked "Create Sprint" at the top of the backlog.
3. Entered sprint name, start date, end date, and sprint goal.
4. Dragged issues from the backlog into the sprint container.
5. Clicked "Start Sprint" to activate the sprint and populate the Scrum board.
6. Observed the sprint countdown timer and board columns with assigned issues.

Screenshots

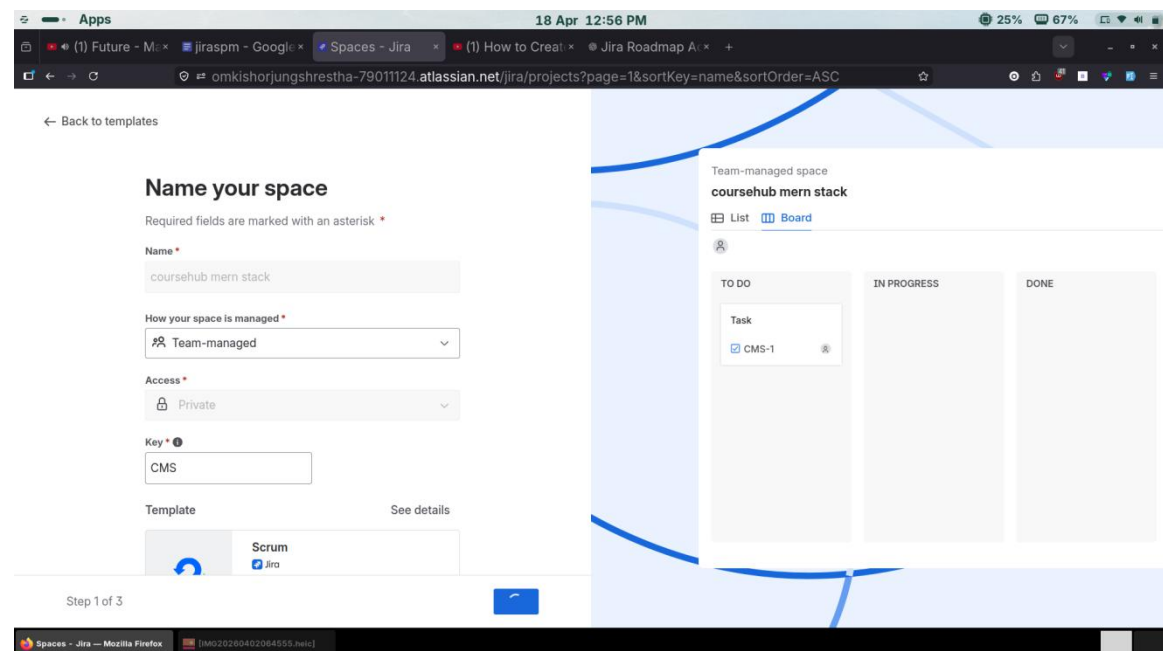


Figure 34: Lab 5 – Backlog with sprint container

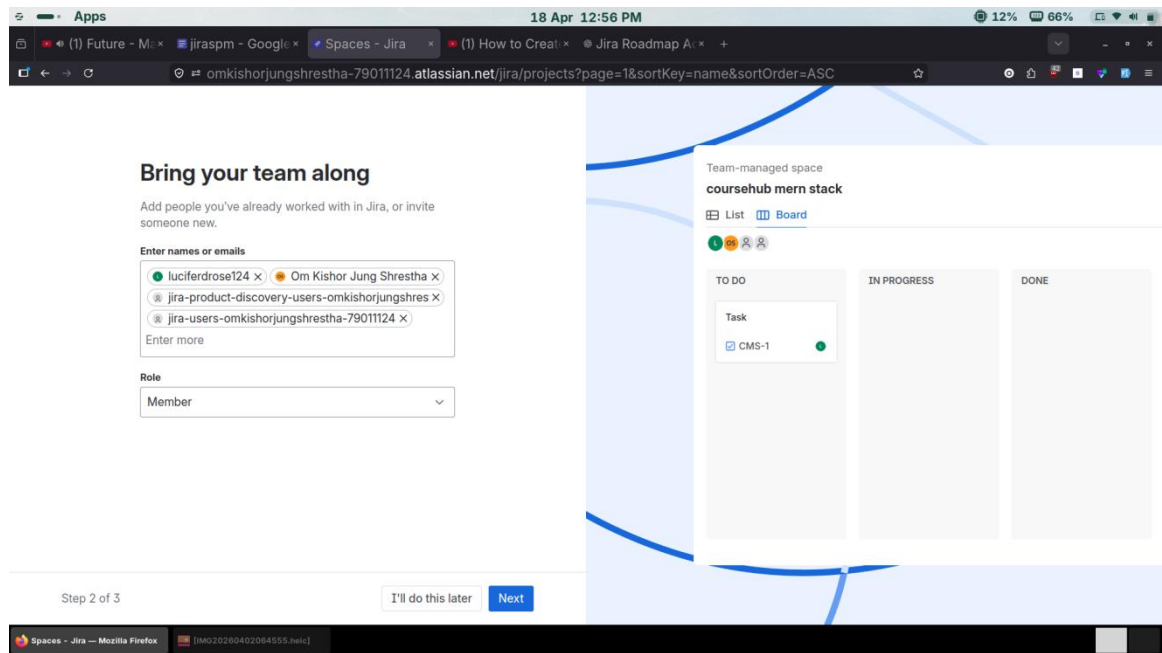


Figure 35:Lab 5 – Create sprint dialog

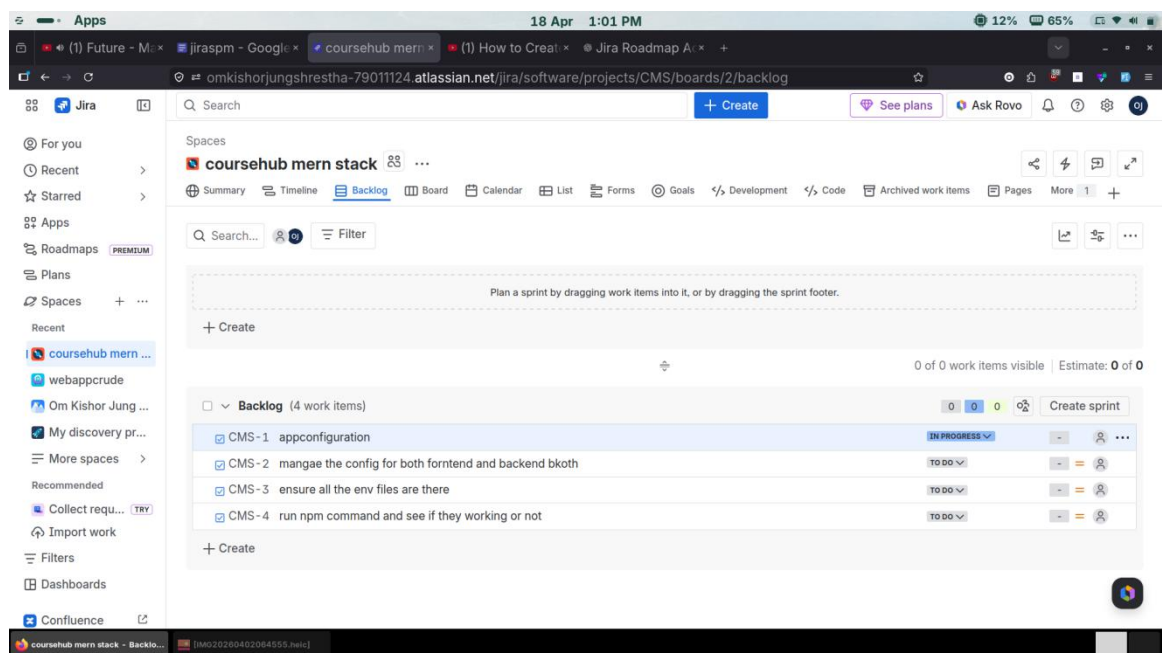


Figure 36:Lab 5 – Issues dragged into sprint

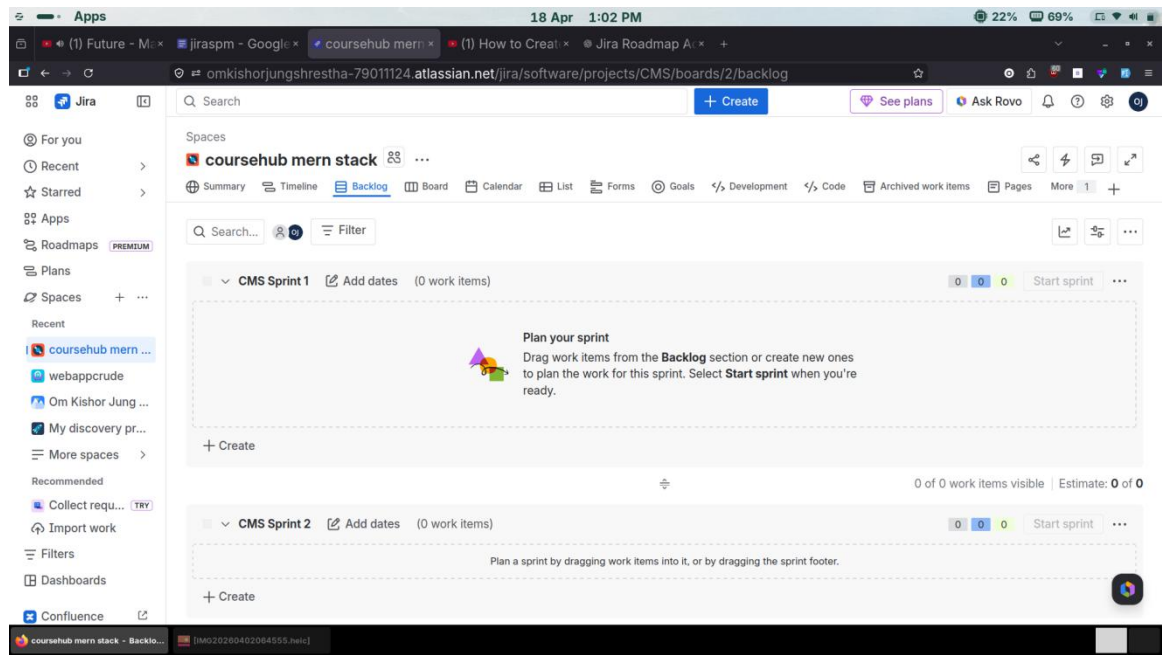


Figure 37:Lab 5 – Start sprint confirmation

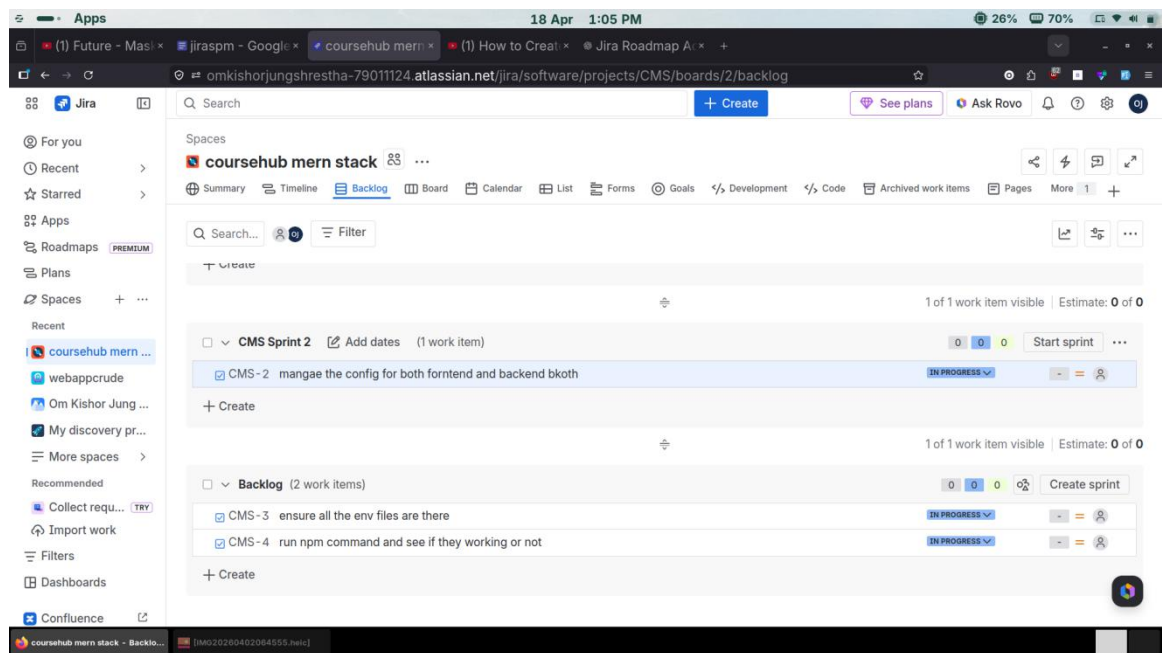


Figure 38:Lab 5 – Active sprint on Scrum board

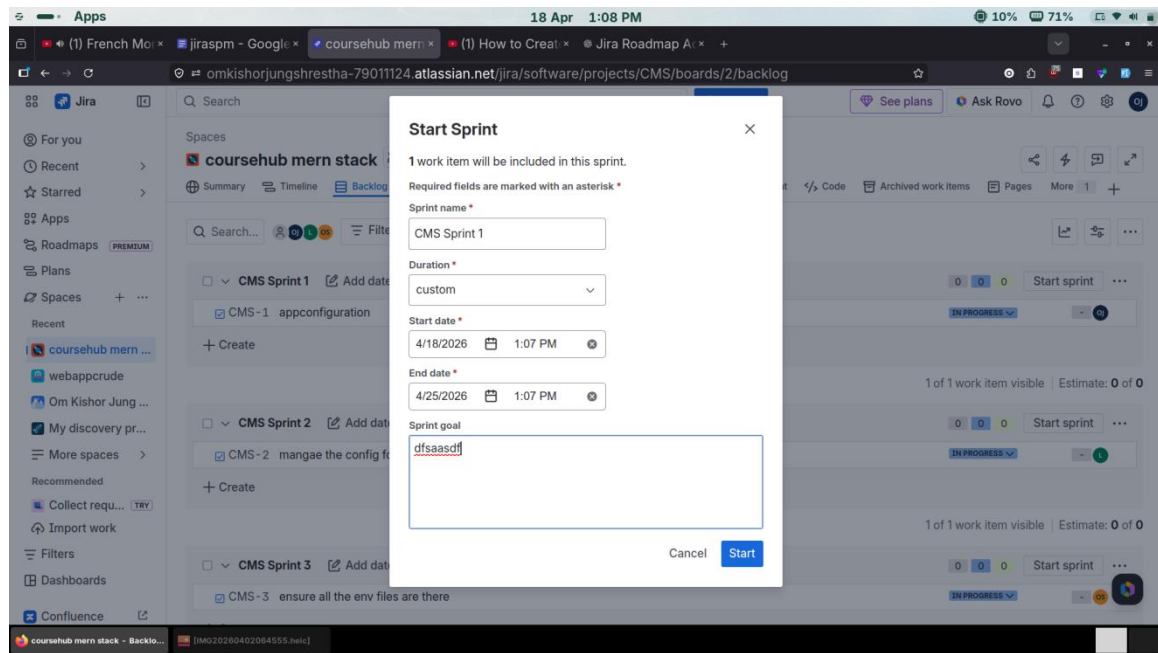


Figure 39:Lab 5 – Sprint countdown timer

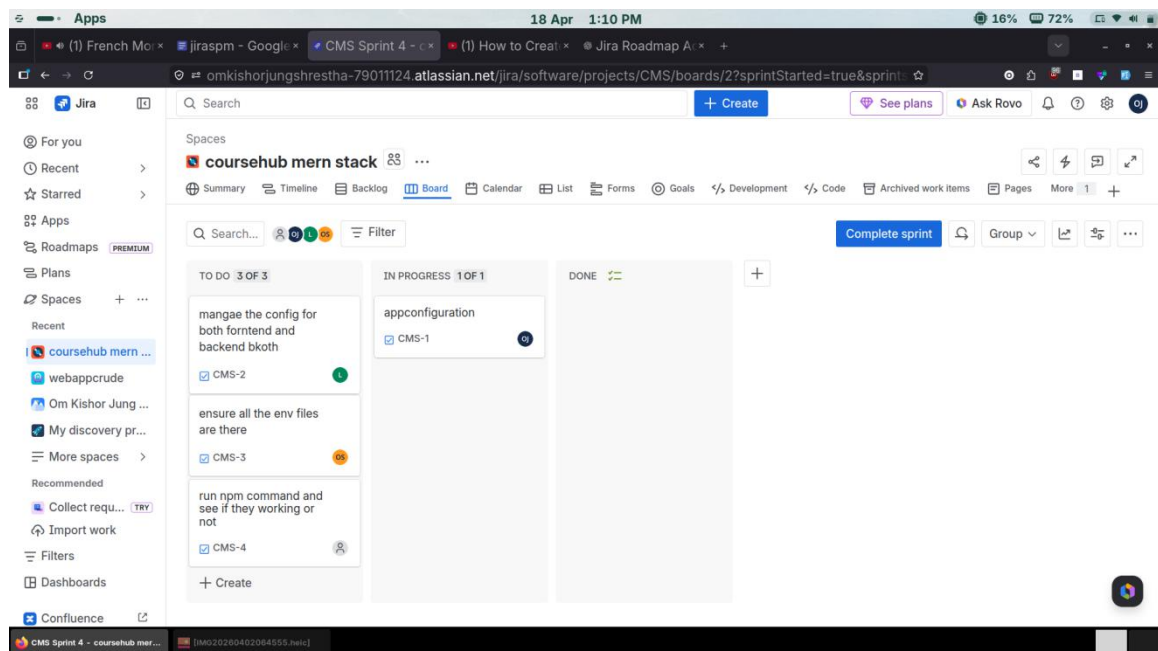


Figure 40:Lab 5 – Sprint board with issues in columns

Conclusion: A sprint was successfully created and started with issues from the backlog. The Scrum board now reflects sprint progress.

Lab 6 – Epic and Roadmap (Creating Roadmap and Epic)

Objective: To create an Epic in Jira and use the Roadmap to plan and visualise work over time.

An Epic is a large work item broken into smaller stories spanning multiple sprints. The Roadmap is a timeline showing Epics with planned start/end dates.

Creating an Epic

1. Clicked "Create" and selected issue type "Epic".
2. Entered the Epic Name (e.g., "User Authentication Module") and description.
3. Set an Epic colour for visual identification on the roadmap.
4. Saved the Epic and confirmed it appeared in the Epic panel.

Using the Roadmap

1. Clicked "Roadmap" in the sidebar to open the timeline view.
2. Observed Epics as horizontal bars across the calendar.
3. Adjusted Epic start/end dates by dragging the bar edges.
4. Expanded an Epic to view linked child stories.

Screenshots

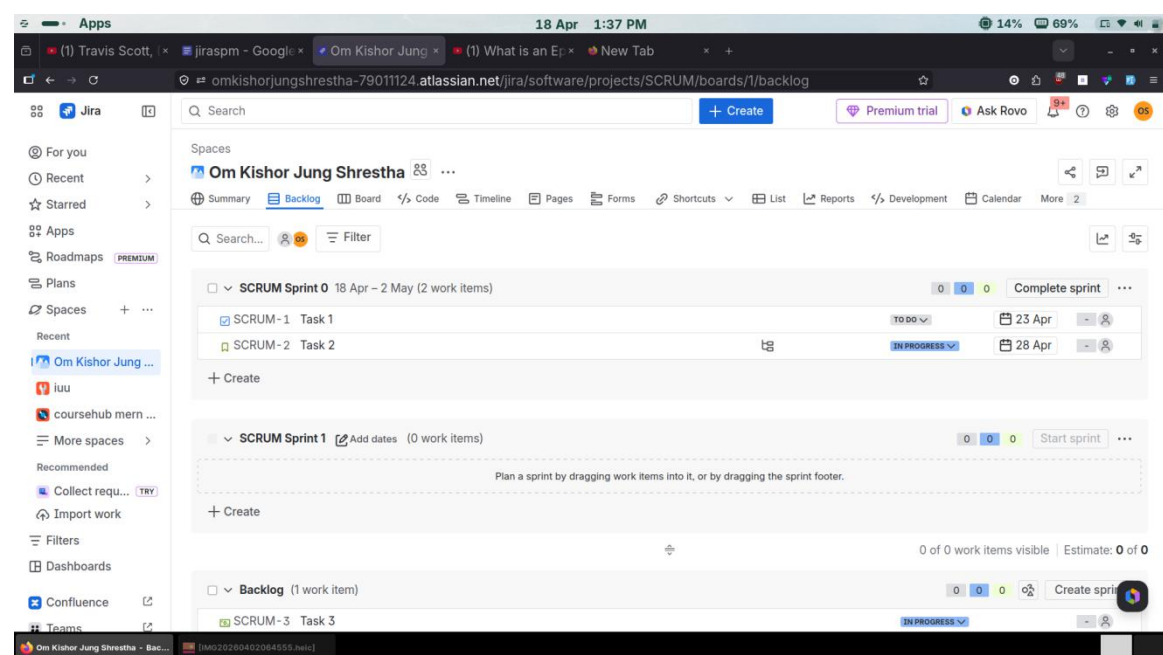


Figure 41:Lab 6 – Roadmap timeline view

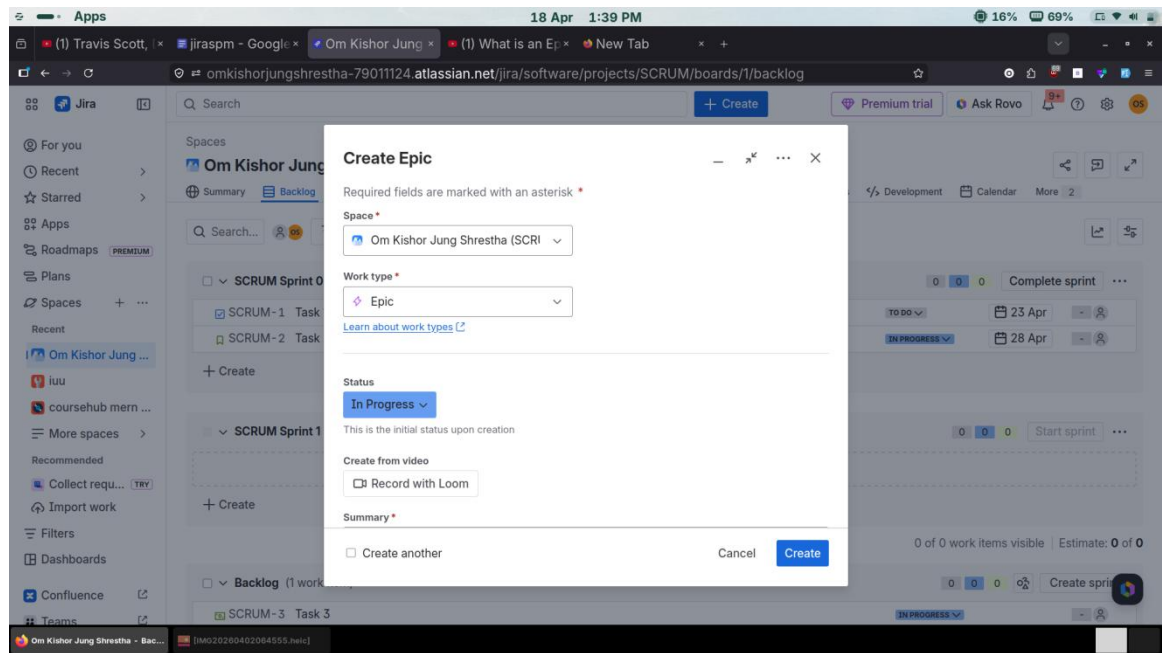


Figure 42:Lab 6 – Create Epic dialog

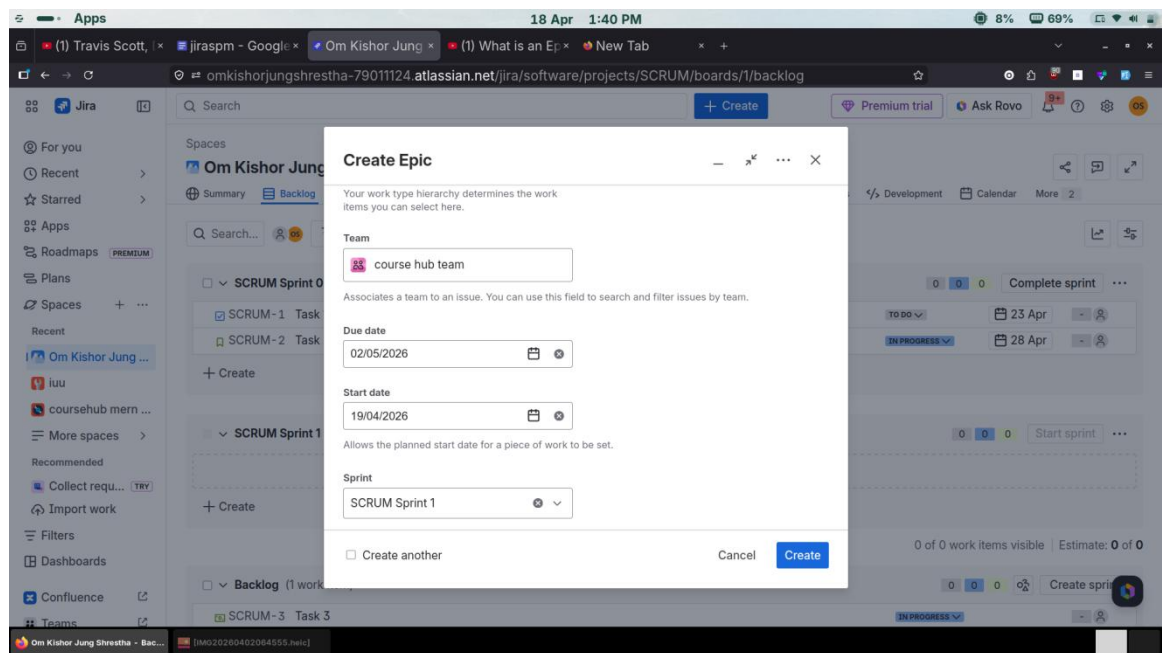


Figure 43:Lab 6 – Epic colour selection

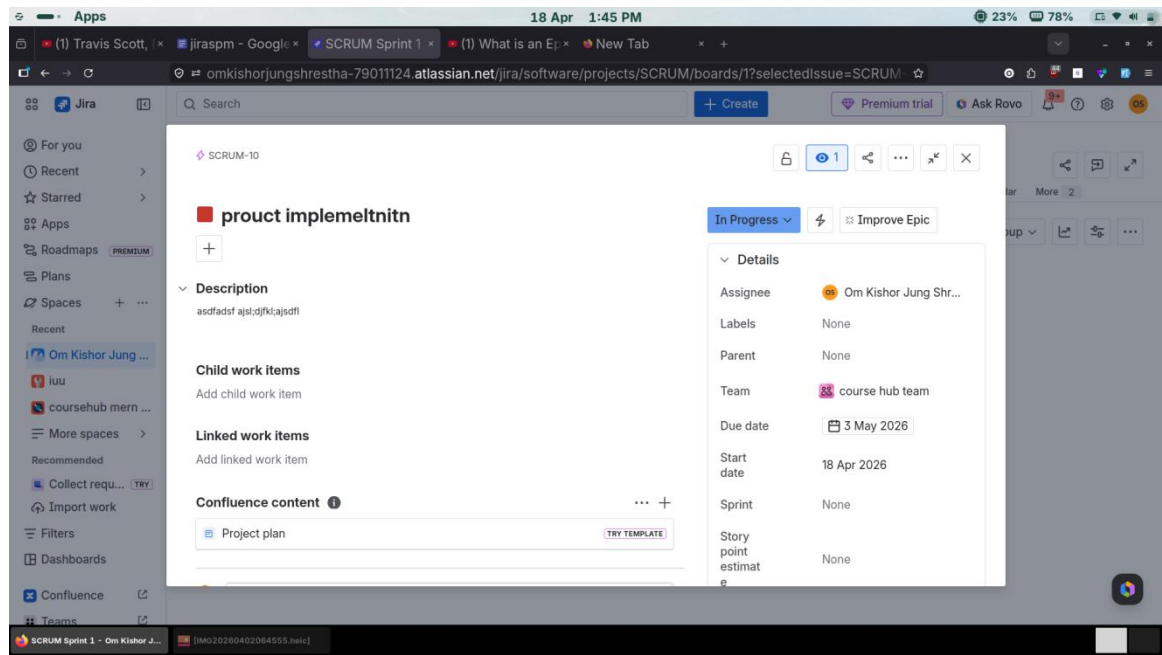


Figure 44:Lab 6 – Epic in Epic panel

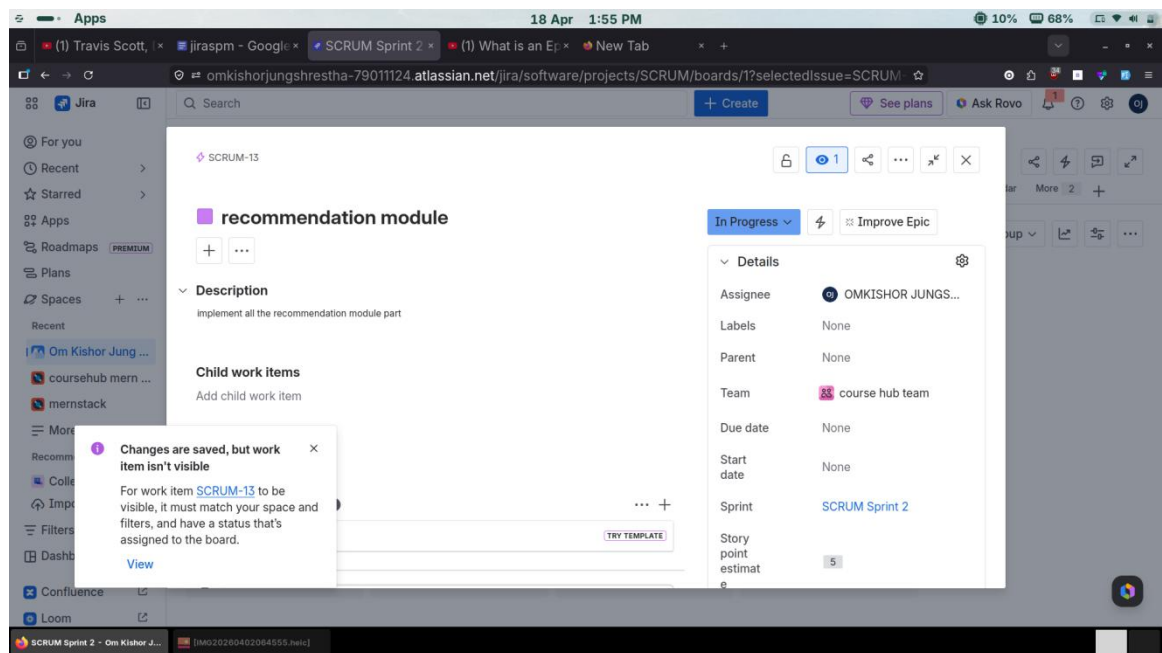


Figure 45:Lab 6 – Roadmap with Epic bar

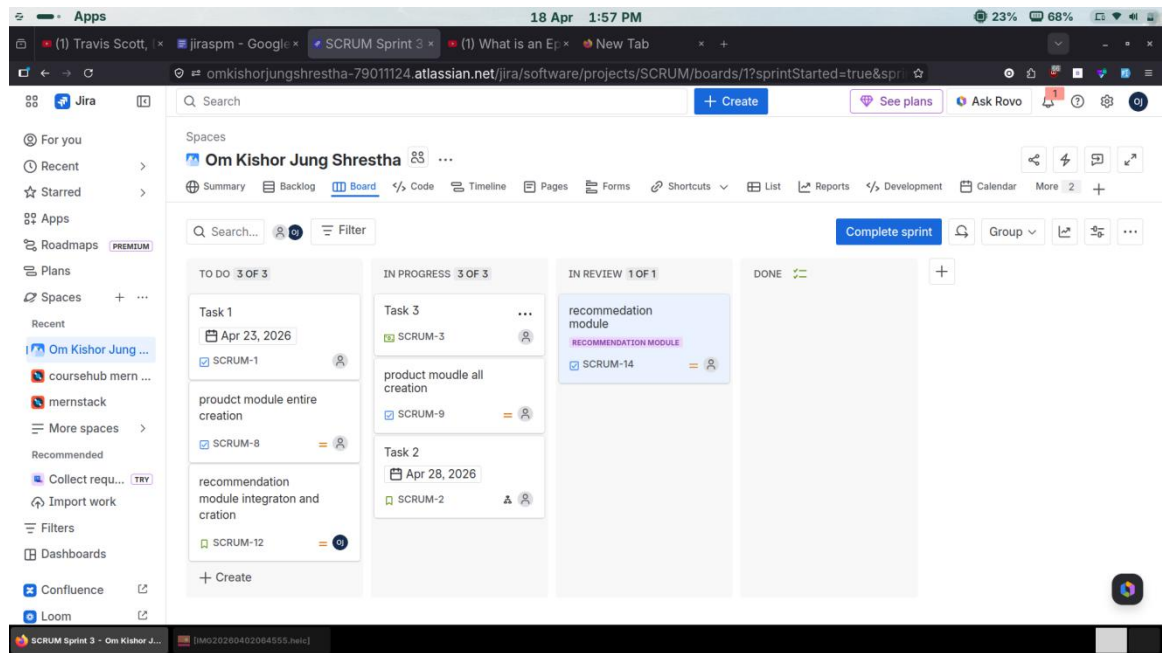


Figure 46: Lab 6 – Epic date range adjustment

Conclusion: An Epic was created and visualised on the Jira Roadmap, providing a high-level view of planned work across sprints.

Lab 7 – Story in Jira

7.1 – Creating, Updating and Deleting a Story

A Story in Jira represents a user-facing feature. Format: "As a [user], I want [goal] so that [benefit]."

Creating: Clicked "Create", selected "Story", filled in Summary, Description, Assignee, and Priority, then saved.

Updating: Opened the story and edited fields inline; changed status via board column drag or transition buttons.

Deleting: Opened the story, clicked the "..." menu, selected "Delete" and confirmed.

7.2 – Story Points, Linking with Epic, and Child Issues

Story Points: Set in the "Story point estimate" field to reflect relative effort/complexity.

Linking to Epic: Used the "Epic Link" field on the story to associate it with a parent Epic.

Child Issues: Added sub-tasks under a story via the "Child Issues" section → "Create child issue".

Screenshots

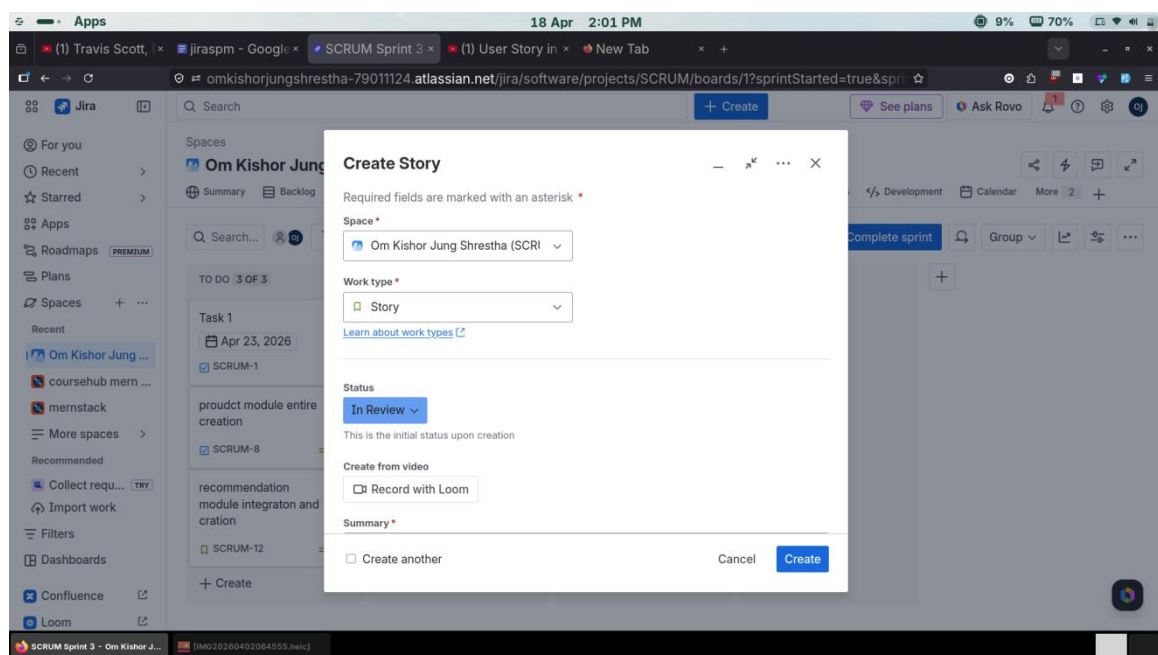


Figure 47: Lab 7 – Create story form

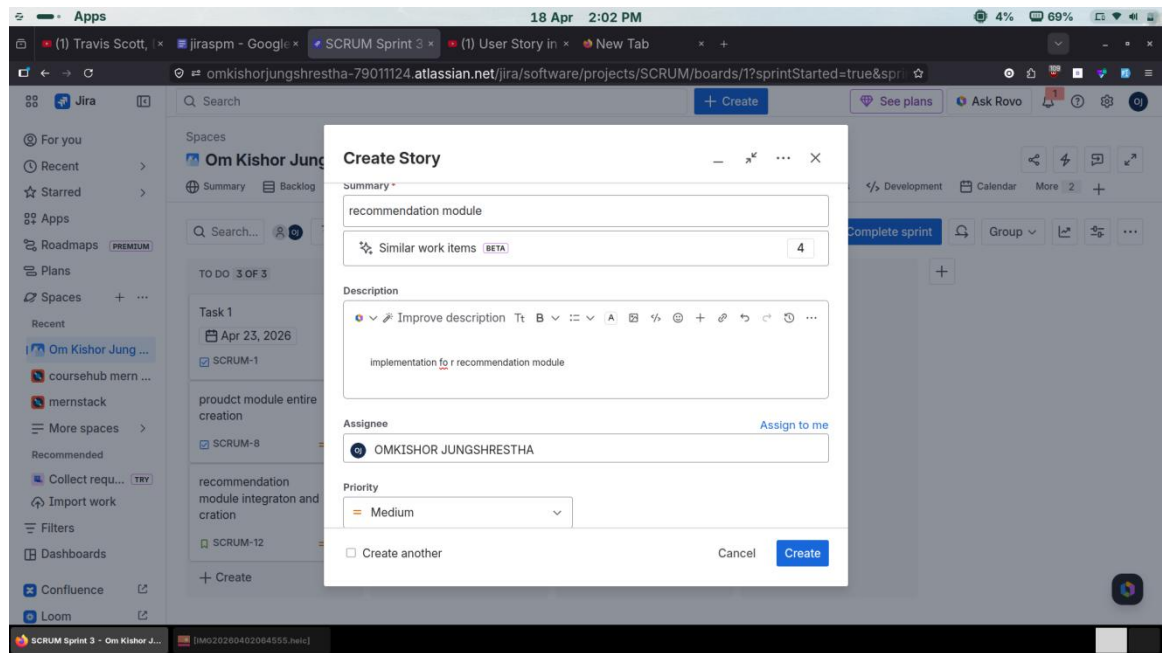


Figure 48:Lab 7 – Story with summary and description

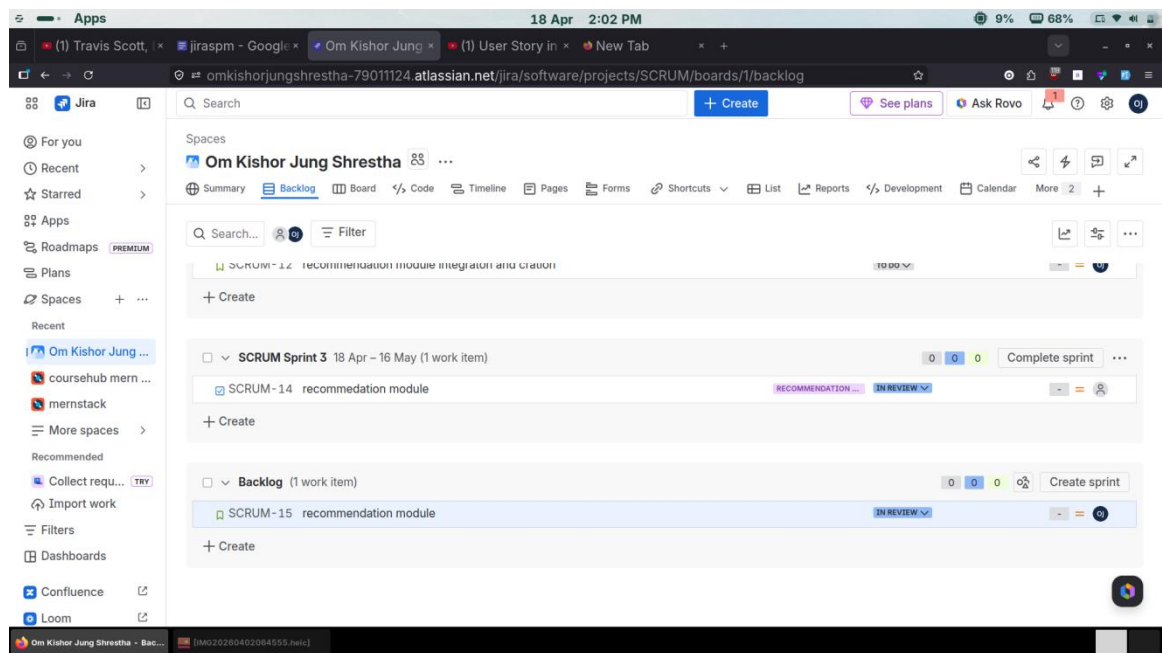


Figure 49:Lab 7 – Story assigned and prioritised

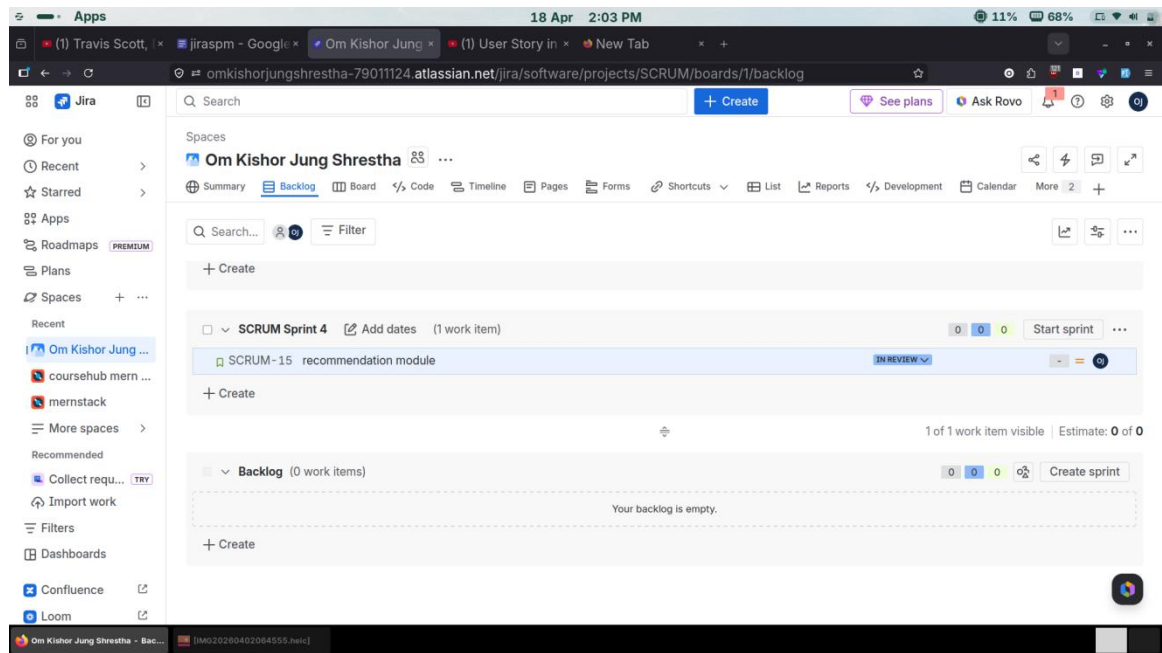


Figure 50:Lab 7 – Story on Scrum board

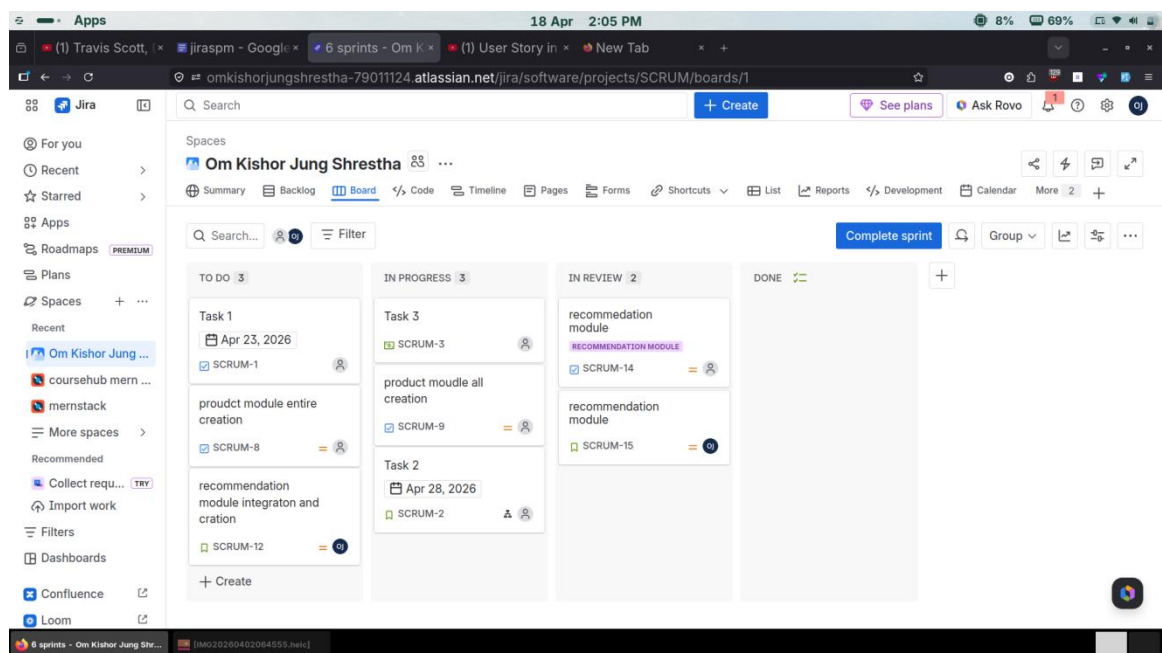


Figure 51:Lab 7 – Update story – status transition

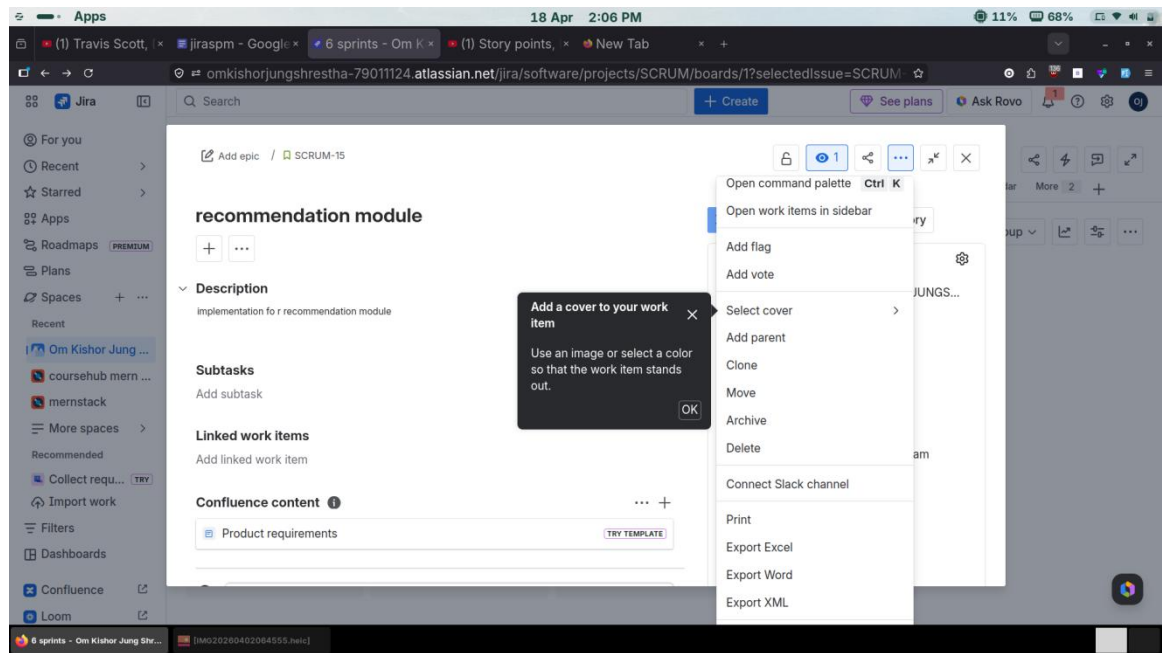


Figure 52:Lab 7 – Delete story confirmation

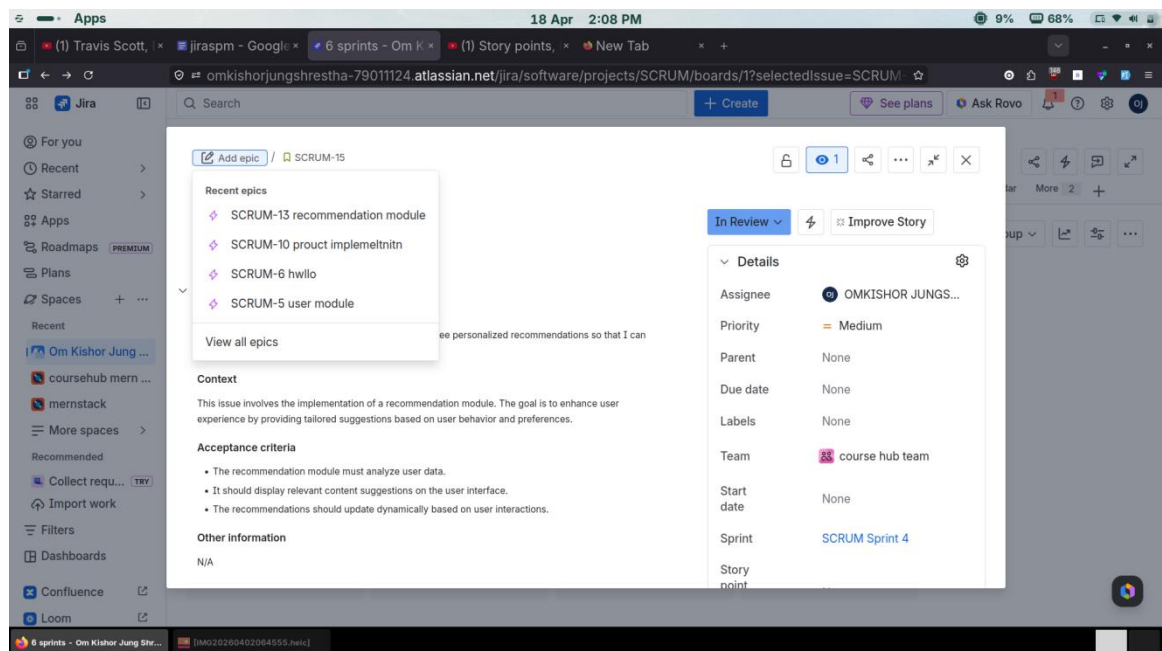


Figure 53:Lab 7 – Story points field

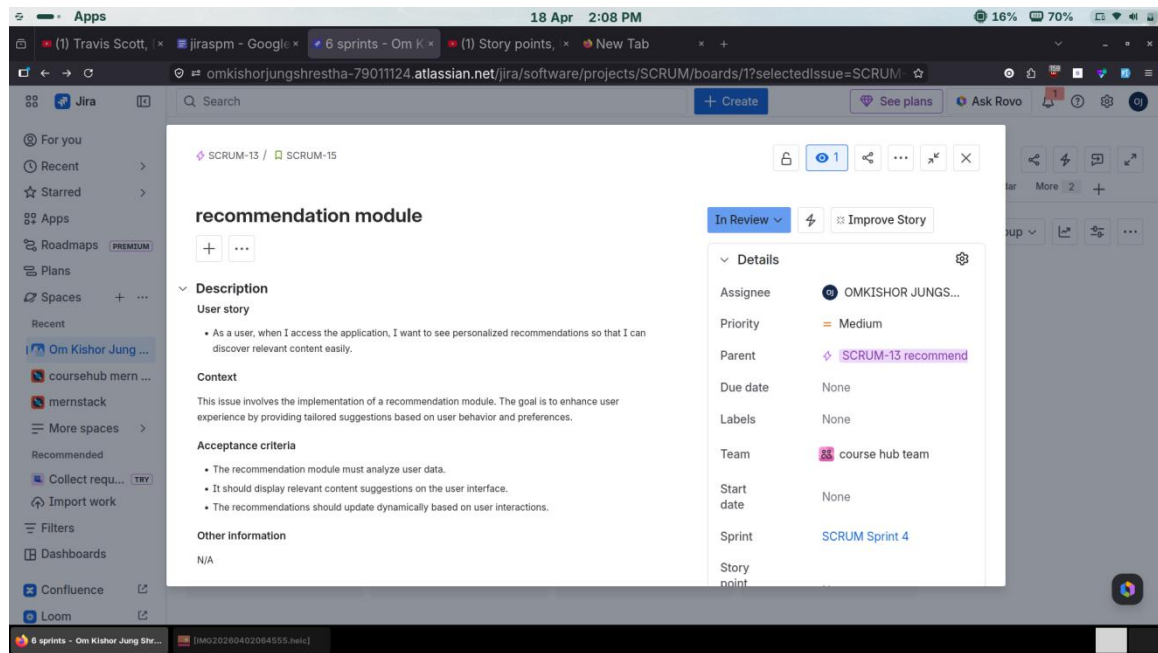


Figure 54:Lab 7 – Epic link field on story

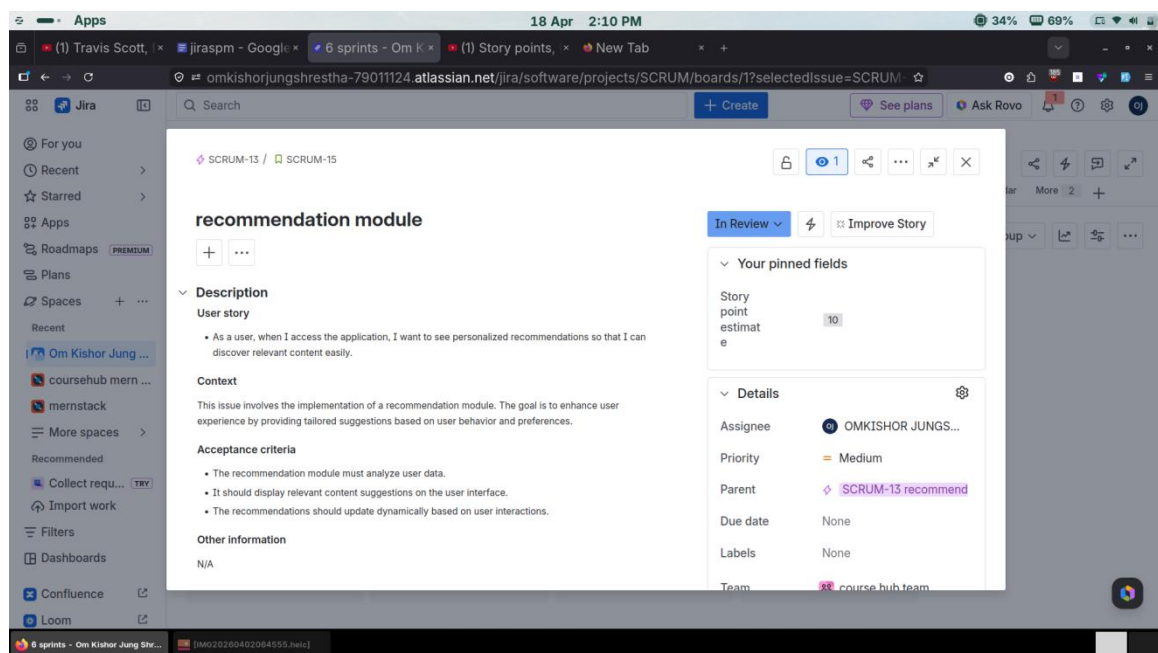


Figure 55:Lab 7 – Child issues section on story

Conclusion: Stories were created, updated, and deleted. Story points were estimated, Epics were linked, and child issues were added to break down stories.

Lab 8 – Creating Sprint and Backlog

Objective: To manage the product backlog and create a sprint with selected backlog items.

Steps Performed

1. Opened the Backlog view and reviewed existing stories, bugs, and tasks.
2. Re-prioritised items by dragging them in the backlog list.
3. Created a new sprint with name, start/end dates, and a sprint goal.
4. Dragged selected issues into the sprint container.
5. Clicked "Start Sprint" – items moved to the active Scrum board.
6. Verified the sprint showed an "ACTIVE" badge in the backlog.

Screenshots

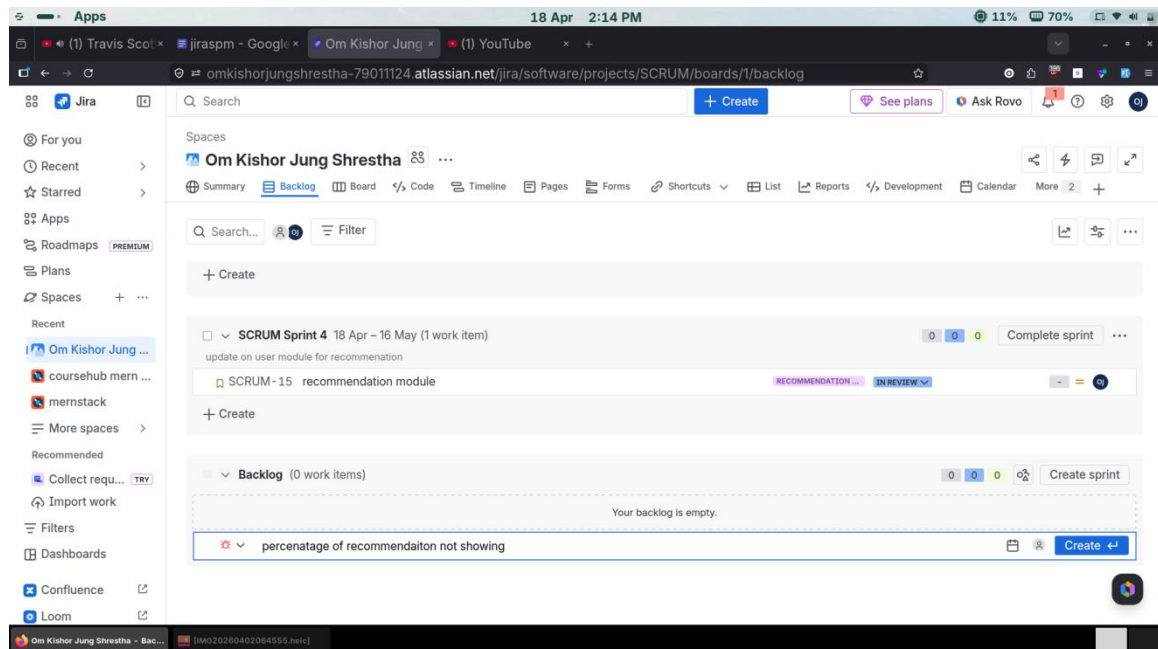


Figure 56: Lab 8 – Backlog re-prioritisation

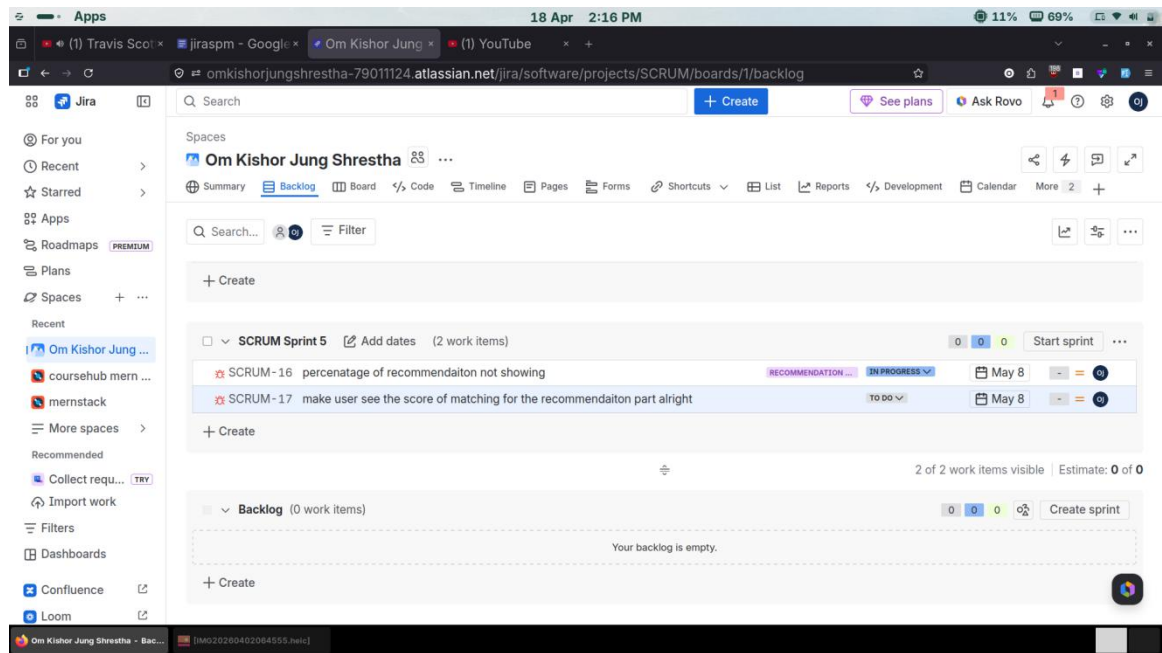


Figure 57: Lab 8 – New sprint container in backlog

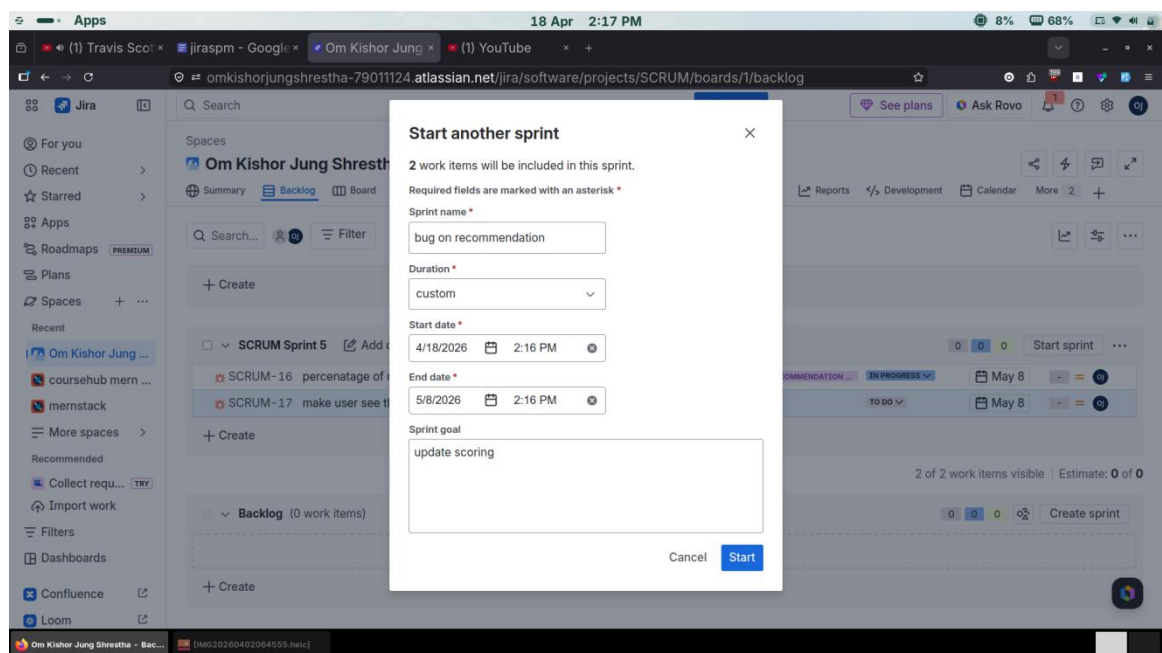


Figure 58: Lab 8 – Sprint goal and dates dialog

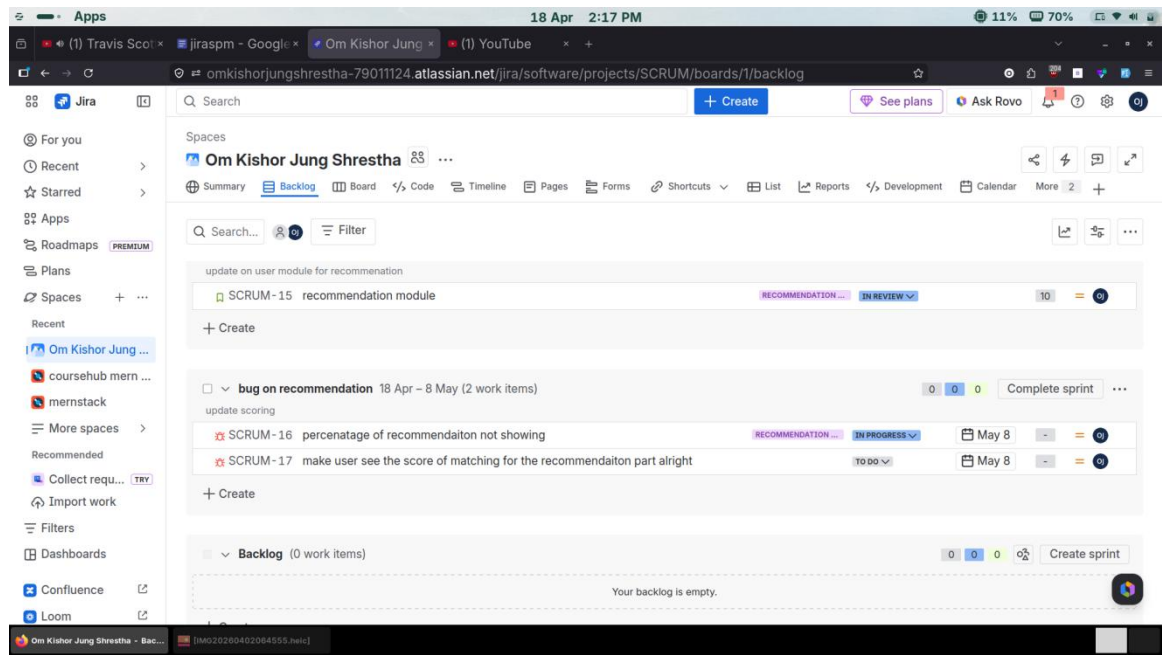


Figure 59:Lab 8 – Issues assigned to sprint

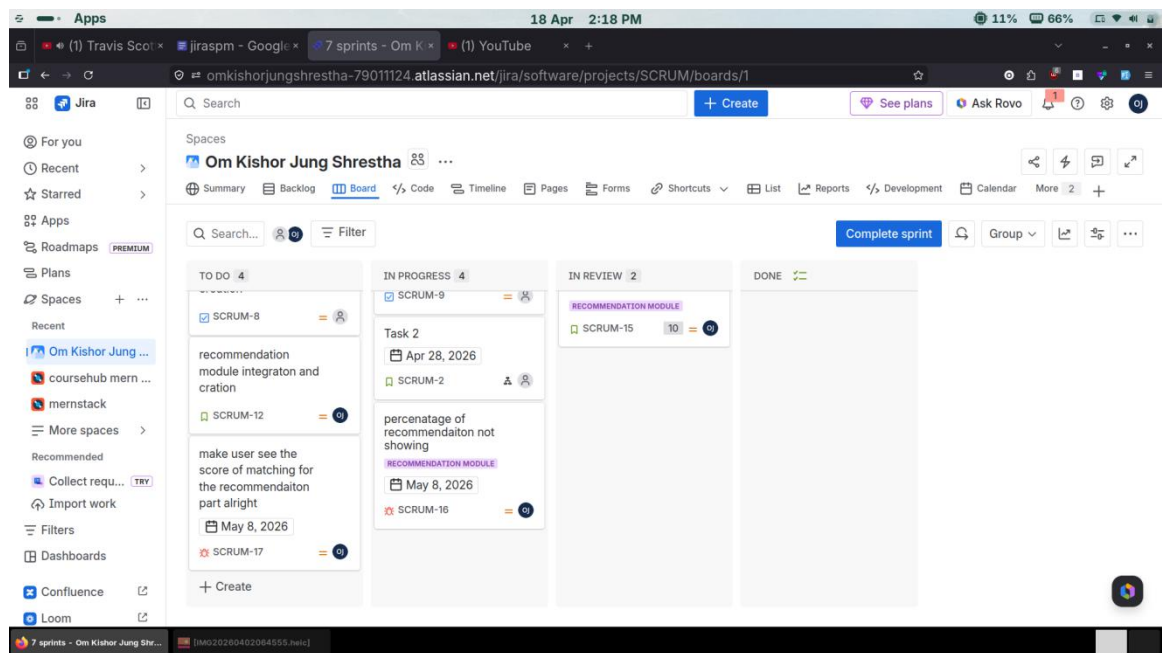


Figure 60:Lab 8 – Active sprint badge on backlog

Conclusion: The backlog was organised and a sprint was created with committed issues. The sprint board shows the active work items.

Lab 9 – Bug Issue in Jira

Objective: To report, track, and resolve a Bug using the dedicated Bug issue type in Jira.

A Bug represents a software defect. It is tracked with priority, severity, assignee, and status fields.

Steps Performed

1. Clicked "Create" and selected issue type "Bug".
2. Entered Summary: "Login button unresponsive on mobile devices".
3. Added Description with: Steps to Reproduce, Expected Behaviour, Actual Behaviour.
4. Set Priority (High), Assignee, and linked the bug to a relevant Story.
5. Added Label "bug" and set the "Affects Version" field.
6. Tracked the bug through: To Do → In Progress → Done.
7. Added resolution note "Fixed" and closed the issue.

Screenshots

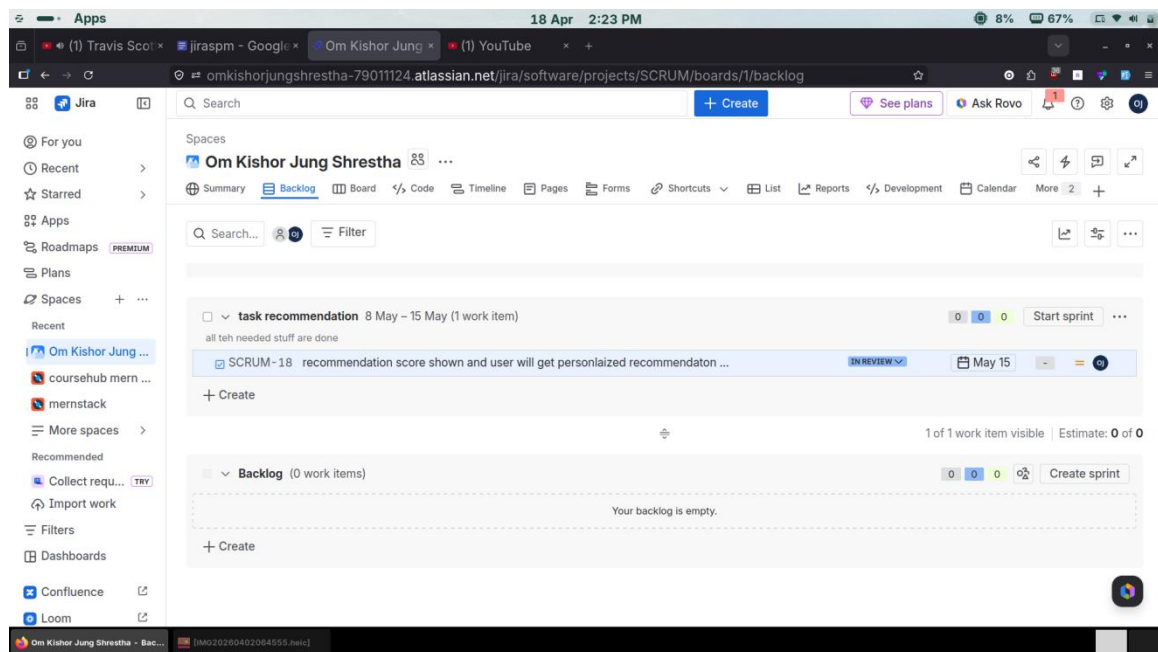


Figure 61: Lab 9 – Create bug form

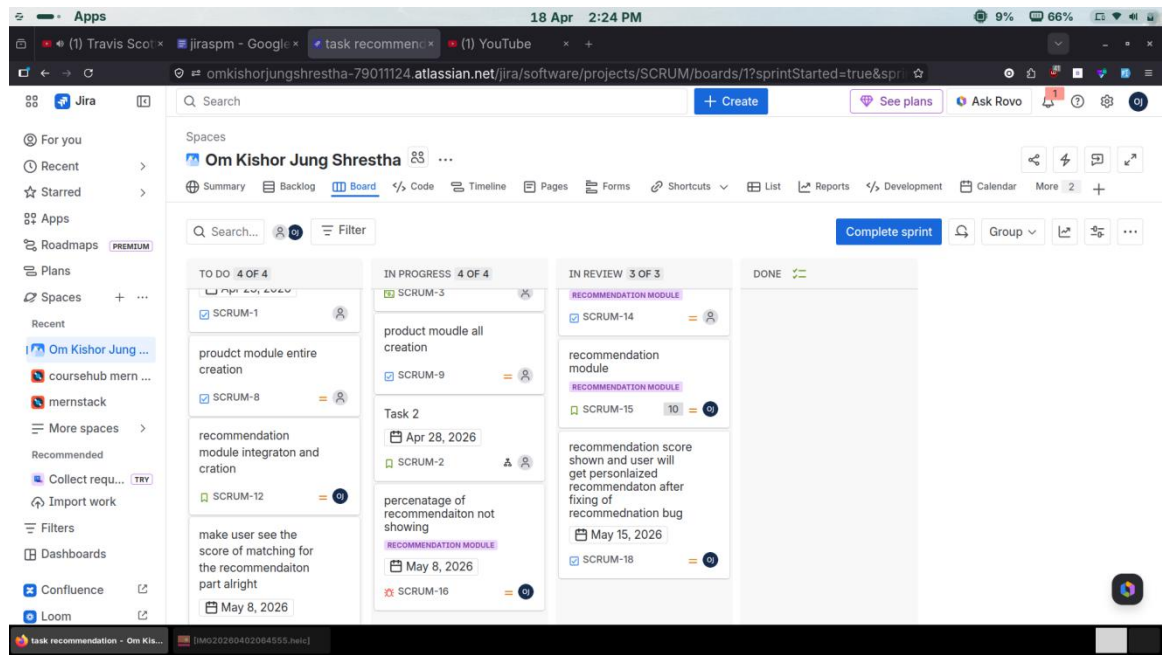


Figure 62: Lab 9 – Bug with priority and assignee

Conclusion: A bug was reported with full reproduction details and tracked from creation to resolution using status transitions.

Lab 10 – Task in Jira

Objective: To understand the Task issue type and how it differs from Stories and Bugs.

A Task represents non-user-facing work items such as "Set up CI/CD pipeline" or "Write unit tests". It is used for technical or administrative work.

Steps Performed

1. Clicked "Create" and selected issue type "Task".
2. Entered Summary: "Configure PostgreSQL database for production".
3. Added acceptance criteria, assigned to a team member, and set a due date.
4. Linked the task to the active sprint and added a story point estimate.
5. Moved the task through board columns: To Do → In Progress → Done.
6. Added comments to document progress and any blockers encountered.

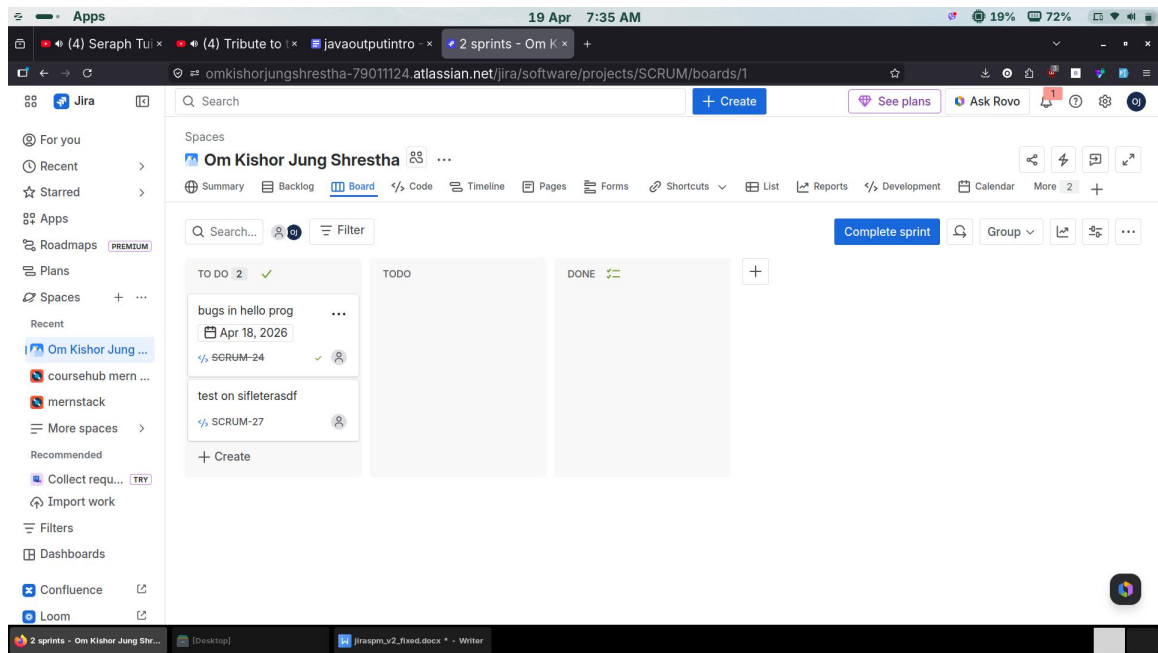


Figure 63:Lab 12 – Issue types configuration page

Figure 64:Lab 12 – Add issue type dialog

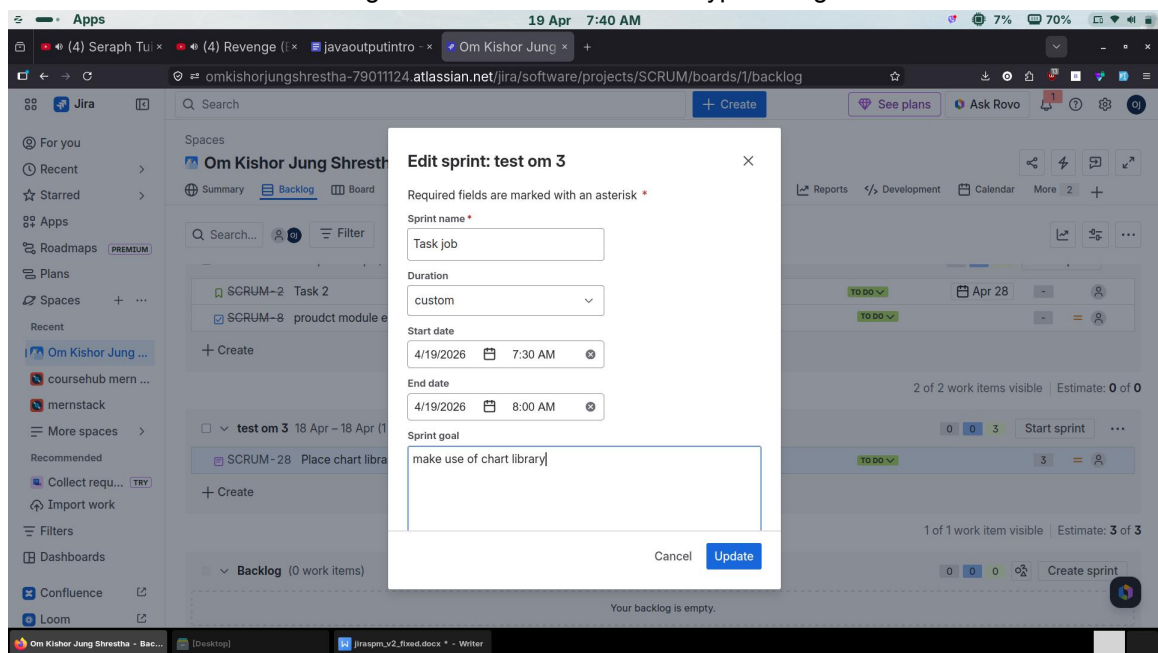


Figure 65:Lab 12 – New issue type in create dropdown

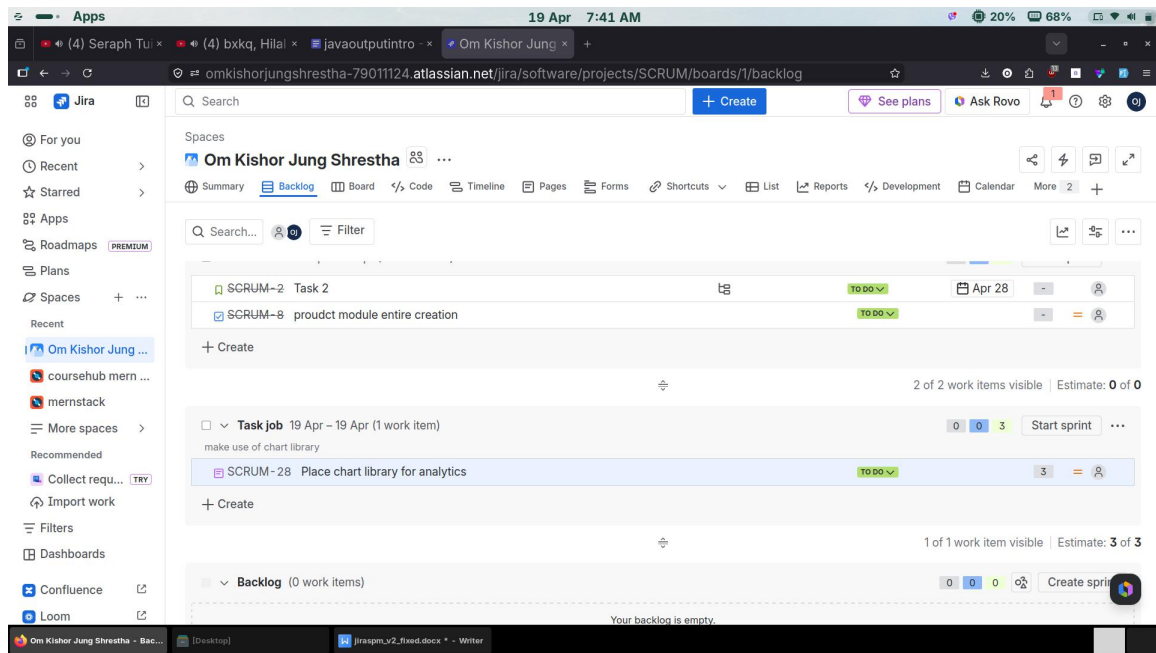
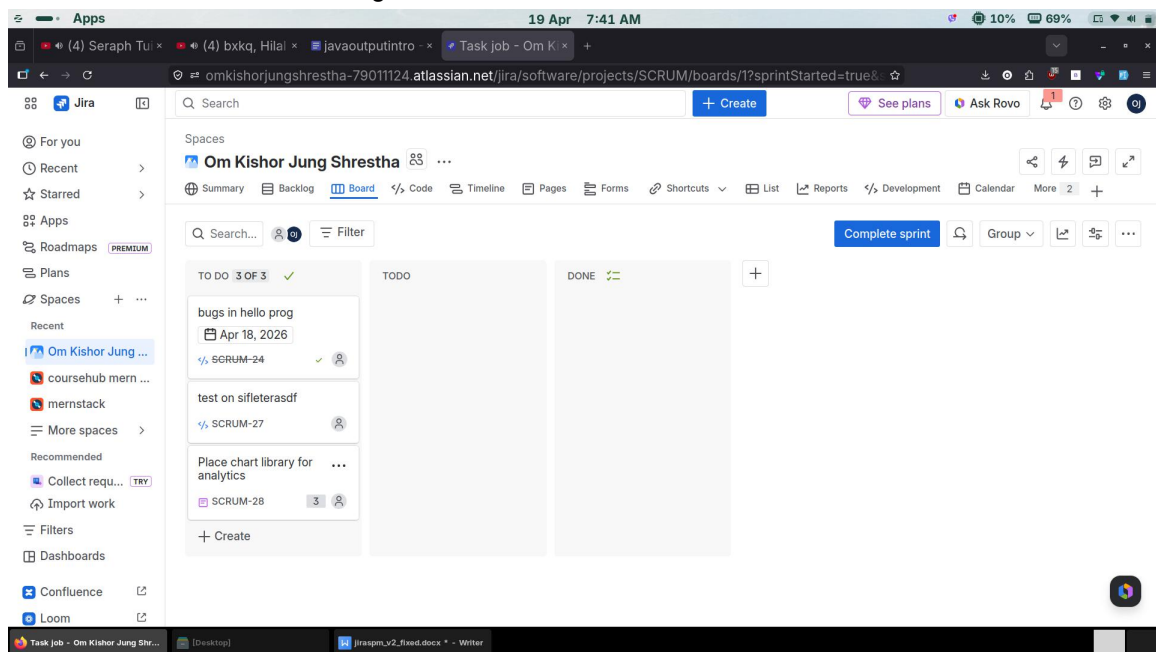


Figure 66: Lab 12 – Labels field on issue



Conclusion: A Task was created and managed in Jira. Tasks are suited for internal technical work that supports stories but is not directly user-facing.

Lab 11 – Search in Jira

Objective: To find and filter issues using Basic Search and Advanced JQL (Jira Query Language).

Basic Search

1. Clicked the search icon and typed a keyword (e.g., "login") to see matching issues.
2. Applied filters: Project, Issue Type, Status, Assignee, Priority.
3. Saved the filter for future reuse via "Save as".

Advanced Search – JQL Examples

project = "MyProject" AND issuetype = Bug AND status = "In Progress"

assignee = currentUser() AND sprint in openSprints()

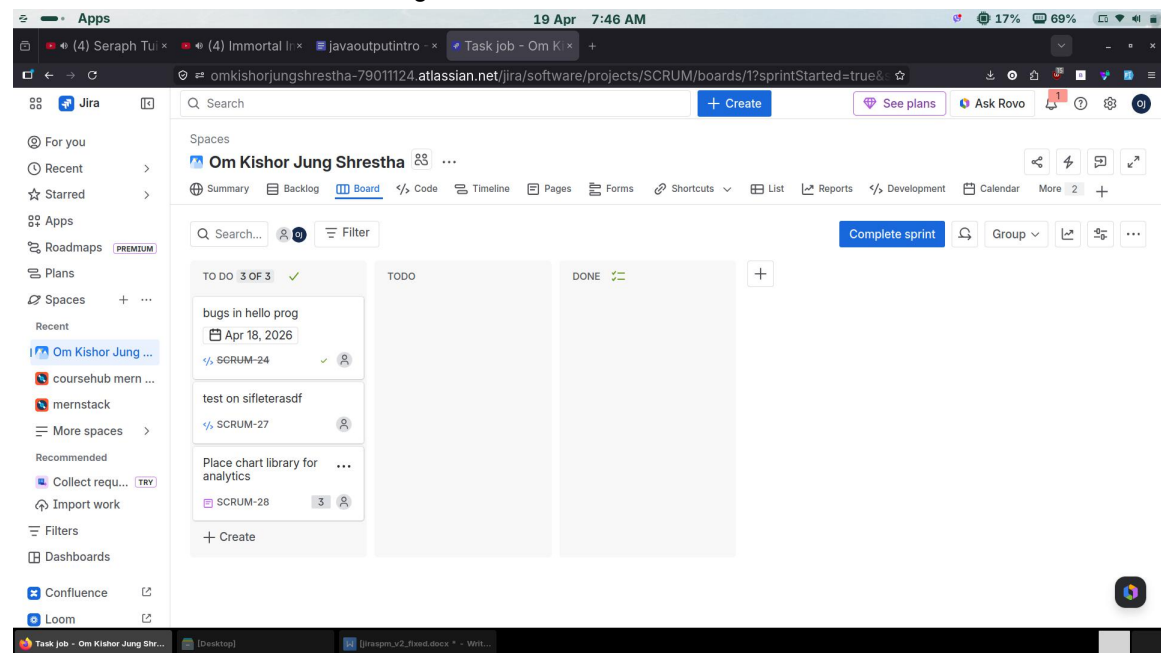
created >= -7d AND priority = High ORDER BY created DESC

Steps: Clicked "Advanced" to switch to JQL mode, typed the query, and exported results to CSV.

Conclusion: Both Basic and Advanced JQL search were used to locate issues. JQL provides flexible, powerful querying across large projects.

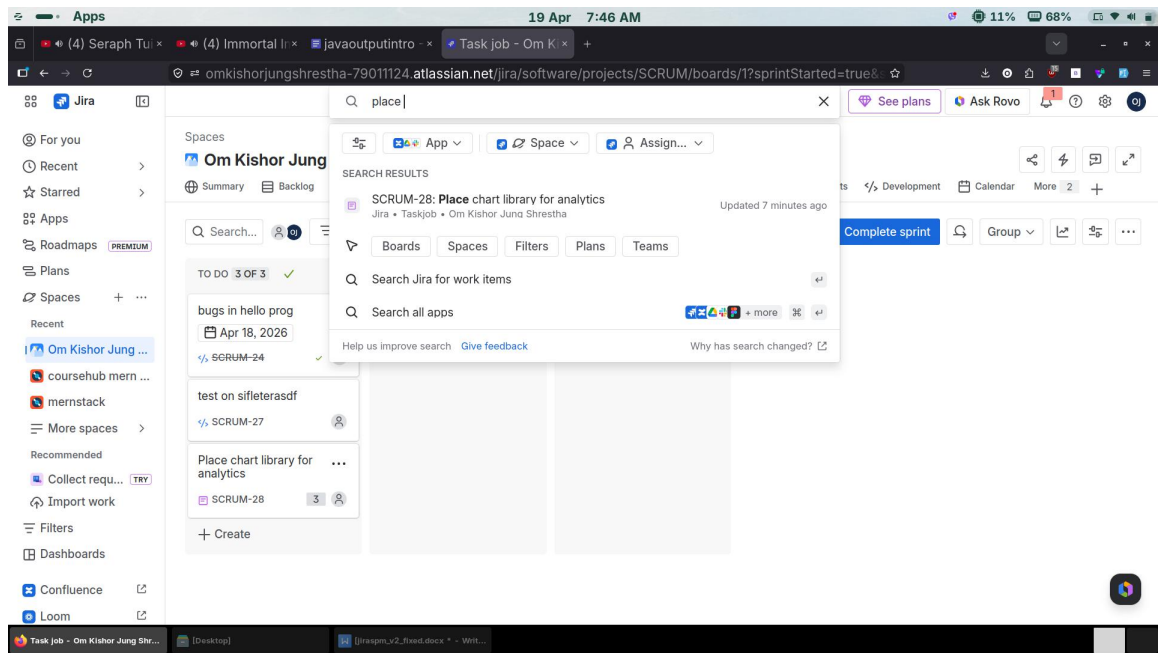
Baic Search example:

Figure 67:Lab 12 – Label creation



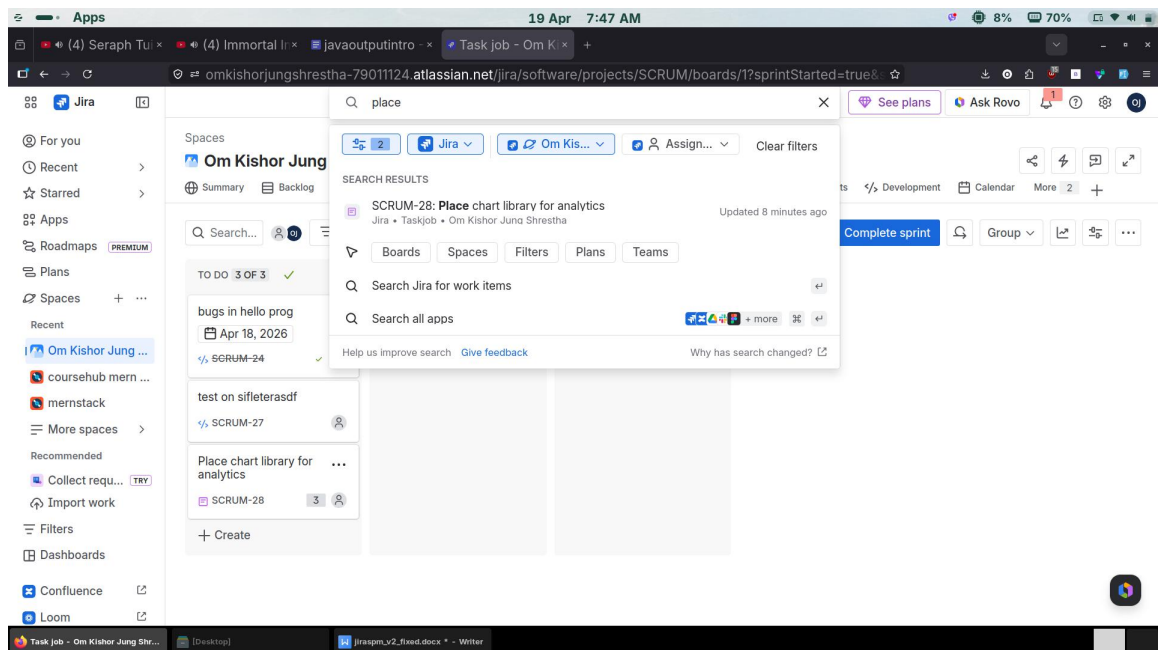
search for place:

Figure 68:Lab 12 – Label filter in search



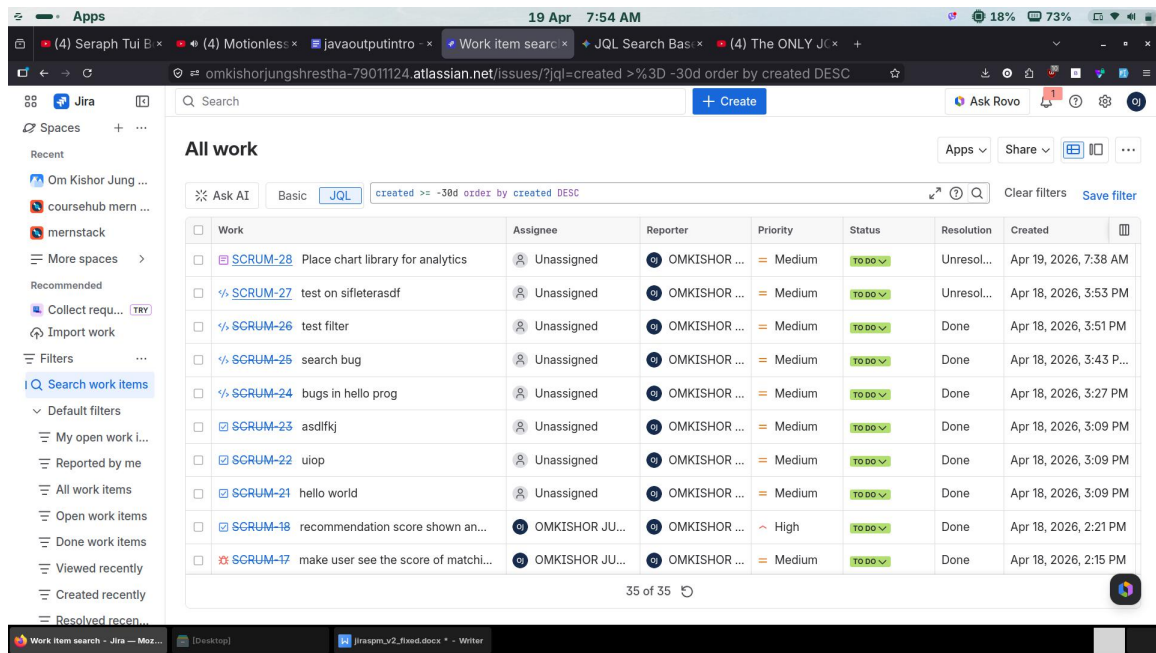
with filters:

Figure 69: Lab 12 – Issues filtered by label



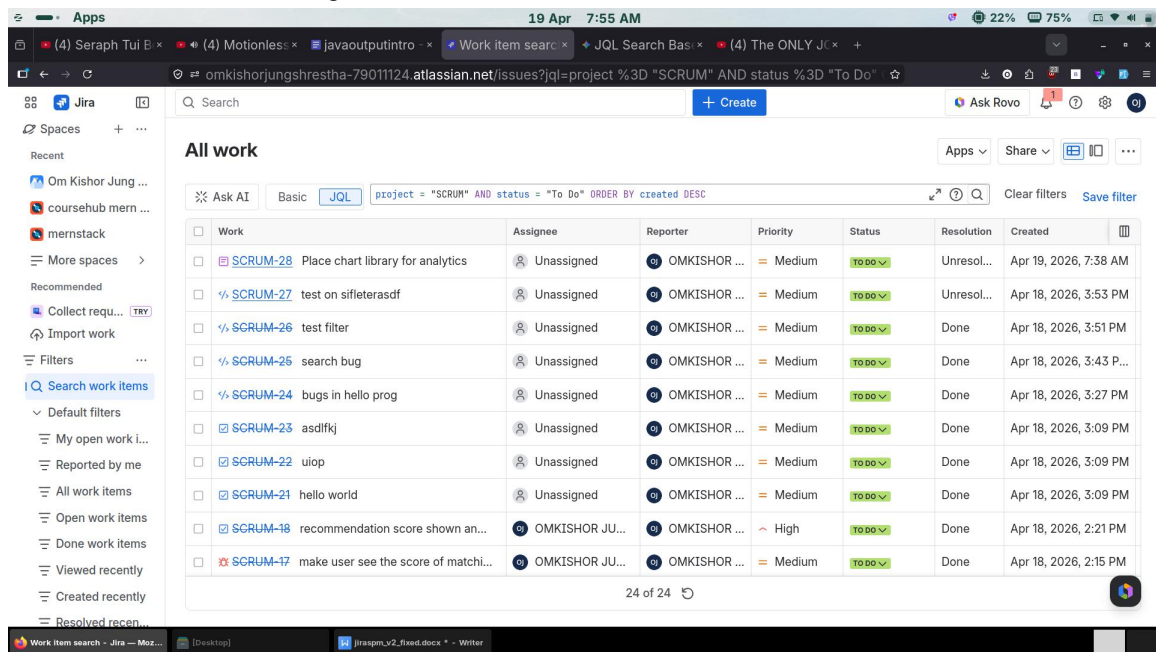
Advanced search: Jql:
from filter

Figure 70: Lab 13 – GitHub for Jira marketplace app



jql command added in the box in jql editor and hit search:

Figure 71:Lab 13 – GitHub authorisation screen



Lab 12 – Adding New Issue Type and Label

Objective: To customise the Jira project with a new issue type and apply labels for issue categorisation.

Adding a New Issue Type

1. Went to Project Settings → Issue Types and clicked "Add issue type".
2. Named it "Feature Request", added a description, selected an icon and colour.
3. Saved and confirmed the new type appeared in the "Create" issue dropdown.

Adding Labels

1. Opened an existing Story and located the "Labels" field.
2. Typed a new label name (e.g., "frontend", "api") and created/selected it.
3. Used the Label filter in Basic Search to find all issues with a given label.

Screenshots

Figure 72: Lab 12 – issues

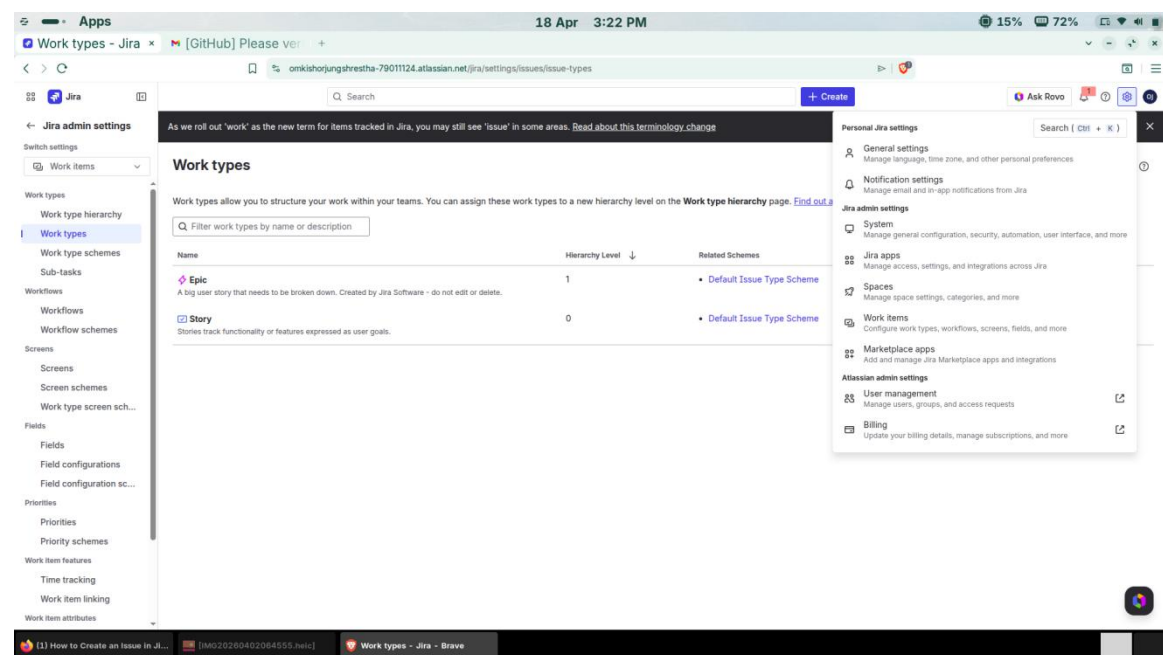


Figure 73: Lab 12 – Development panel on Jira issue

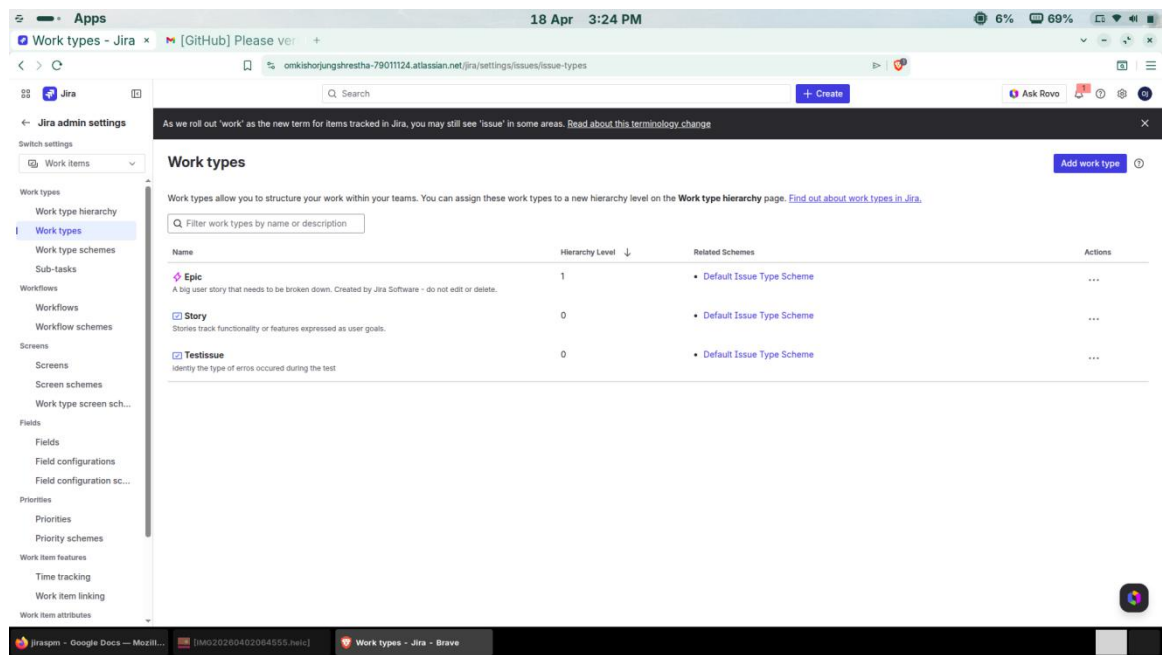


Figure 74:Lab 12 – issues linked

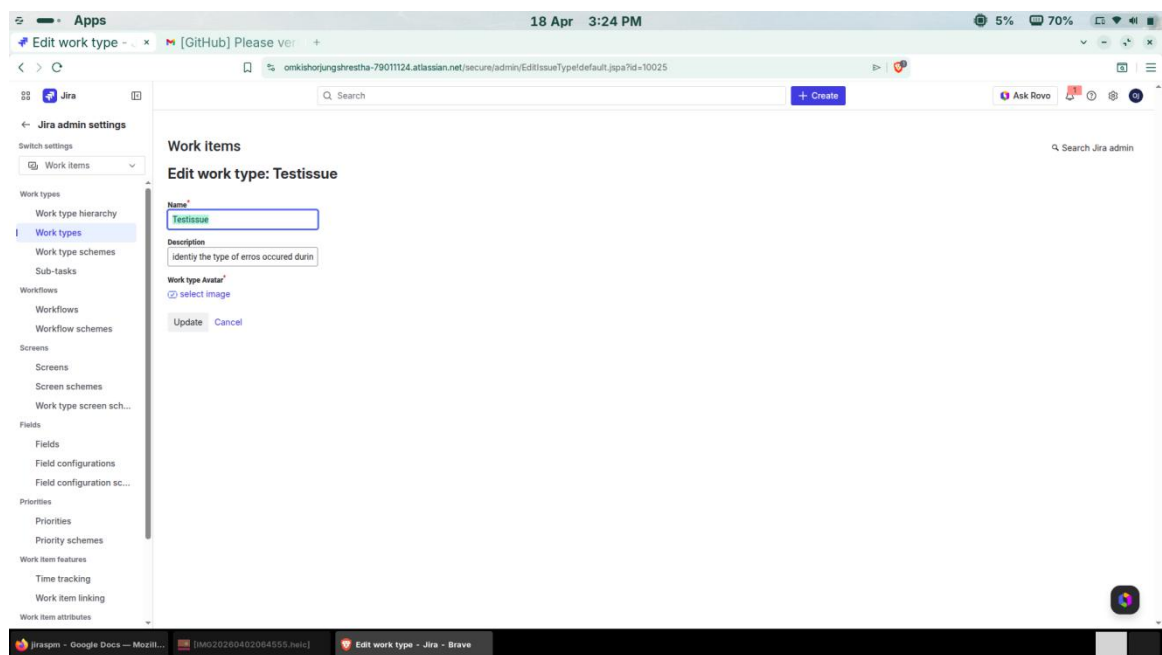


Figure 75:Lab 12 – issues cration

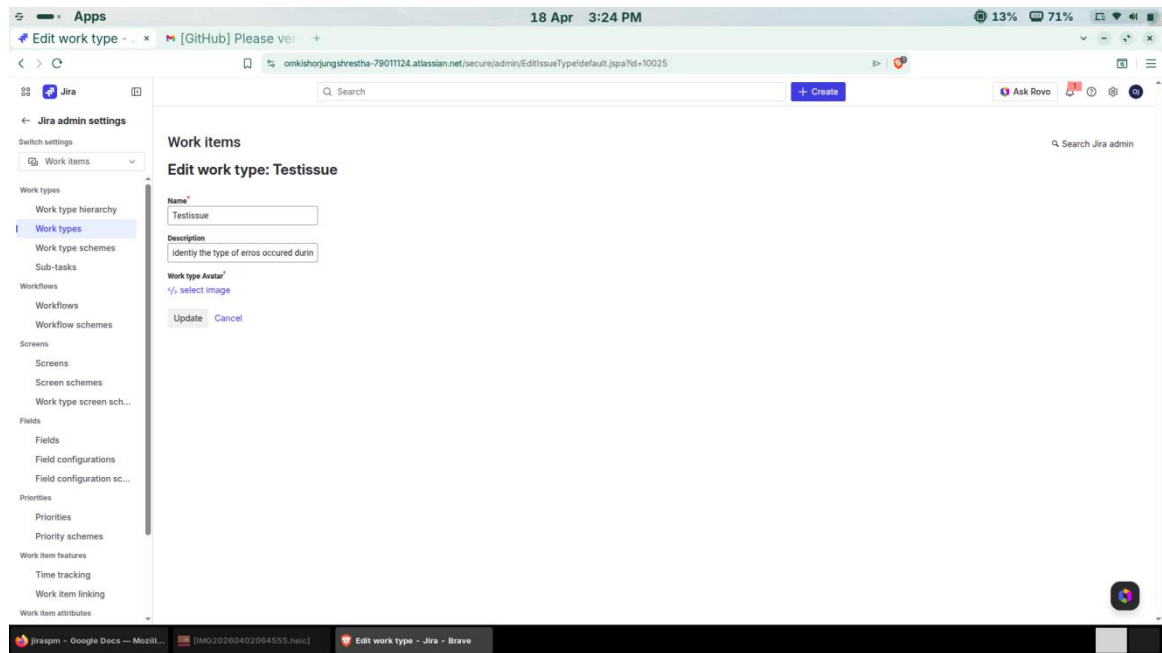


Figure 76:Lab 12 –issue creation

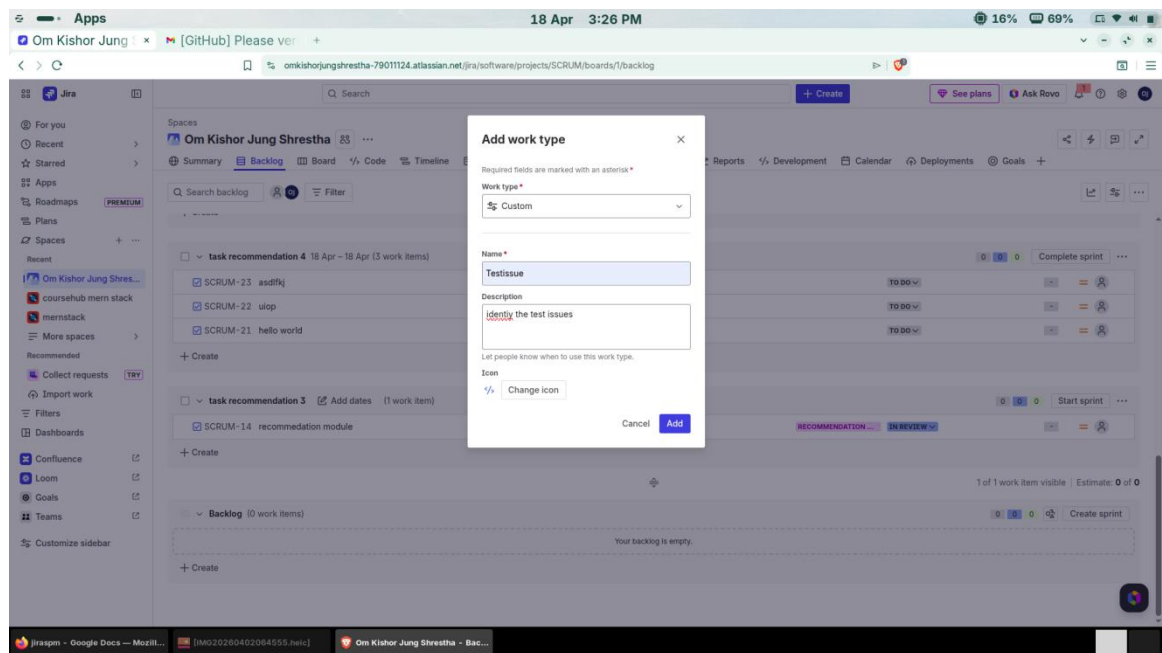


Figure 77:Lab 12 – form filled for issue

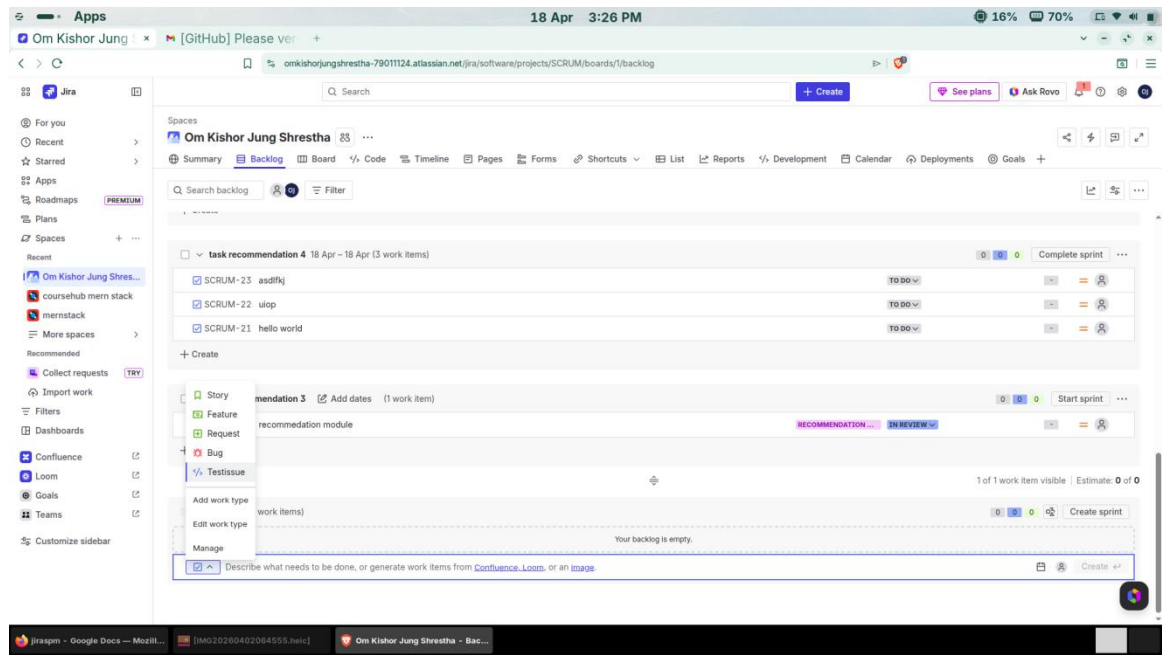


Figure 78:Lab 12 – issue created well

Spaces

 **Om Kishor Jung Shrestha**

 Summary  Backlog  **Board**

   Filter

TO DO 6 OF 6

Task 1

 Apr 23, 2026

☒ SCRUM-1 

proudct module entire creation

☒ SCRUM-8  

asdlfkj

☒ SCRUM-23  

uiop

☒ SCRUM-22  

hello world

☒ SCRUM-21  

bugs in hello prog ...

 Apr 18, 2026

 SCRUM-24 

 Create

Figure 79:lab 12 issue seen

Conclusion: A custom issue type was added and labels were applied. Labels provide flexible cross-cutting classification outside the standard issue hierarchy.

Lab 13 – Connecting Git and Creating Dashboard

Objective: To integrate a GitHub repository with Jira and create a project health dashboard.

Connecting Git (GitHub Integration)

1. Went to Project Settings → Apps → GitHub for Jira and clicked "Connect GitHub account".
2. Authorised Atlassian to access GitHub and selected repositories to link.
3. In a GitHub commit message, referenced a Jira issue key (e.g., "PROJ-42: Fixed login bug").
4. Confirmed the commit, branch, and PR appeared in the Jira issue's "Development" panel.
5. Created a branch directly from Jira using the "Create branch" button.

Creating a Dashboard

1. Clicked "Dashboards" → "Create Dashboard", entered name and sharing settings.
2. Added gadgets: Assigned to Me, Sprint Health, Activity Stream, Issue Statistics.
3. Arranged gadgets using drag-and-drop and saved the dashboard.
4. Set the new dashboard as the default home page.

Screenshots

Figure 80:Lab 13 – Assigned to Me gadget

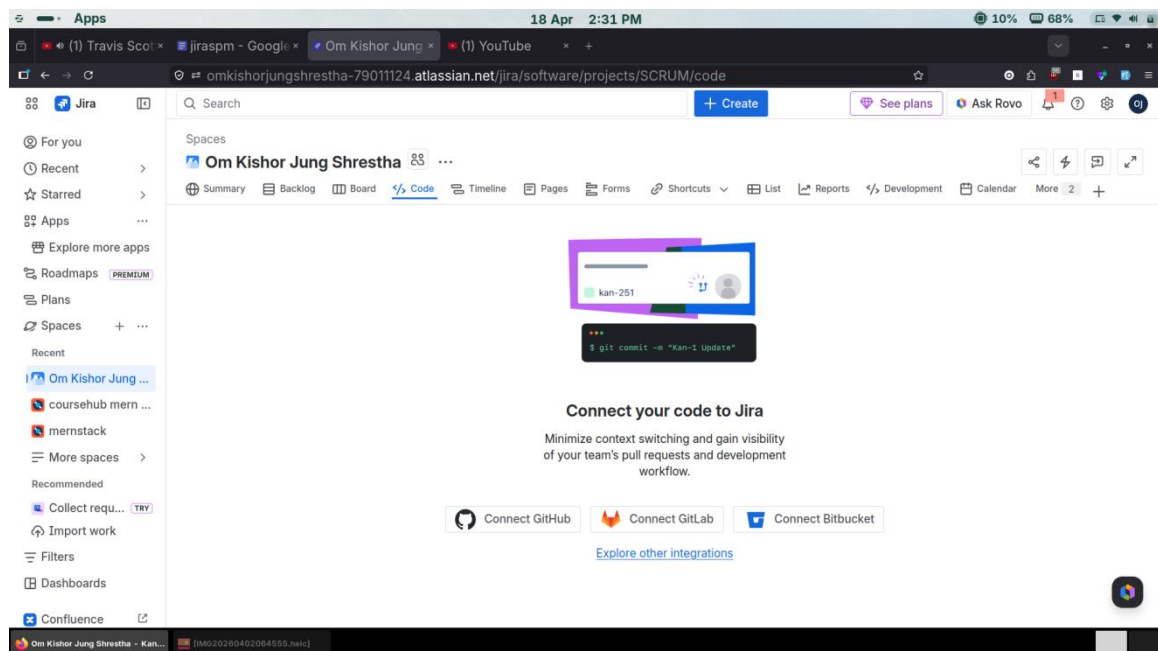


Figure 81:Lab 13 – Sprint Health gadget

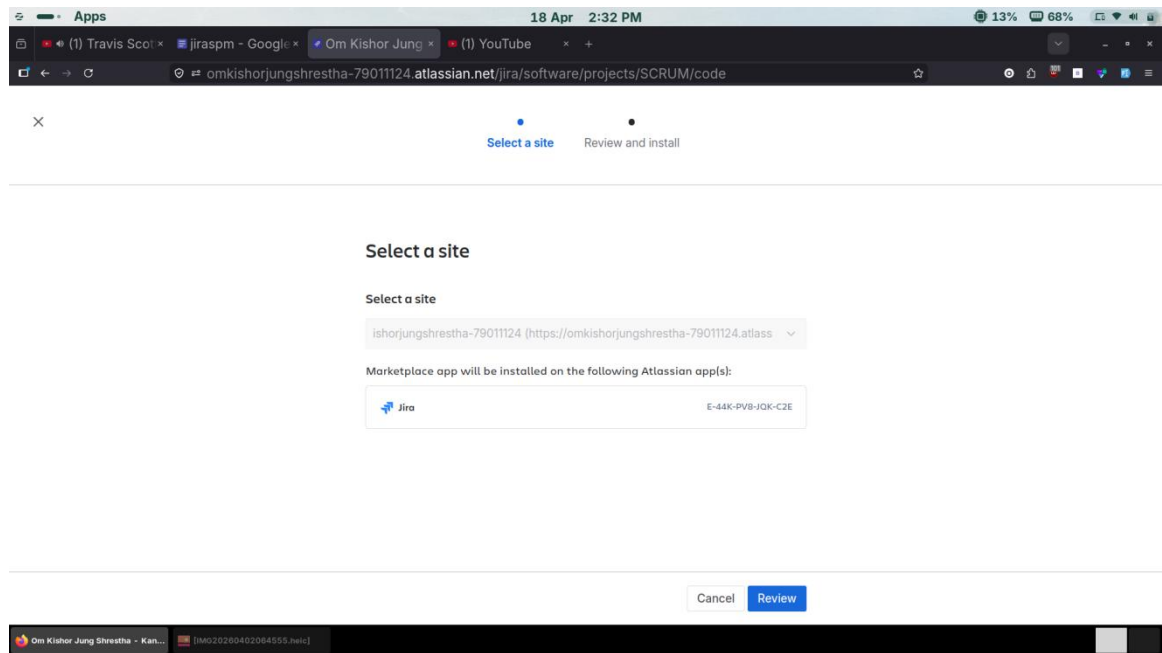


Figure 82:Lab 13 – Activity Stream gadget

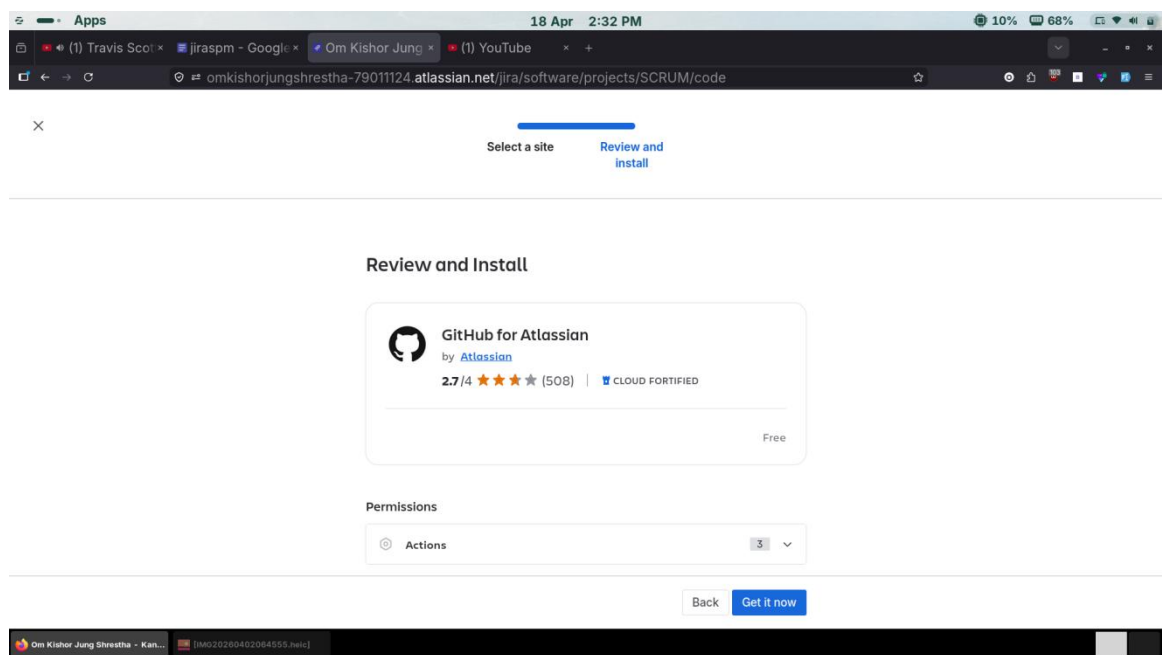


Figure 83:Lab 13 – Issue Statistics pie chart

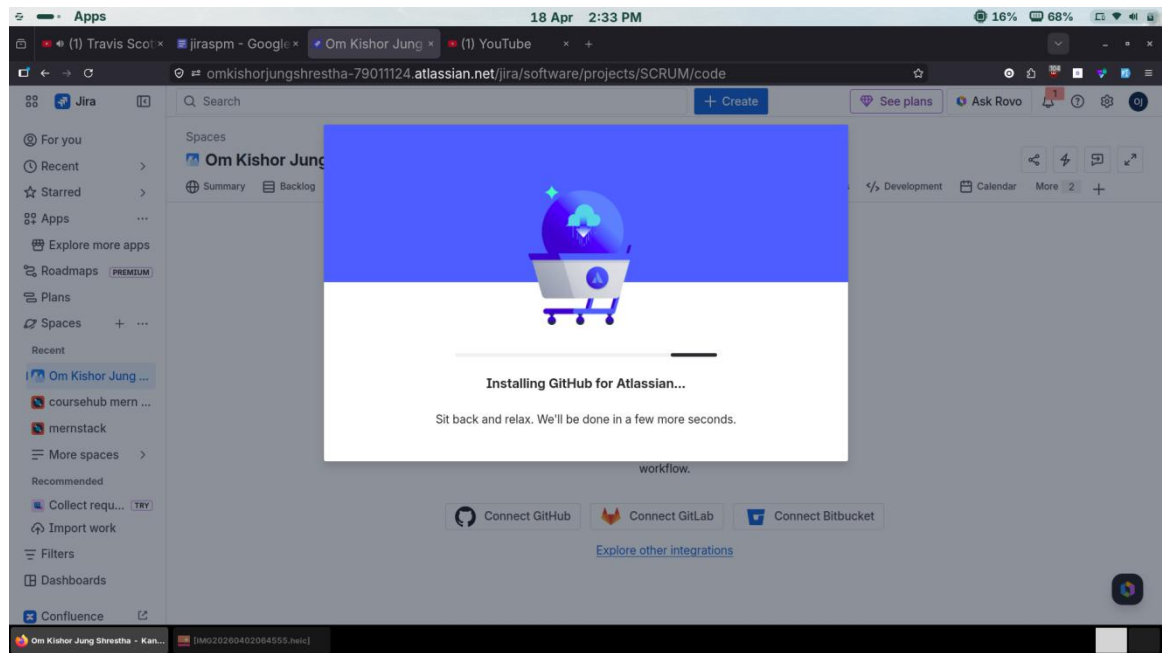


Figure 84:Lab 13 – Dashboard layout view

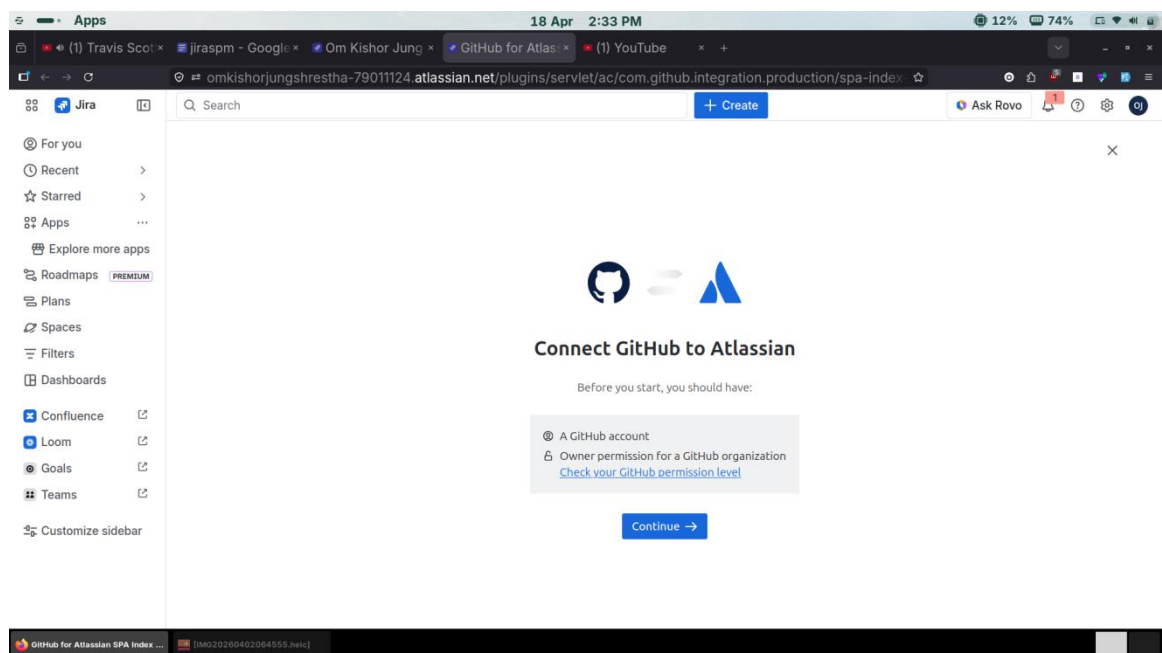


Figure 85:Lab 13 – Dashboard – full populated view

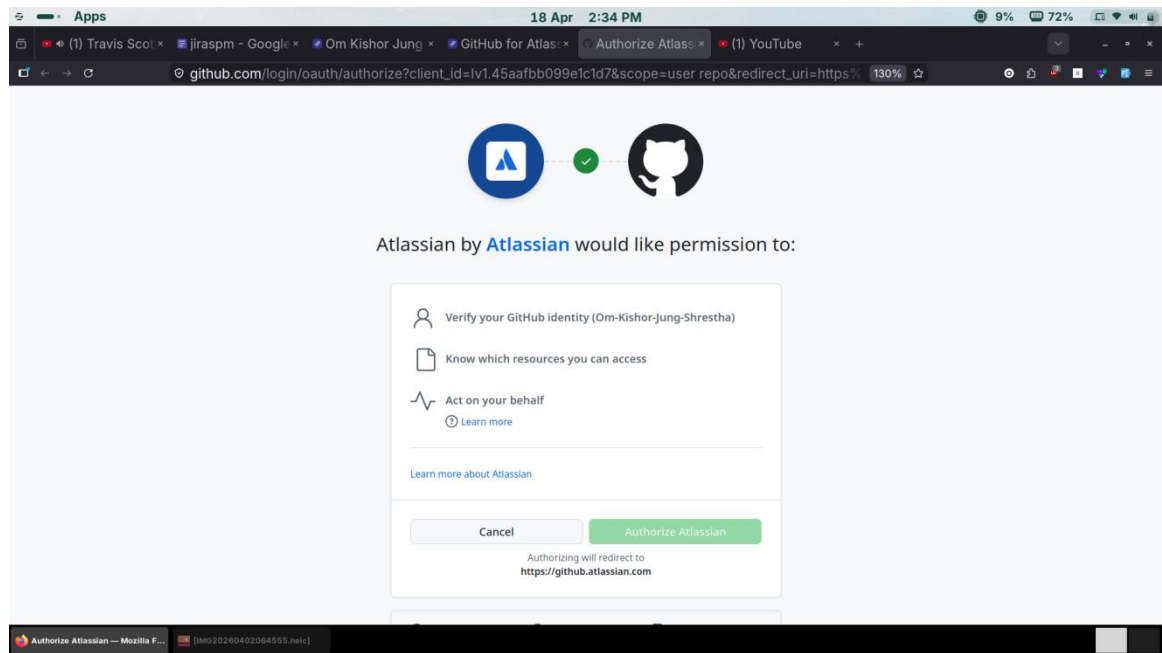


Figure 86:Lab 13 – Dashboard set as home page

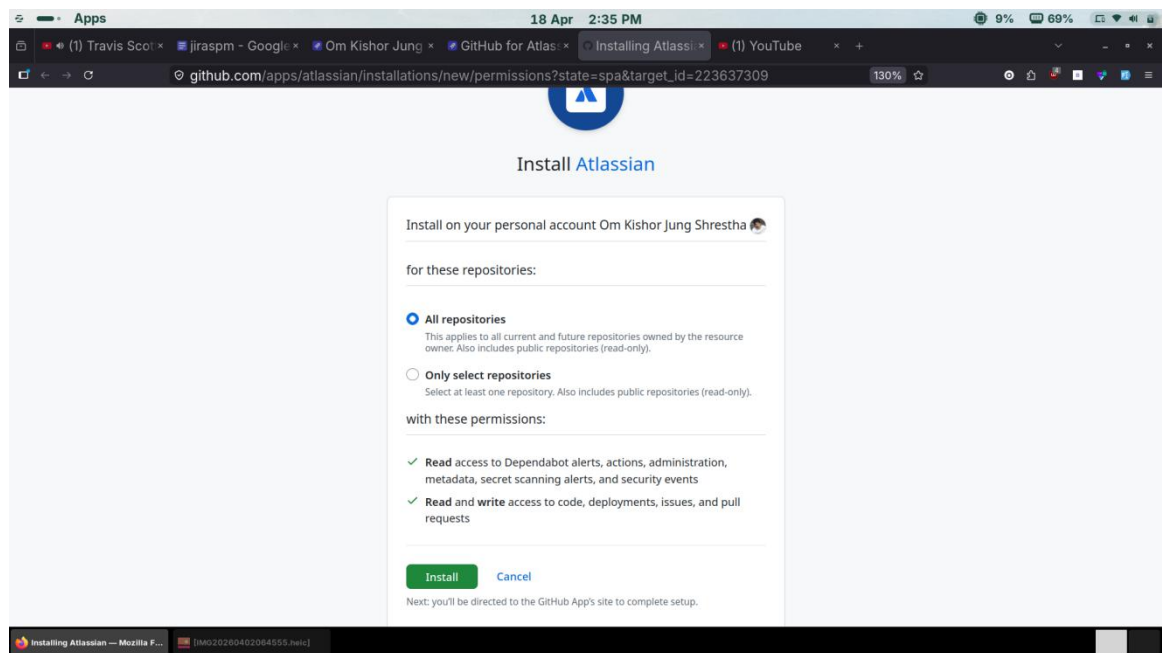


Figure 87:Lab 13 – Issue Statistics by priority

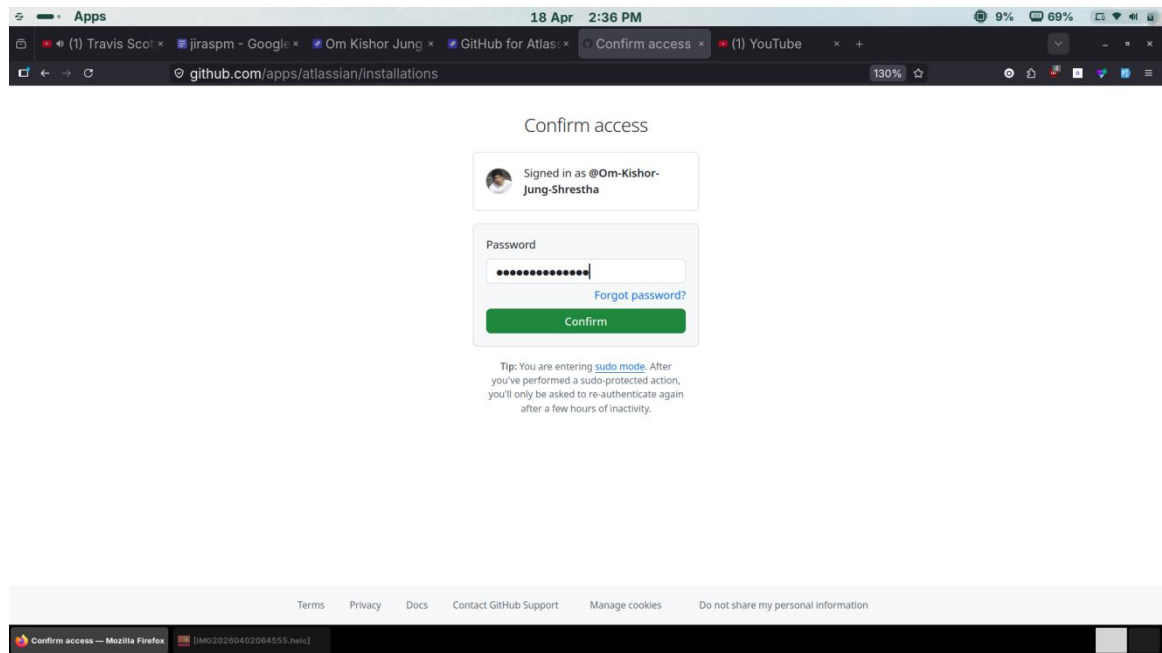


Figure 88:Lab 13 – GitHub commit log in Jira

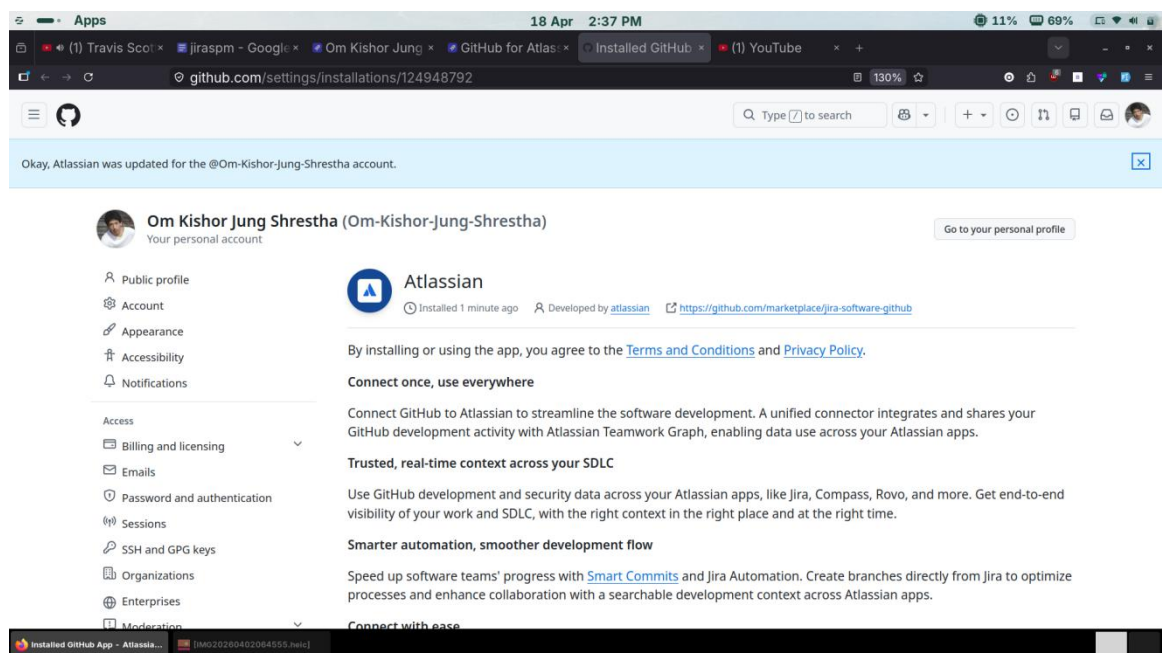


Figure 89:Lab 13 – Branch list on issue

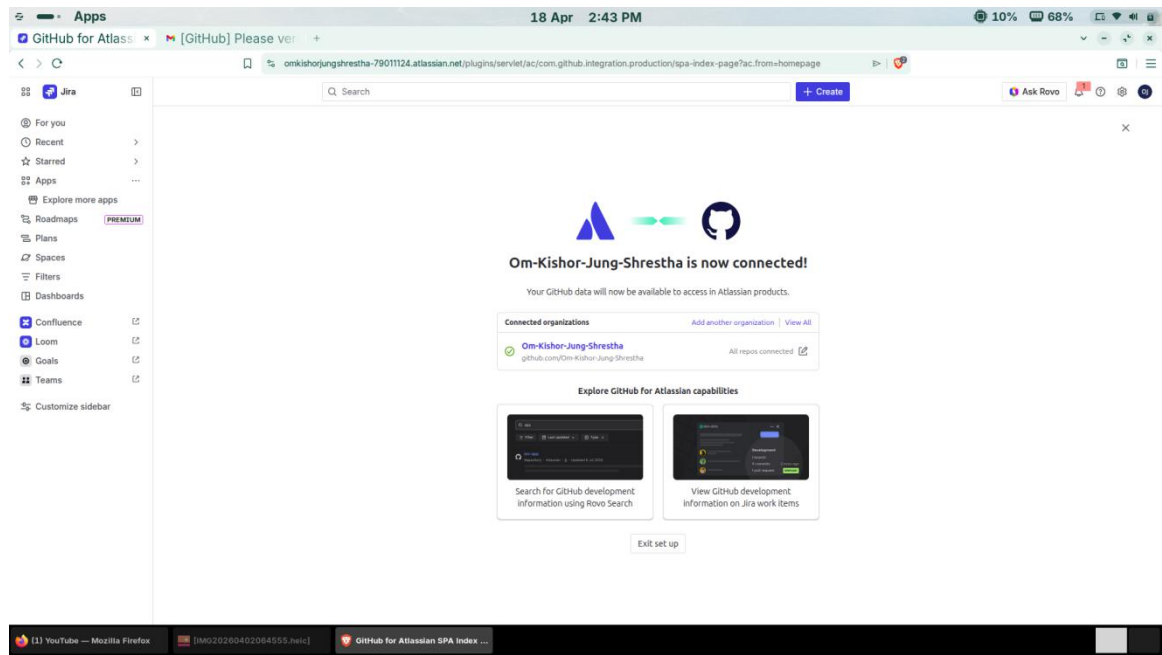


Figure 90:Lab 13 – Dashboard gadget arrangement

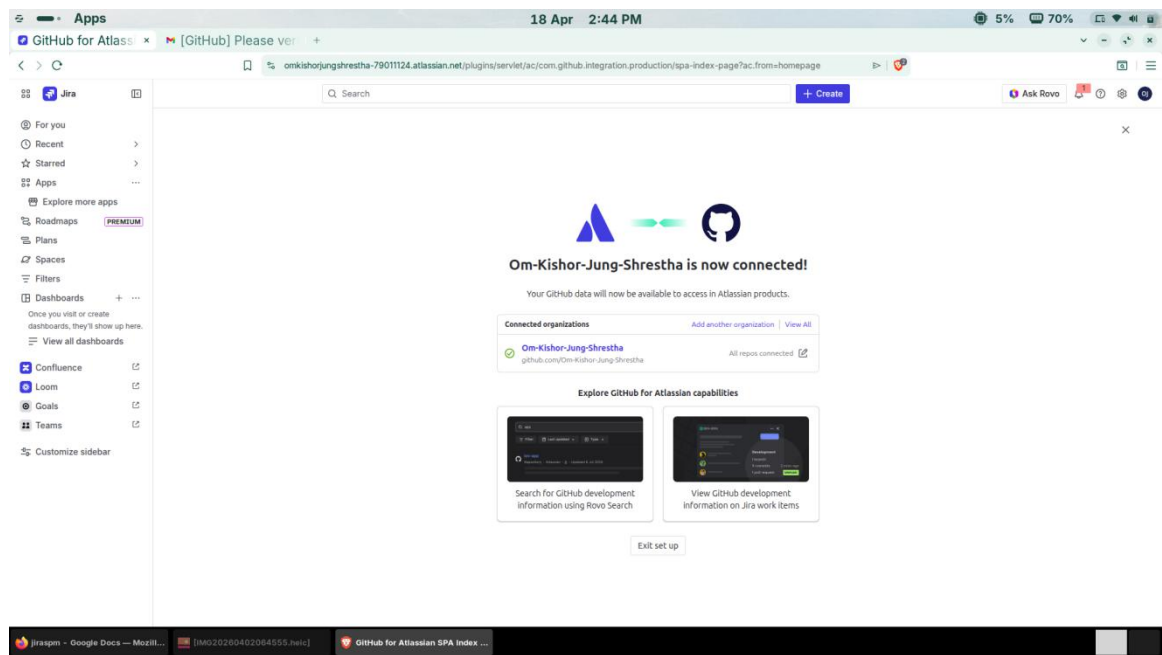


Figure 91:Lab 13 – Final dashboard overview

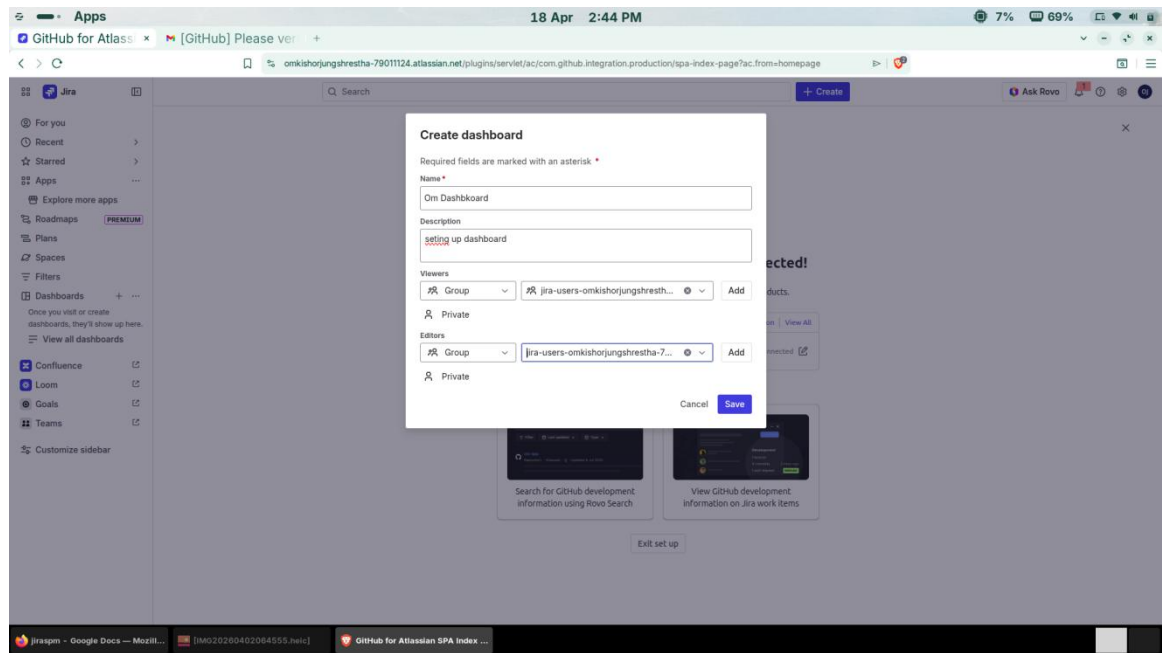


Figure 92:Lab 13 – Dashboard sharing settings

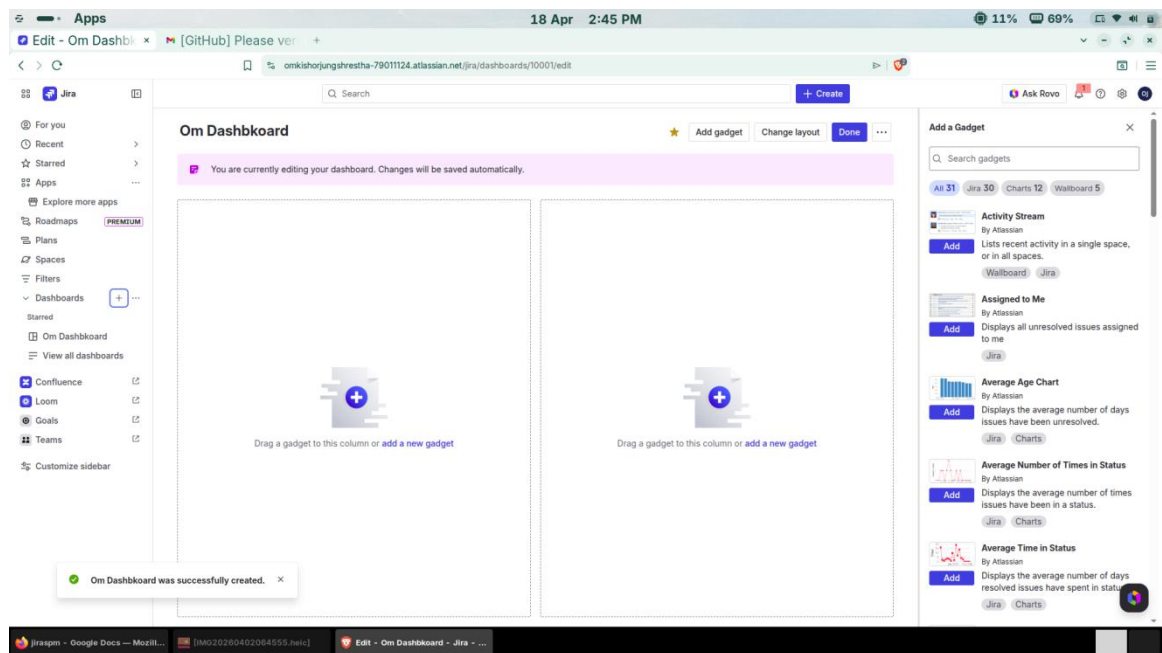


Figure 93:Lab 15 – Project Settings – Features panel

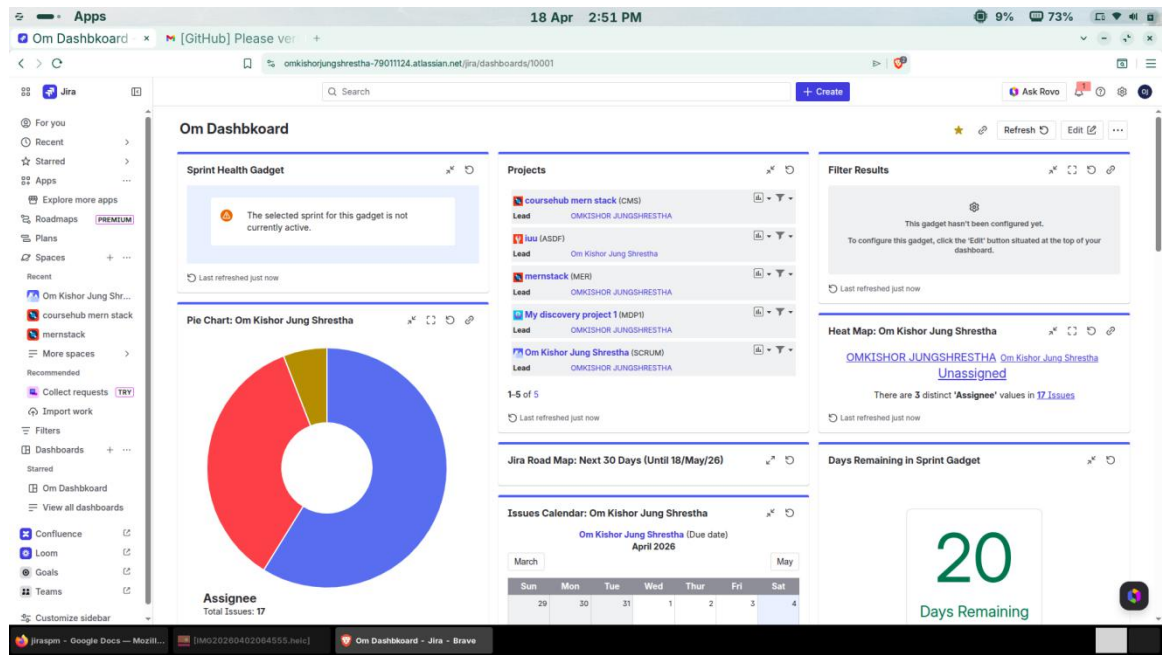


Figure 94:Lab 15 – Features toggles enabled

Conclusion: GitHub was integrated with Jira so commits reference issues automatically. A dashboard with multiple gadgets was created for real-time project monitoring.

Lab 14 – Adding Field in Issue Template and Issue Workflow

Objective: To add a custom field to the issue screen and configure a custom workflow with additional statuses.

Adding a Custom Field

1. Went to Project Settings → Issue Types → Story → Edit fields.
2. Created a "Text Field (single line)" named "Test Environment".
3. Added it to the Create, Edit, and View screens.
4. Created a new Story and confirmed "Test Environment" appeared on the form.

Configuring Issue Workflow

1. Went to Project Settings → Workflows → Edit the active workflow.
2. Added a new status "In Review" between "In Progress" and "Done".
3. Added transitions: "Submit for Review" (In Progress → In Review) and "Approve" (In Review → Done).
4. Added a condition requiring "Fix Version" to be set before closing.
5. Published the workflow and verified "In Review" appeared as a board column.

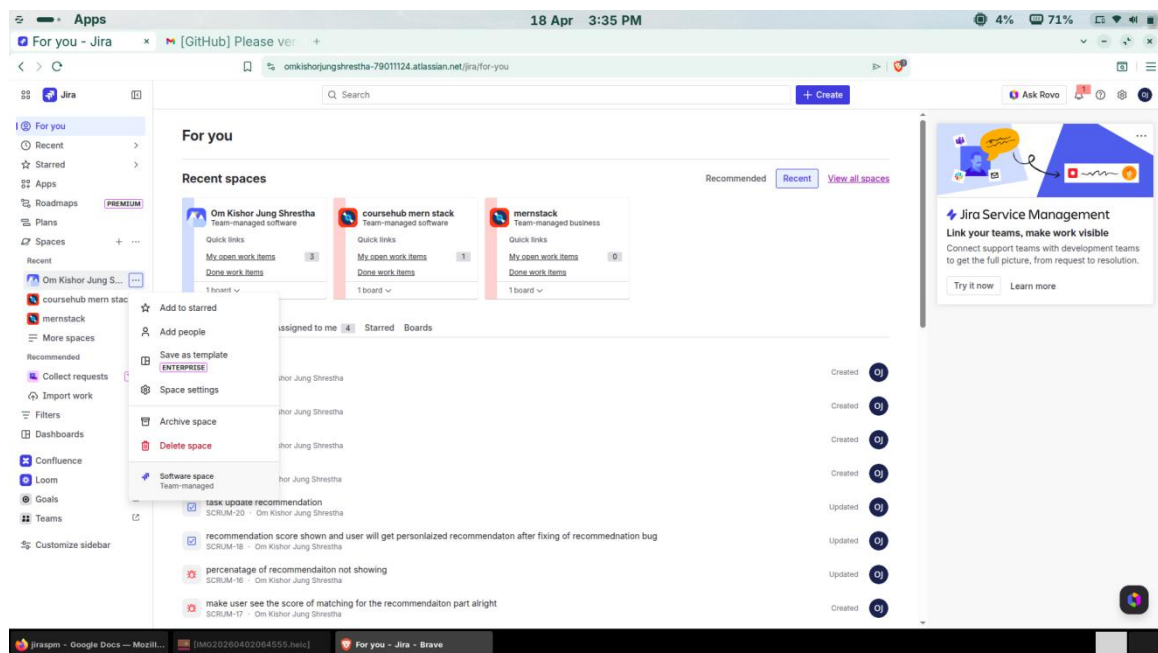


Figure 95:Lab 15 – Releases and Components in sidebar

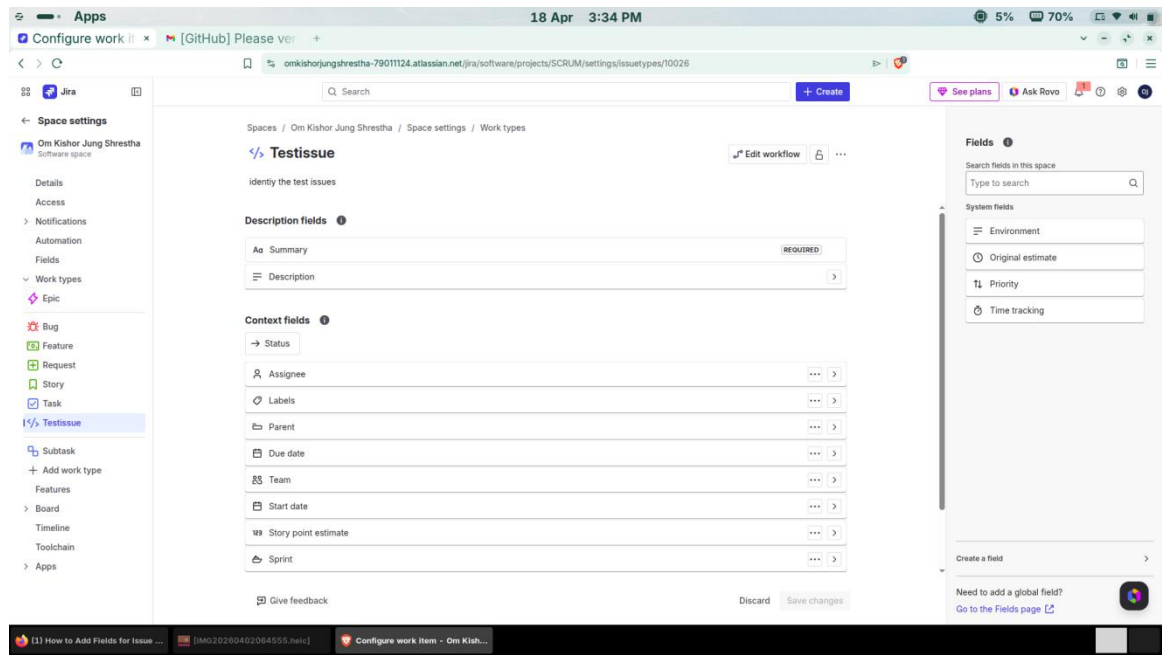


Figure 96:Lab 17 – Burndown chart – ideal vs actual

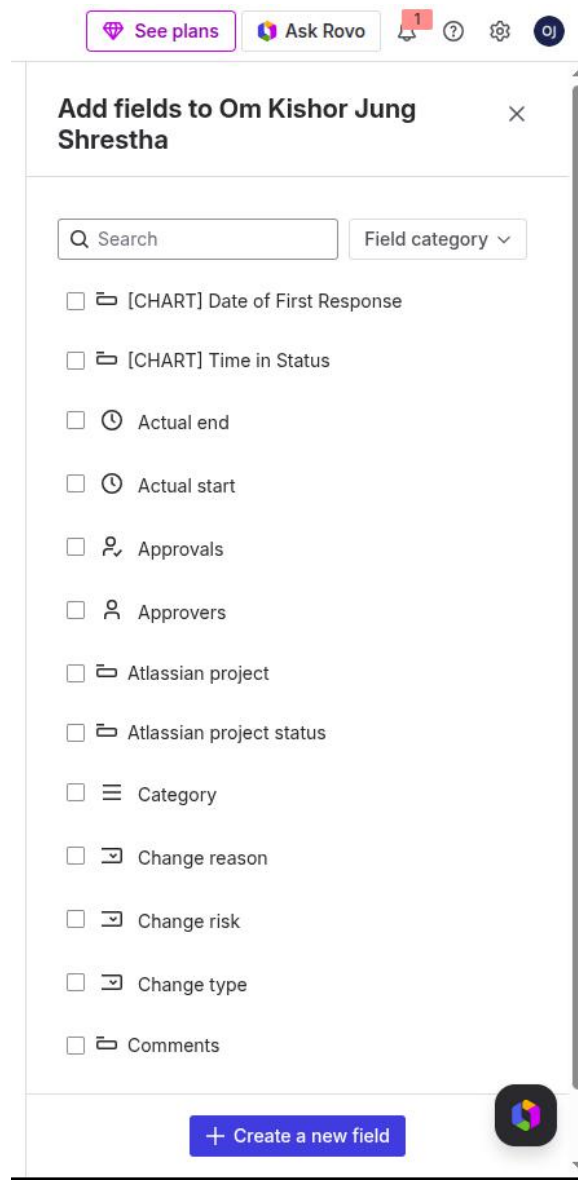
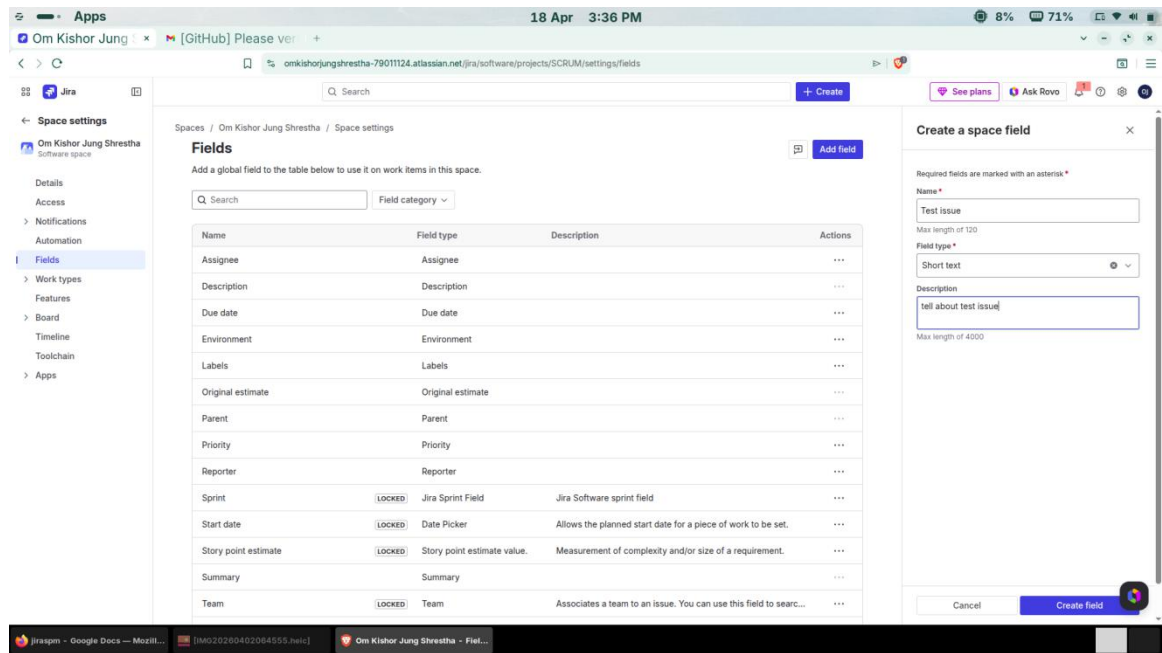


Figure 97:Lab 17 – Burndown chart – sprint details



Spaces / Om Kishor Jung Shrestha / Space settings

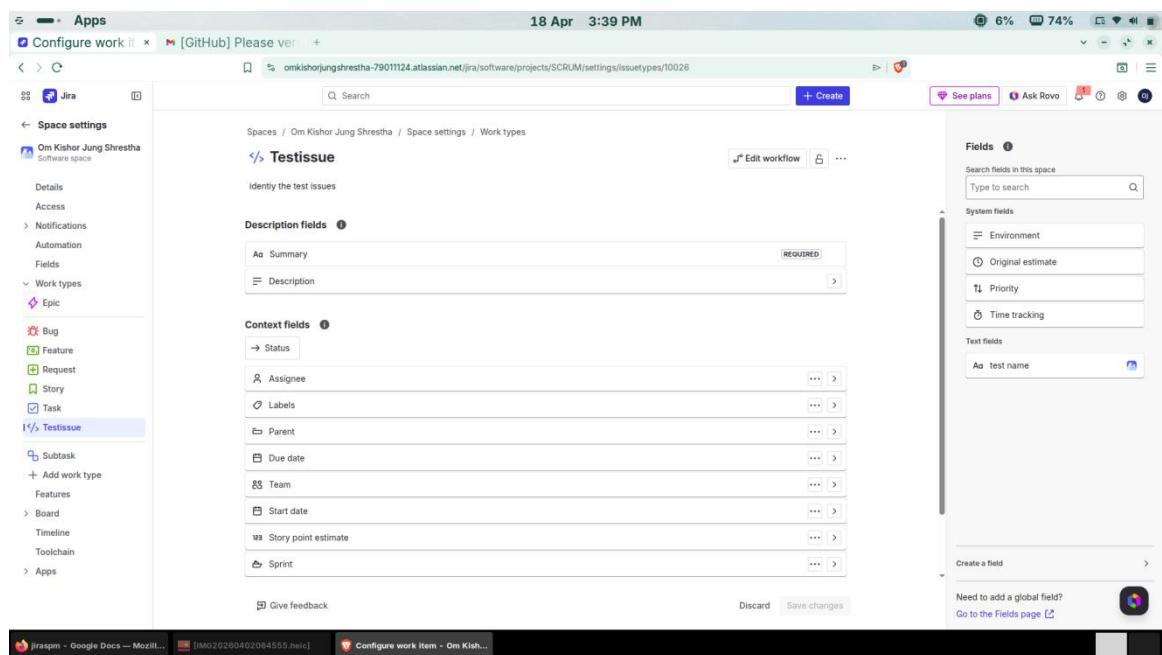
Fields

Add a global field to the table below to use it on work items in this space.

Q Search Field category

Name	Field type	Description	Actions
test name	Short text (plain text only)	name of the test	...
Assignee	Assignee		...
Description	Description		...
Due date	Due date		...

Figure 99: Lab 17 – Burnup chart – completed vs total



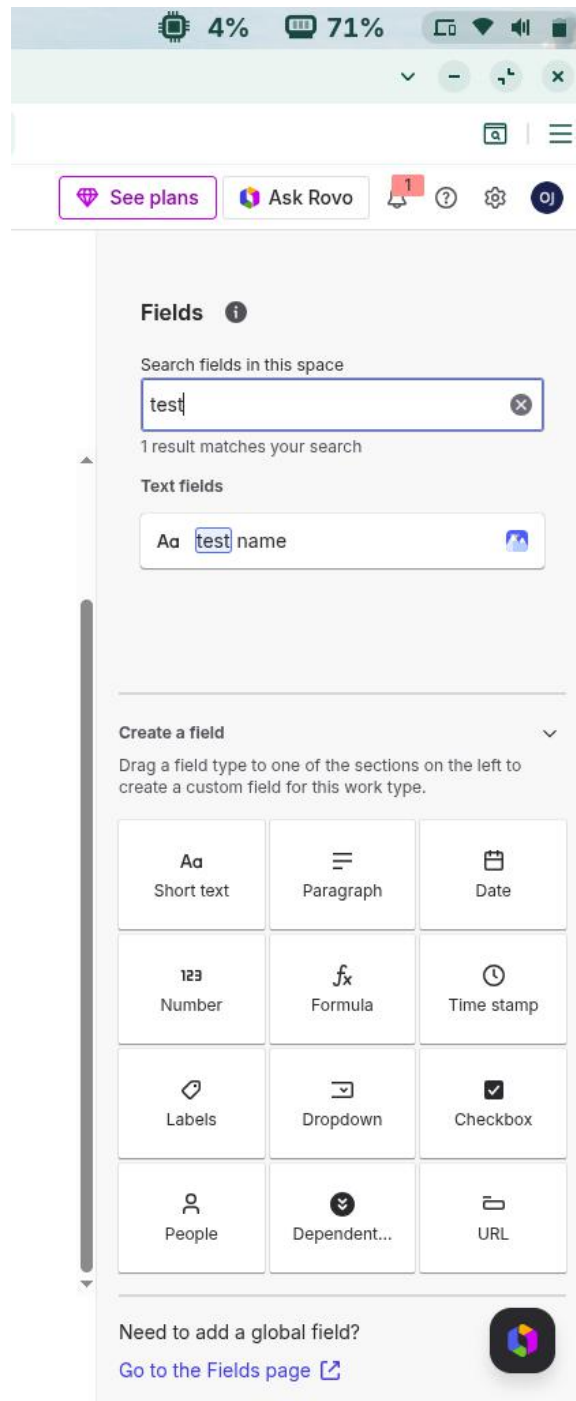


Figure 101:Lab 18 – CFD – bottleneck identified

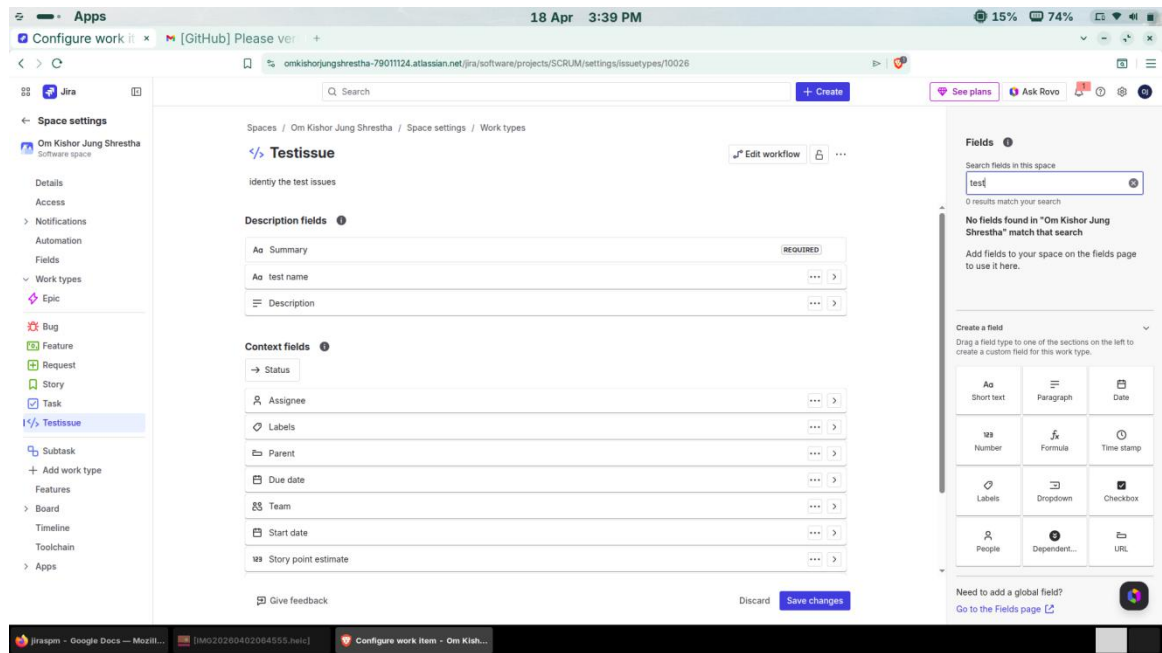


Figure 102field created well

Conclusion: A custom field and a custom workflow status were added, giving more control over the issue lifecycle and data captured on each issue.

Lab 15 – Enabling Features in Project

Objective: To enable additional Jira project features to extend the project's functionality.

Features Enabled

Backlog: Prioritised list of issues separate from the sprint board – toggled on in Project Settings → Features.

Roadmap (Timeline): Gantt-style Epic timeline view for long-term planning.

Releases: Enables "Fix Version" tracking to tag issues to product release versions.

Components: Logical grouping of issues by system layer (e.g., "Backend API", "UI Layer").

Steps Performed

1. Went to Project Settings → Features and reviewed the available toggles.
2. Enabled "Backlog", "Roadmap", "Releases", and "Components".
3. Returned to the sidebar and confirmed new menu items appeared.
4. Created a Release version "v1.0.0 – Beta" and two Components: "Frontend" and "Backend API".

Screenshots

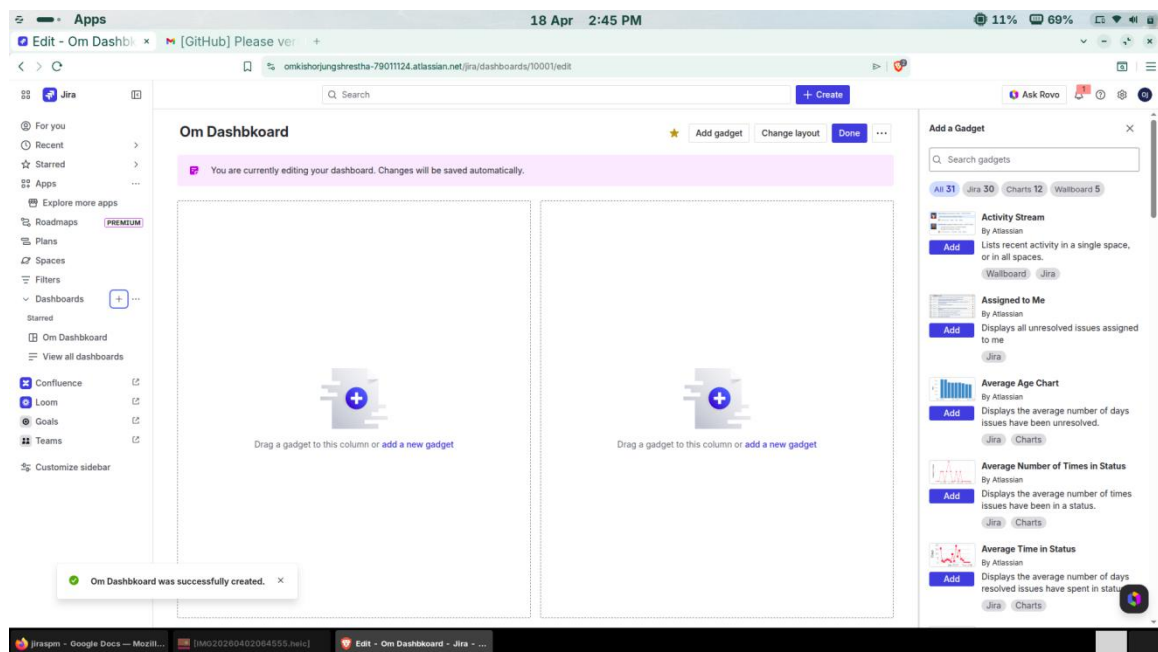


Figure 103:Lab 15 – Project Settings – Features panel

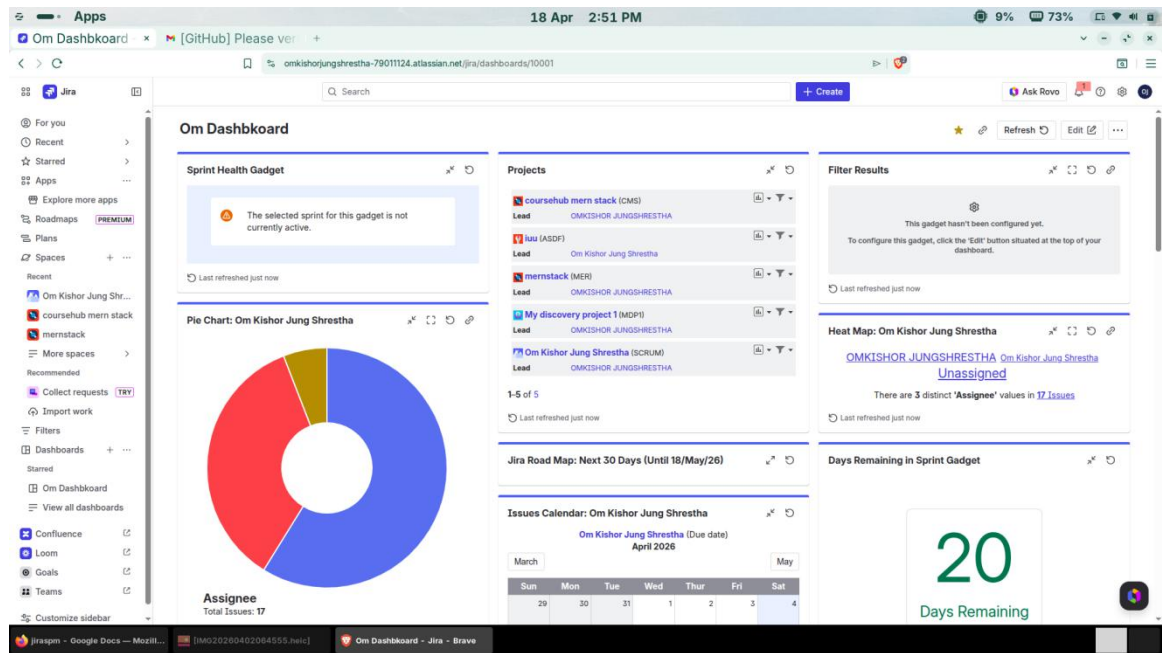


Figure 104:Lab 15 – Features toggles enabled

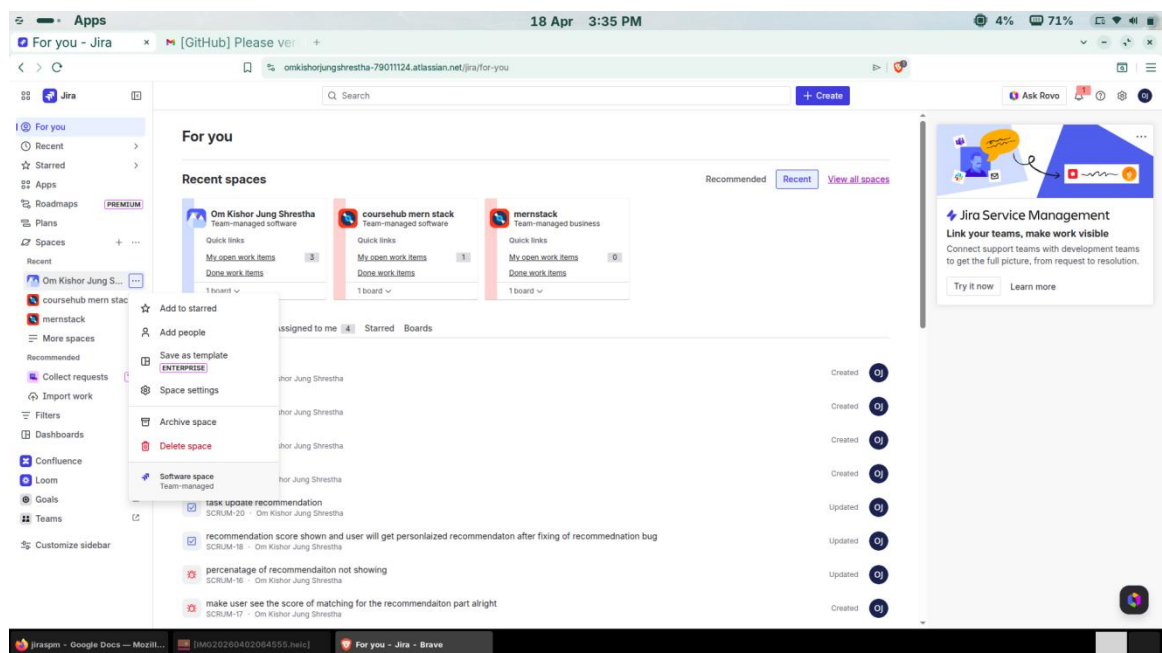


Figure 105:Lab 15 – Releases and Components in sidebar

Figure 106:Lab 15 feautres panel

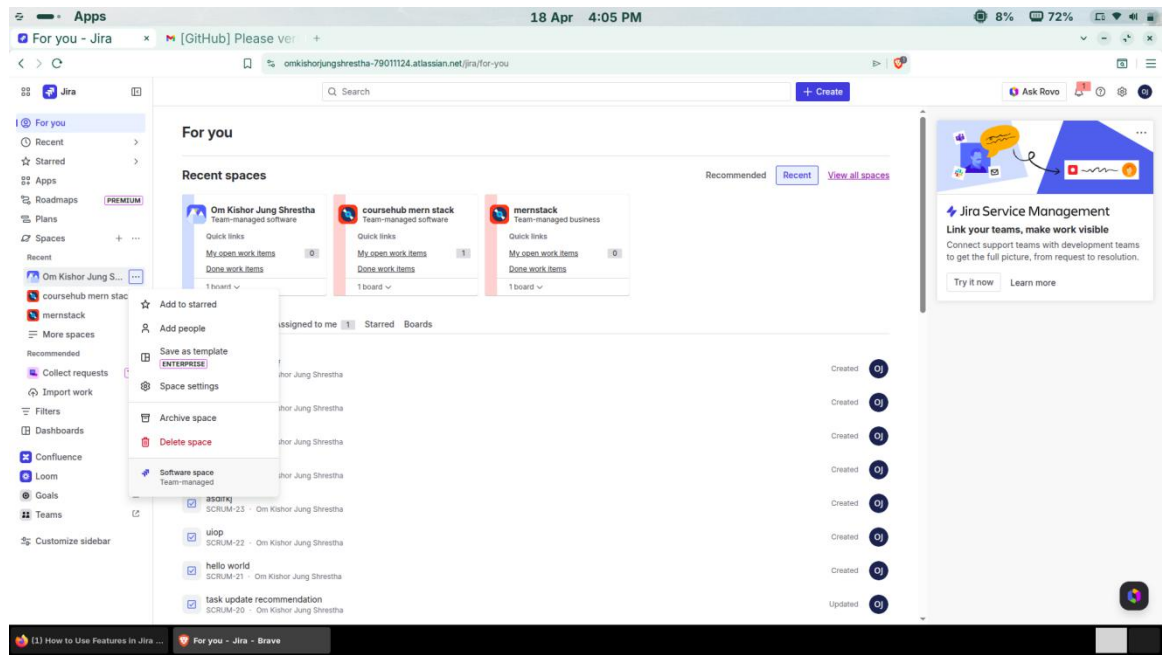


Figure 107:Lab 15 fearutes section

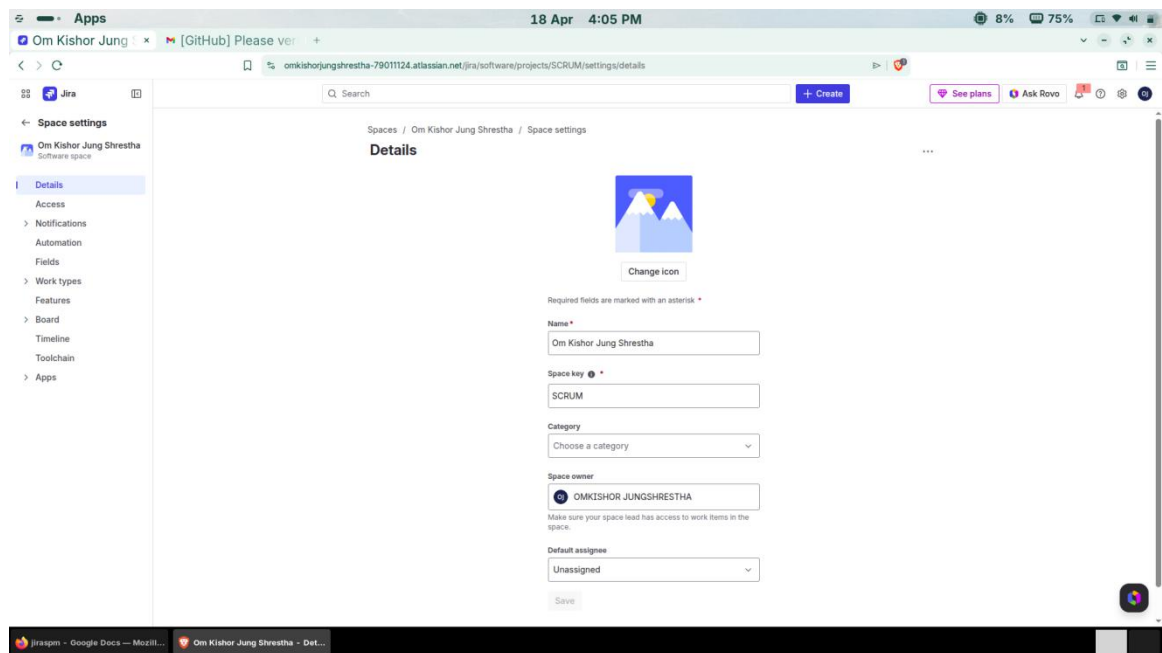


Figure 108:Lab 15 details

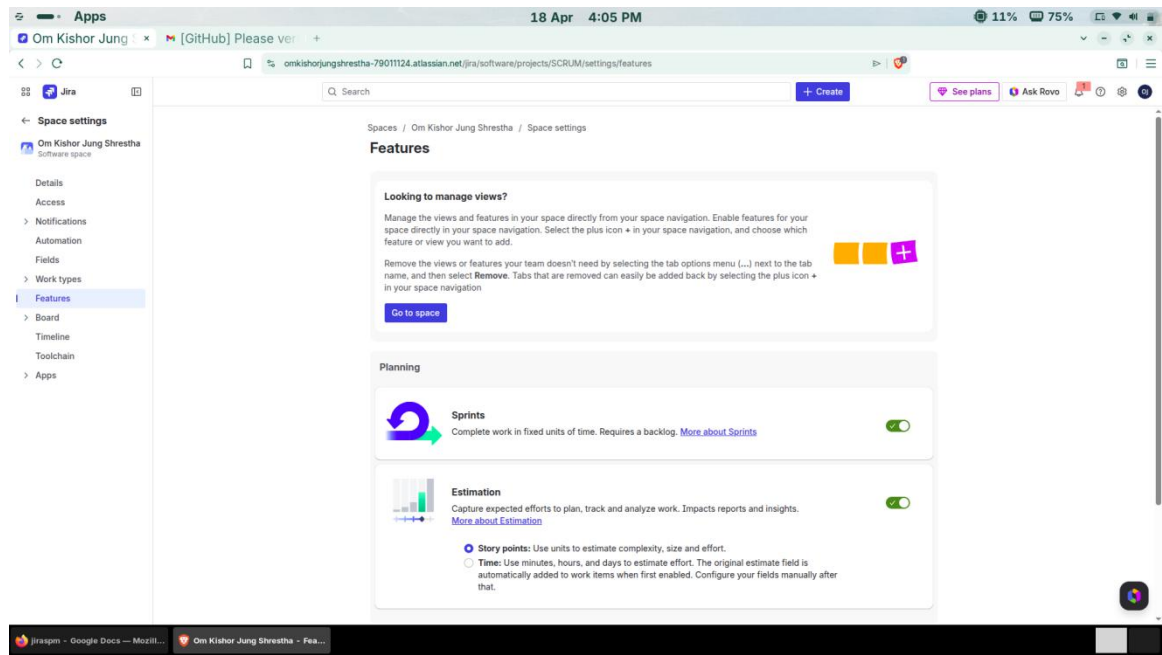


Figure 109: Lab15 end

Conclusion: Key project features were enabled, extending the project's functionality to include releases, components, and the roadmap timeline.

Lab 16 – Reports and Charts in Jira

Objective: To explore the Agile reports available in Jira and understand their purpose.

Jira provides built-in reports accessible from the "Reports" section of a Scrum project. These help teams make data-driven decisions.

Reports Explored

Burndown Chart: Tracks remaining story points vs time in a sprint.

Burnup Chart: Shows completed work against total sprint scope – highlights scope creep.

Sprint Report: Summary of completed and incomplete issues at sprint end.

Velocity Chart: Committed vs completed story points across sprints – used for capacity planning.

Cumulative Flow Diagram: Stacked area chart of issue volumes per status over time – identifies bottlenecks.

Cycle Time Report: Scatter plot of time from In Progress to Done per issue.

Steps Performed

1. Clicked "Reports" in the left sidebar and reviewed all available report types.
2. Opened each report and read its description and axis definitions.
3. Ensured sprint data existed to populate the charts.

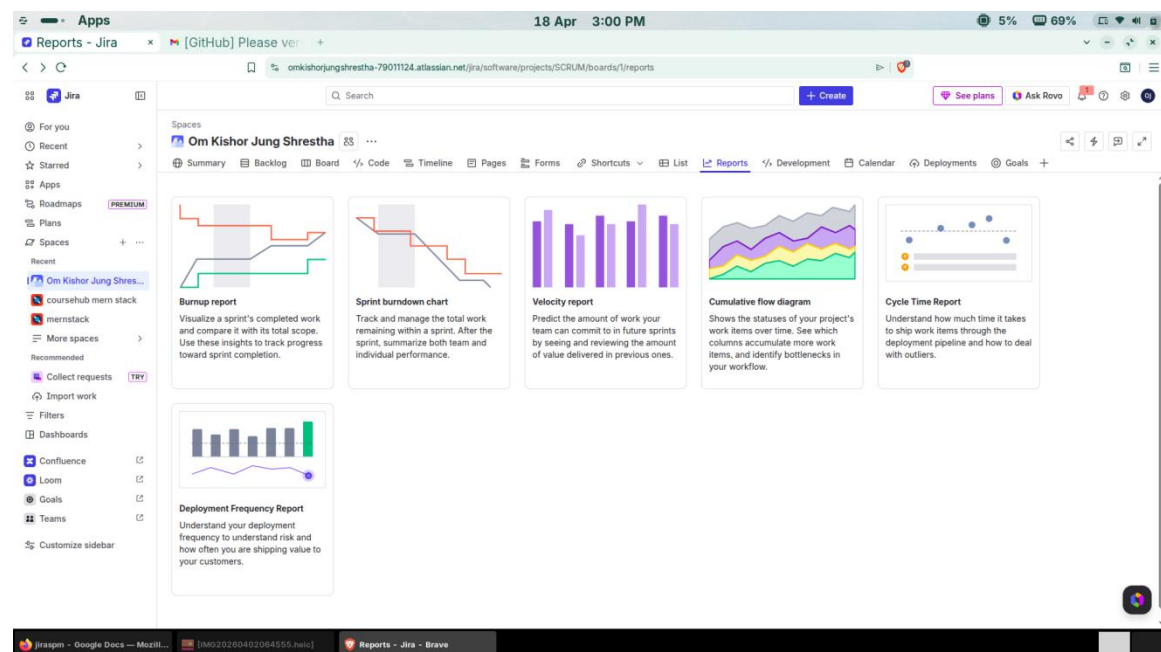


Figure 110: lab 16 report section in jira

Conclusion: All major Jira Agile reports were reviewed. Each serves a distinct purpose in monitoring sprint health, velocity, and flow efficiency.

Lab 17 – Sprint Burndown and Sprint Burnup Chart

Objective: To analyse sprint progress using Burndown and Burnup charts and understand the difference between them.

Sprint Burndown Chart

Shows remaining work (Y-axis: story points) over sprint days (X-axis). The ideal line is a straight diagonal from total points to zero. The actual line reflects real progress.

If the actual line is above the ideal line, the team is behind schedule. If below, ahead of schedule.

How to view: Reports → Burndown Chart → select sprint.

Sprint Burnup Chart

Shows completed work vs total scope. Two lines: Work Completed (rises) and Total Scope (shows scope changes as jumps).

Burnup charts are preferred over Burndown for visualising scope creep.

How to view: Reports → Burnup Chart → select sprint.

Steps Performed

1. Navigated to Reports → Burndown Chart and selected the active sprint.
2. Compared ideal vs actual lines and noted the team's pace.
3. Switched to Burnup Chart and observed scope vs completion lines.
4. Documented observations in sprint retrospective notes.

Screenshots

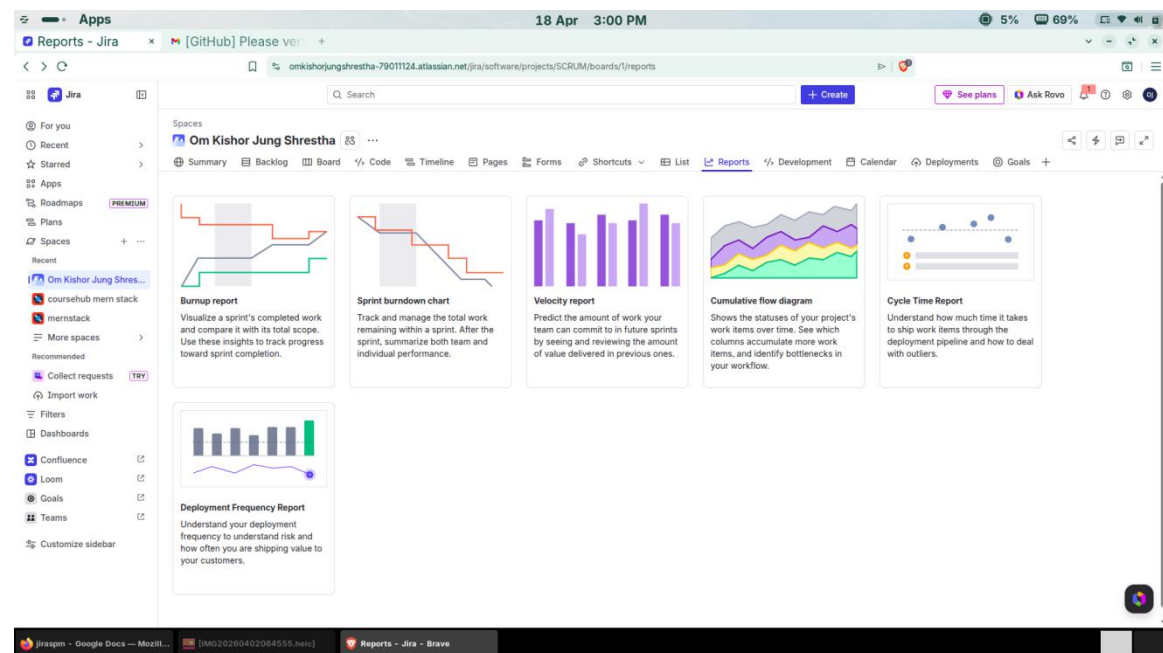


Figure 111: Lab 17 -charts

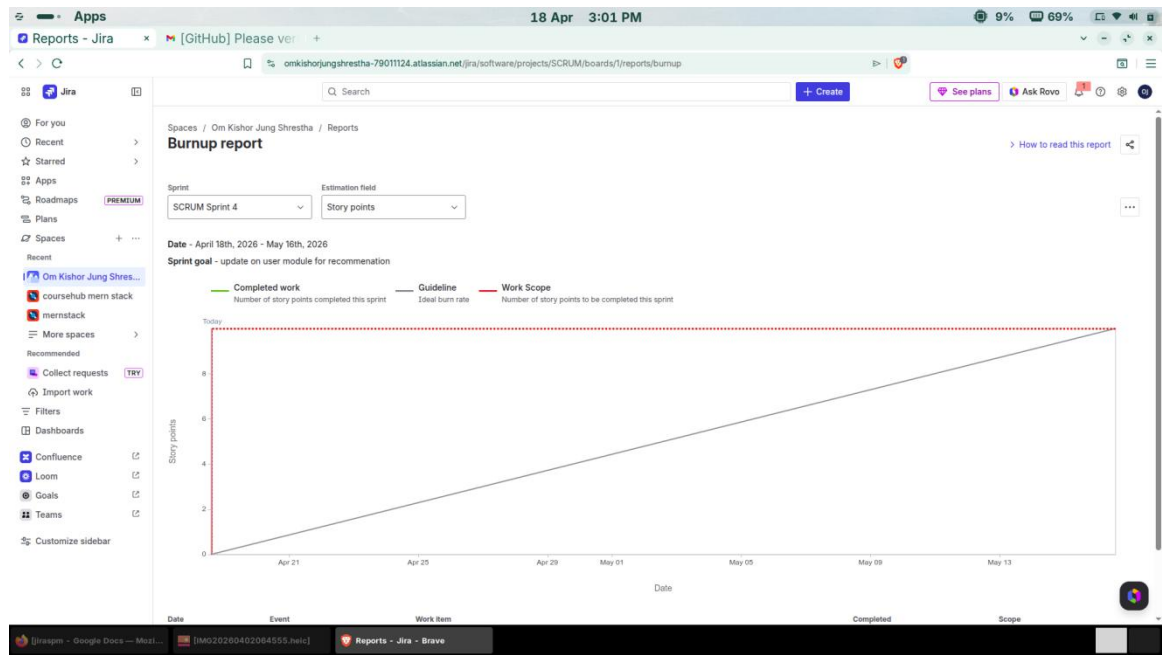


Figure 112 lab 17-burnup chart

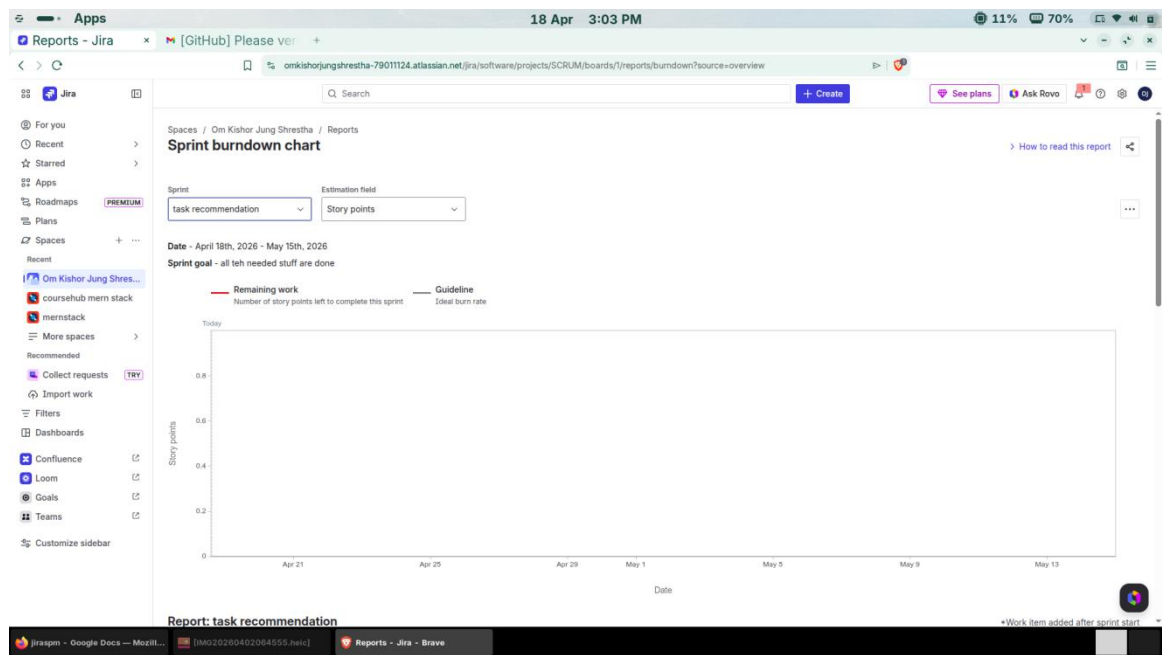


Figure 113: lab 17-burndown chart

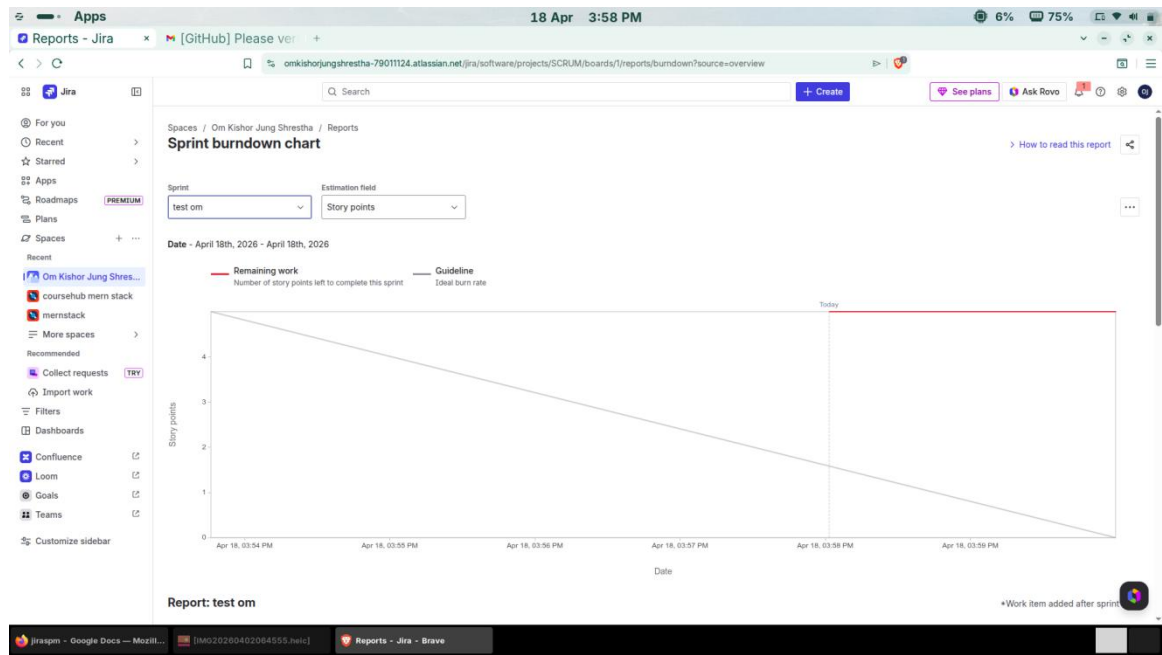


Figure 114 Lab 17- final chart

Conclusion: Burndown and Burnup charts were analysed. The Burndown showed progress pace; the Burnup revealed any scope changes during the sprint.

Lab 18 – Cumulative Flow Diagram and Velocity Chart

Cumulative Flow Diagram (CFD)

Objective: To identify workflow bottlenecks using the CFD.

The CFD is a stacked area chart showing the number of issues in each status (To Do, In Progress, Done) over time. A widening "In Progress" band signals a bottleneck; a steep rise in "Done" indicates high throughput.

How to view: Reports → Cumulative Flow Diagram → set date range.

Velocity Chart

Objective: To use historical velocity data for future sprint capacity planning.

Shows two bars per sprint: Committed (planned points) and Completed (actual points).

Average velocity = sum of completed points / number of sprints.

How to view: Reports → Velocity Chart.

Steps Performed

1. Opened the CFD, set a two-week date range, and identified any widening status bands.
2. Opened the Velocity Chart and calculated average velocity across completed sprints.
3. Used the average velocity to estimate capacity for the next sprint.

Screenshots

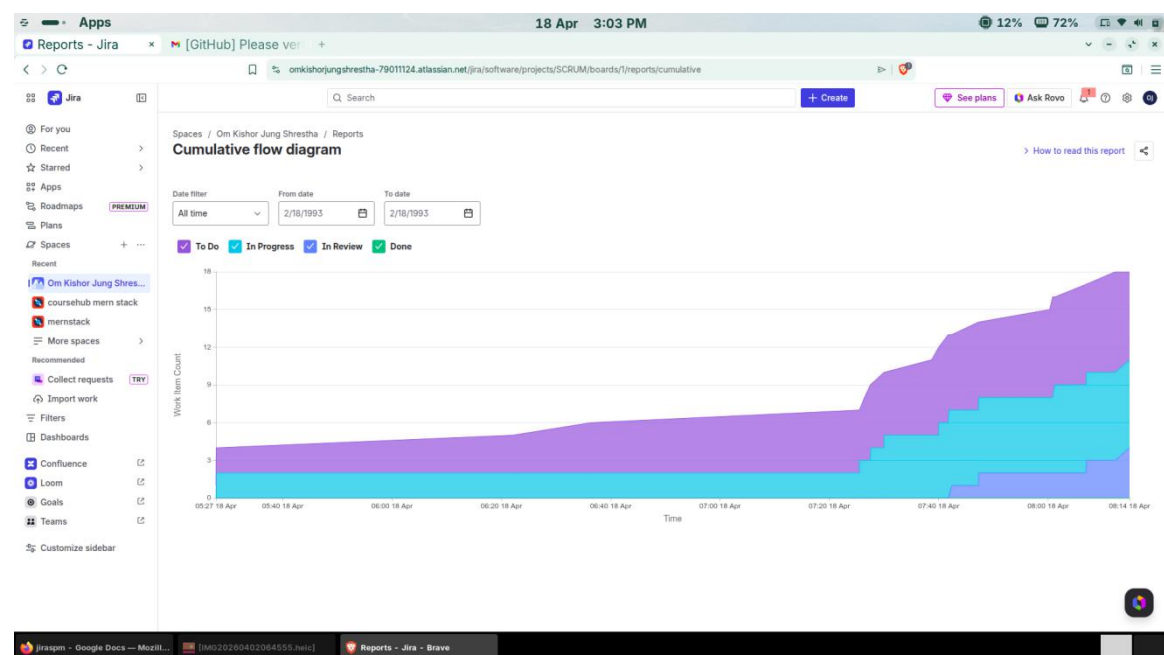


Figure 115:lab 18-cumulative flow diagram

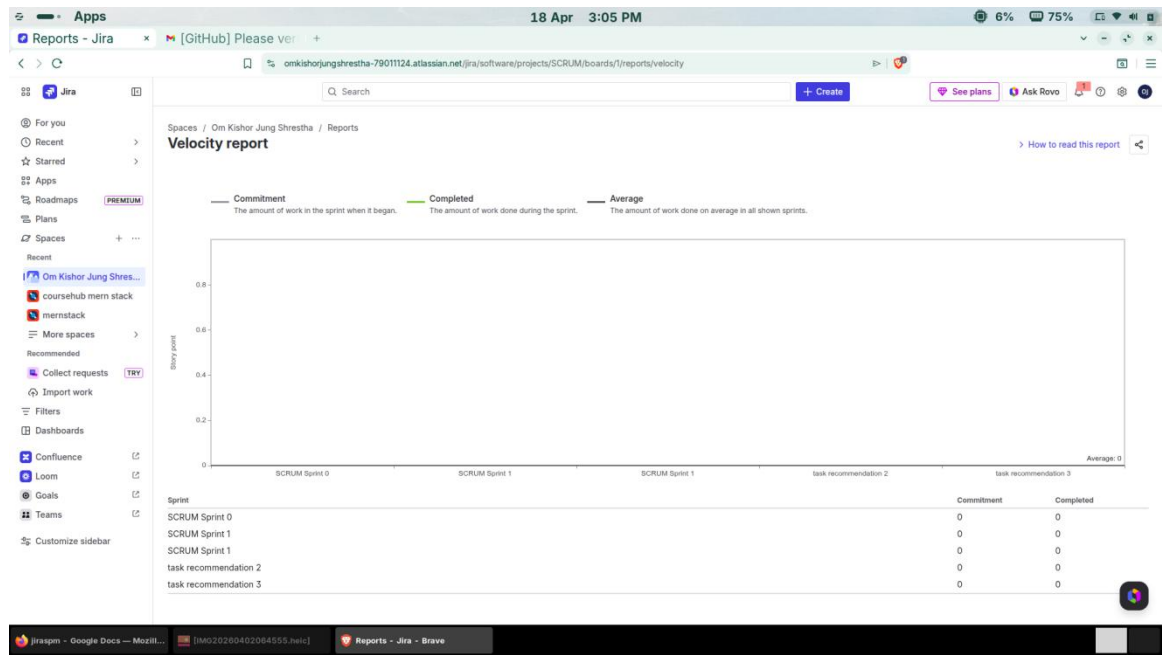


Figure 116: lab 18-velocity report 1

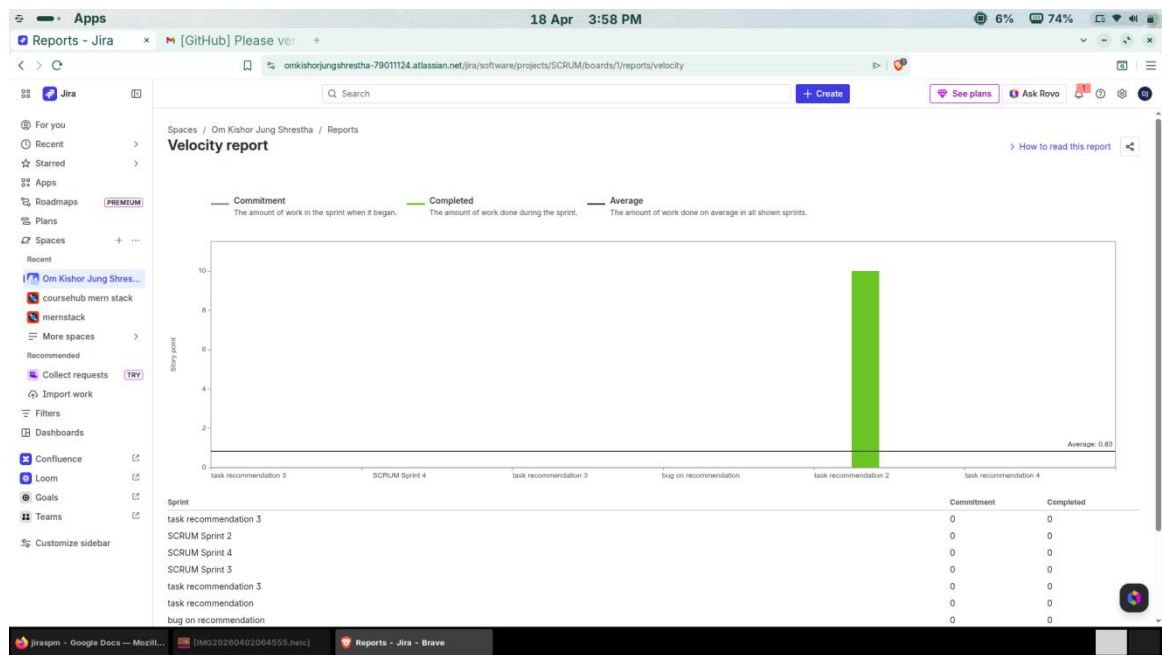


Figure 117: lab 18-velocity report 2

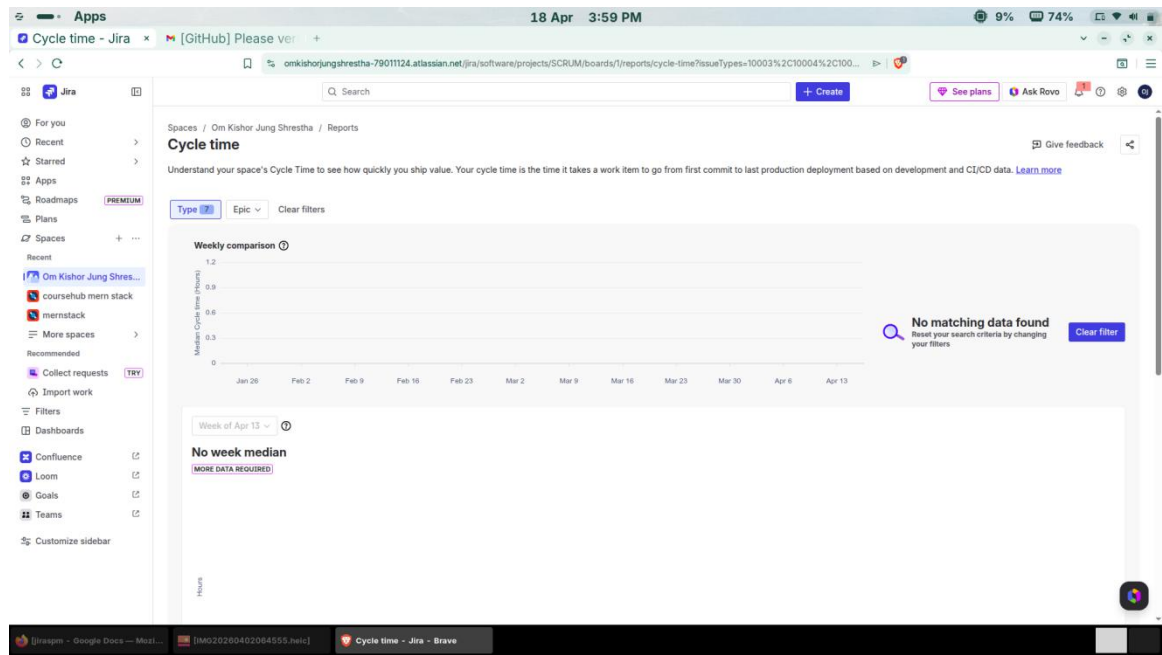


Figure 118: Lab 18-cycle time

Conclusion: The CFD revealed bottlenecks through status band widths; the Velocity Chart provided historical data for sprint capacity forecasting.

Lab 19 – Cycle Time and Deployment Reports

Cycle Time Report

Objective: To measure how long issues take from "In Progress" to "Done" and identify outliers.

The Cycle Time report shows each issue as a dot on a scatter plot. X-axis: completion date; Y-axis: cycle time (days). Percentile lines (50th, 85th, 95th) identify typical vs exceptional cycle times.

How to view: Reports → Cycle Time → select issue type and date range.

Deployment Frequency Report

Objective: To track how often code is deployed to production (a key DORA metric).

When Jira is integrated with CI/CD (e.g., GitHub Actions), deployment events are sent to Jira. The report shows deployments per day/week over a selected period. High frequency indicates mature DevOps practice.

Steps Performed

1. Opened Reports → Cycle Time, filtered by "Story", and identified issues above the 85th percentile.
2. Investigated outlier issues to understand causes of delay.
3. Opened Reports → Deployment Frequency and reviewed the weekly deployment count.

Screenshots

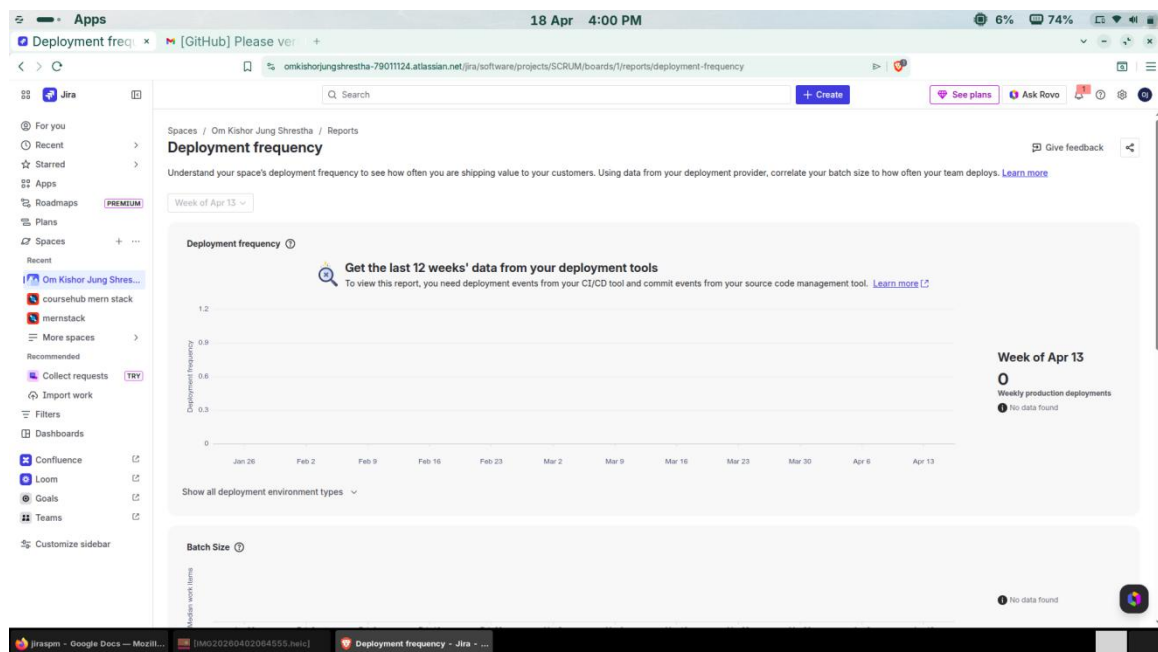


Figure 119: lab 19-deployment frequency

Conclusion: Cycle Time data revealed how long issues spent in development. Deployment Frequency (with CI/CD integration) allows teams to measure and improve release cadence.